GALAXITECH

**DDD Belgium · workshop 2026**

# See the system

Wardley mapping, one click at a time

## How the evening runs

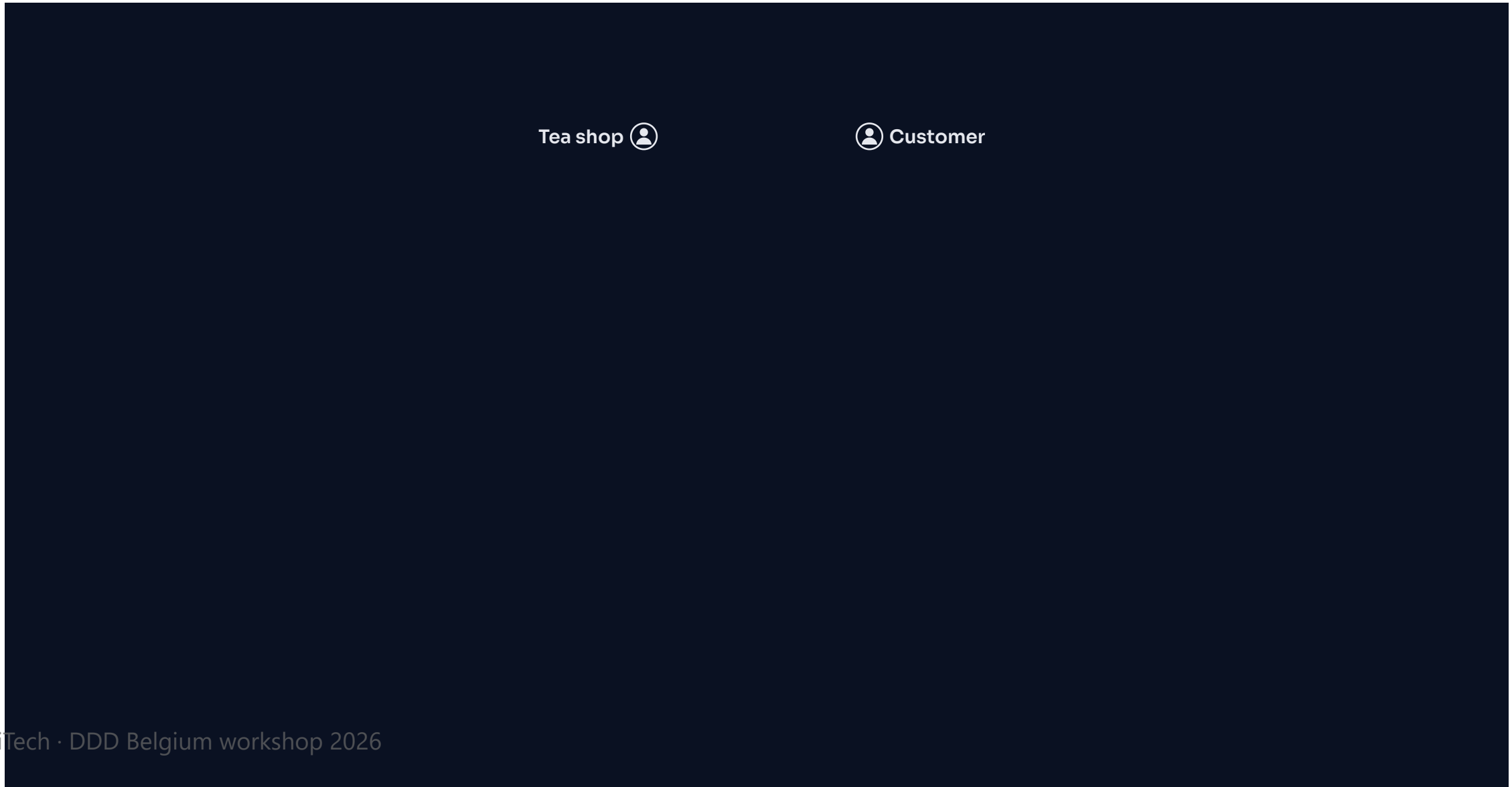
# Talk, group work, discussion

| Talk (5–10 min) | Then, in your group |
|---|---|
| 01 Cup of tea | Exercise 1 · Who needs what? |
| 02 Strategy: how we use the map | Exercise 2 · "Spread it out!" |
| 03 Pipelines | Exercise 3 · Three pipelines |
| 04 Tea shop at rush hour | Exercise 4 · Find the constraint and work it |
| 05 Knowledge architecture | Exercise 5 · Cut it down |
| 06 A real map: EM tooling | Exercise 6 · Make a move |

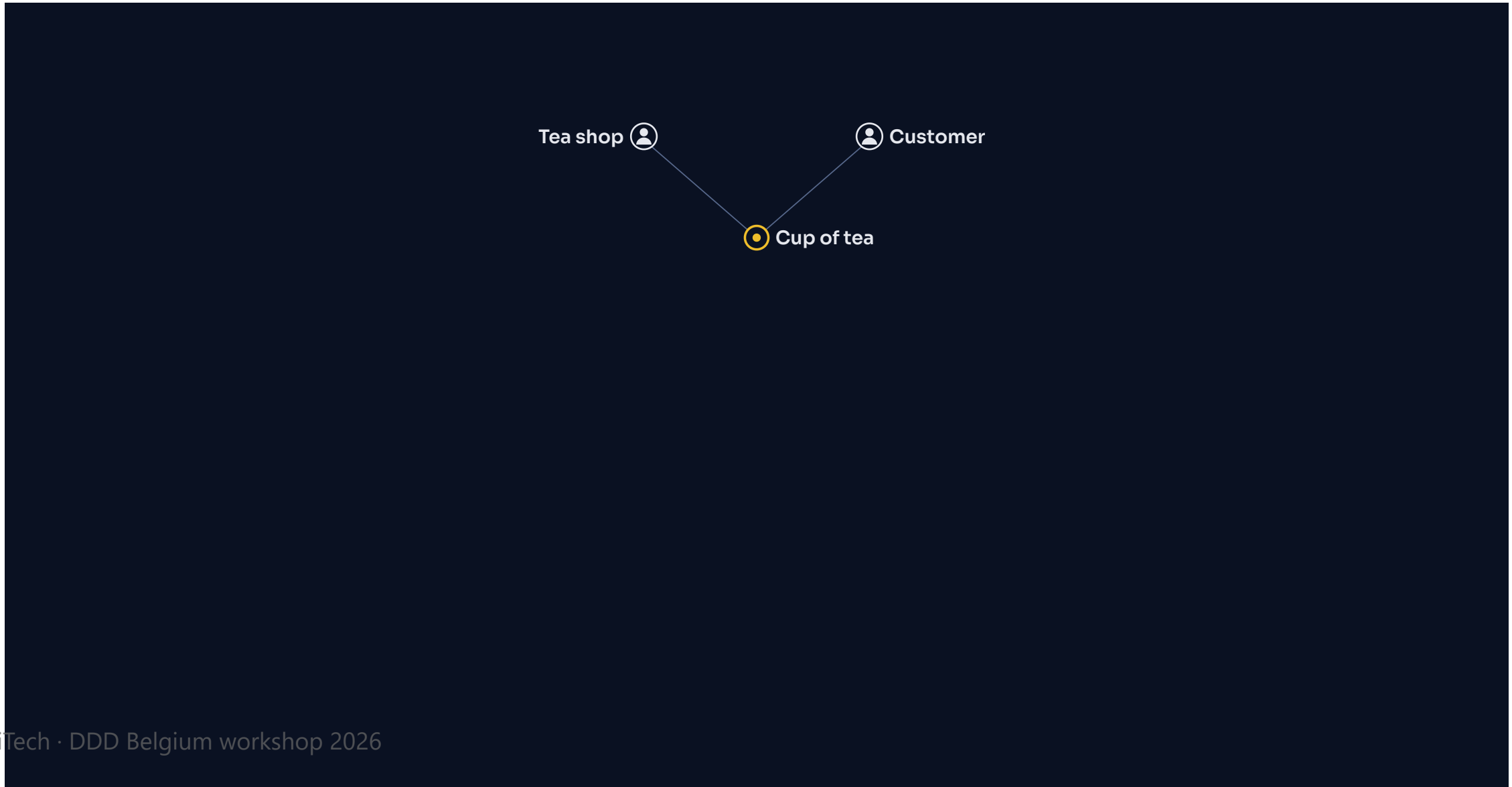
01

# Cup of tea

# Cup of tea · 1 of 12 — the users



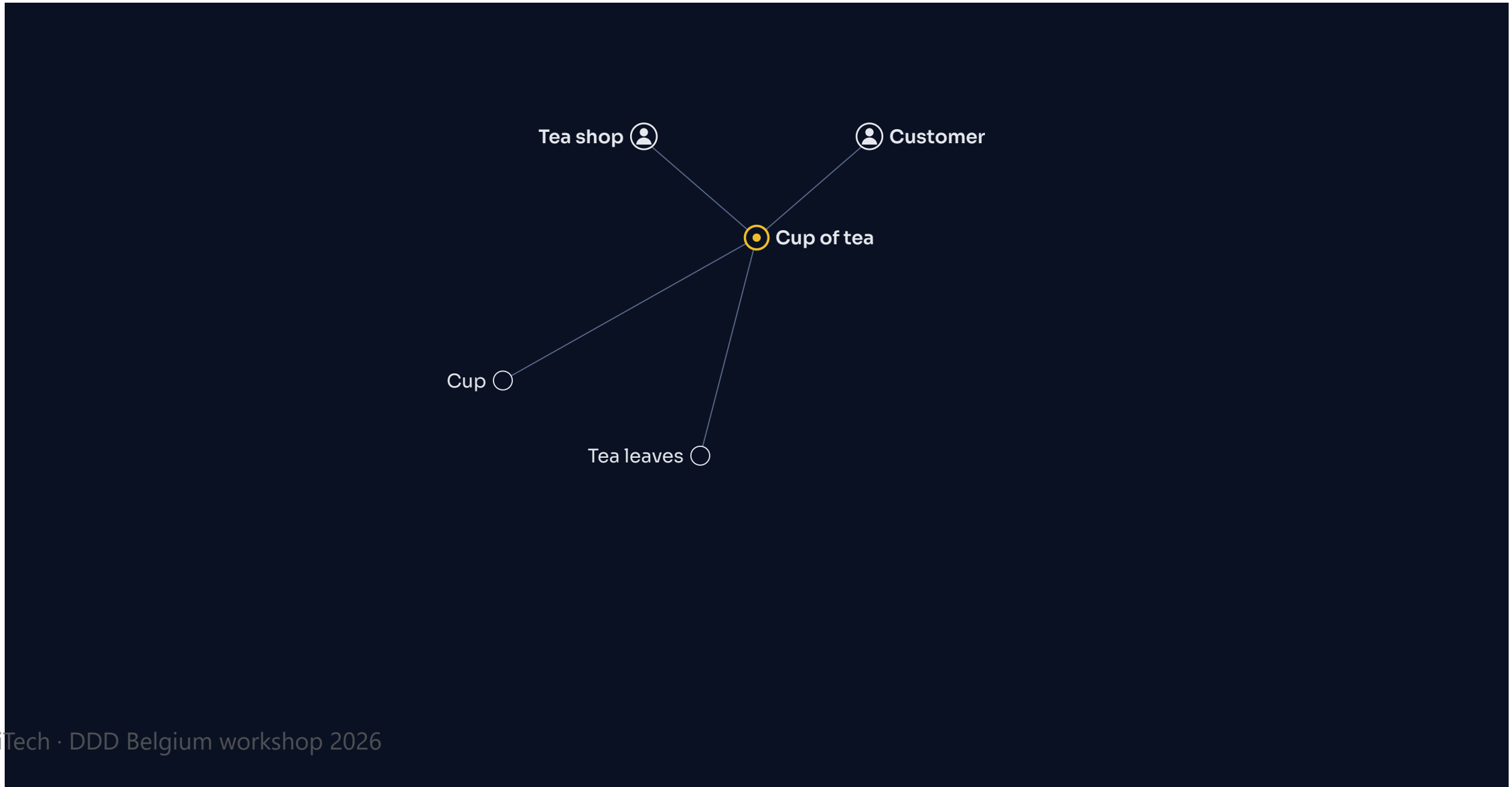
# Cup of tea · 2 of 12 — the need



# Cup of tea · 3 of 12 — the cup



# Cup of tea · 4 of 12 — tea leaves



# Cup of tea · 5 of 12 — hot water



# Cup of tea · 6 of 12 — the kettle



# Cup of tea · 7 of 12 — cold water



# Cup of tea · 8 of 12 — the tap



# Cup of tea · 9 of 12 — power



# Cup of tea · 10 of 12 — the vertical axis



# Cup of tea · 11 of 12 — problem domain



# Cup of tea · 12 of 12 — solution domain



# Reading the value chain

- Upper components are visible to the user. Lower components are hidden from the user
- Read down to ask how: how is this need met?
- Read up to ask why: why do we need this?
- Each link is a dependency and an exchange: the component above pays money or effort, and gets a capability back
- So every component has a cost, and provides value to the one above it
- A map is a chain of needs

# What goes on a map

- Activities: what we do. Making a cup of tea
- Practices: how we do it. Hand brewing
- Data: facts we keep. The menu and the prices
- Knowledge: what we understand. How strong tea should be
- Things we use: the cup, the kettle

# Mapping · the Wardley Mapping Canvas, by Ben Mosior

## Wardley Mapping Canvas

### 1. Purpose

What is your purpose? Why does this organization or project exist?

### 2. Scope

What is it that you are mapping? What does it include? What does it *not* include?

### 3. Users

Who uses or interacts with the thing you are mapping?

### 4. User Needs

Copy your users over. What do they need from you? What is each user's journey?

### 5. Value Chain

Copy the users over, with their needs underneath. What sorts of things do you need to be doing to fulfill those needs? Arrange them according to dependence — create a value chain.




### 6. Map

Copy the value chain over. Use the evolutionary characteristics to decide where to place each component along the horizontal axis (Evolution).



# Exercise 1: six steps to a map

- 1 Purpose: why are we mapping?
- 2 Scope: what is in, what is out?
- 3 Users: who is it for?
- 4 User needs: what do they need?
- 5 Value chain: what do the needs depend on?
- 6 Map: place it all on the evolution axis (exercise 2)
- On the printed canvas, or sticky notes on a table or wall

The exercises · one scenario all evening

# Exercise 1: Harbour Insurance

- Imagine you are a product team at Harbour Insurance, a fictional company
- Every team now has AI coding agents. Code arrives in hours, not days
- Yet customers get new features no faster than before
- Management asks: we pay for AI coding agents, but we don't see faster delivery. What is happening?
- Your domain expert knows more about the company. Ask them
- Your task: map this situation on the canvas, steps 1 to 5

# Exercise 1: the company and its purpose

- Harbour Insurance insures ships, cargo and the businesses around the port of Antwerp
- About 2,000 employees and 300,000 policies. Quotes, policies and claims all run through its own software
- The product and engineering organisation: 6 product teams, about 40 developers, one architect
- Its purpose: get new insurance products and changes to customers quickly and safely
- Its scope for today: from a product hypothesis to customers using the change

Your group

# Exercise 1: four people, four roles

- Domain expert: has the final word and decides
- Facilitator: starts the talk, makes sure all are heard
- Timekeeper: keeps time and keeps you moving
- Clerk: materials, the canvas, runs to CJ with questions
- Feel free to swap roles after each exercise

Exercise 1 · group work

# Exercise 1: who needs what?

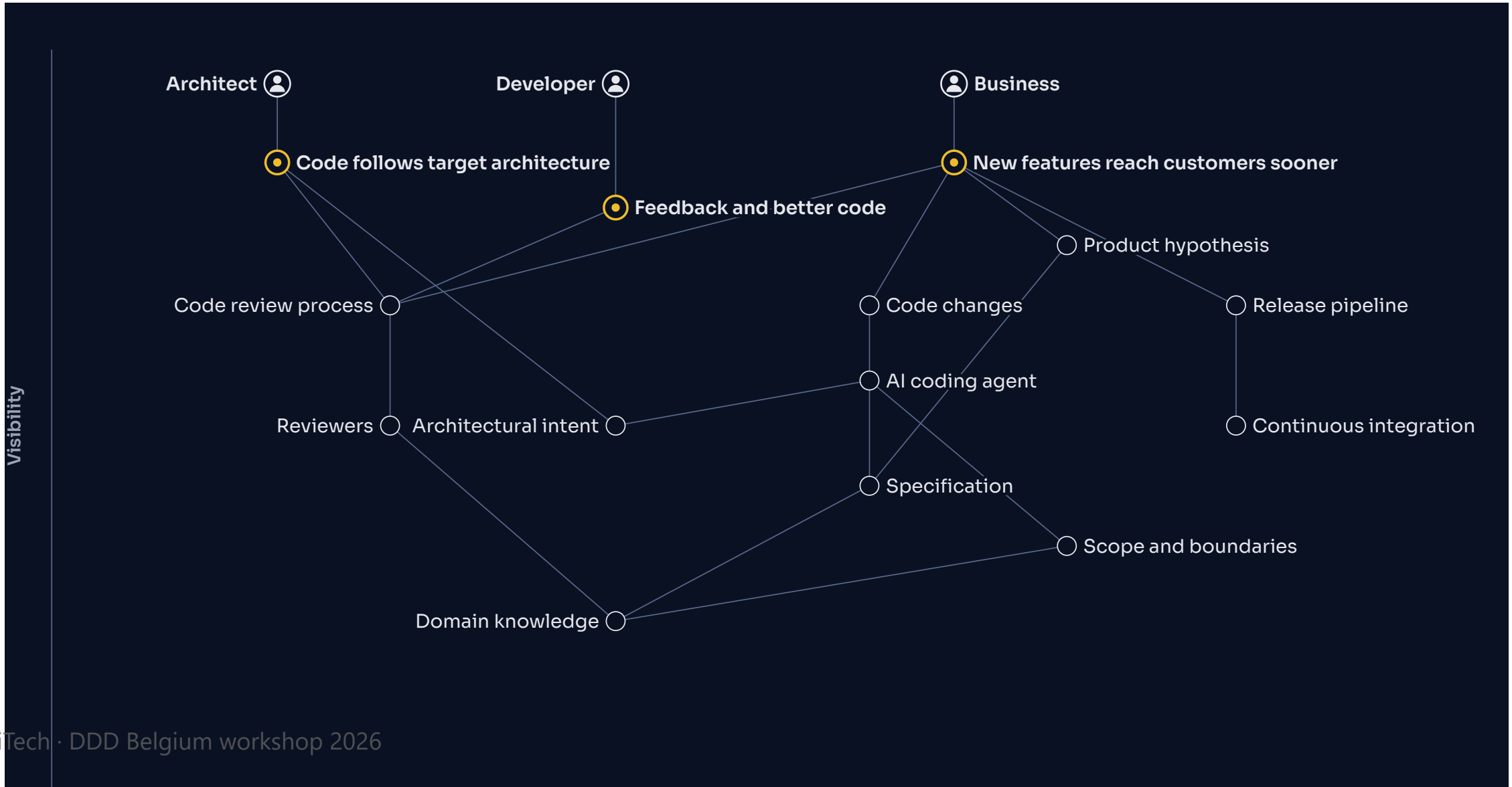
- Follow the canvas, steps 1 to 3
- Write the purpose and the scope first
- Then identify the users
- Under each user, write their needs on stickies
- **Stop here** - value chain comes in Exercise 2

Exercise 1 · back together

# Exercise 1: discuss your findings

- How many different users did you identify?
- Which user has the most needs? The fewest?
- Are any needs shared across multiple users?
- Next: we'll pick ONE anchor to build our map from

# Exercise 1 · one example

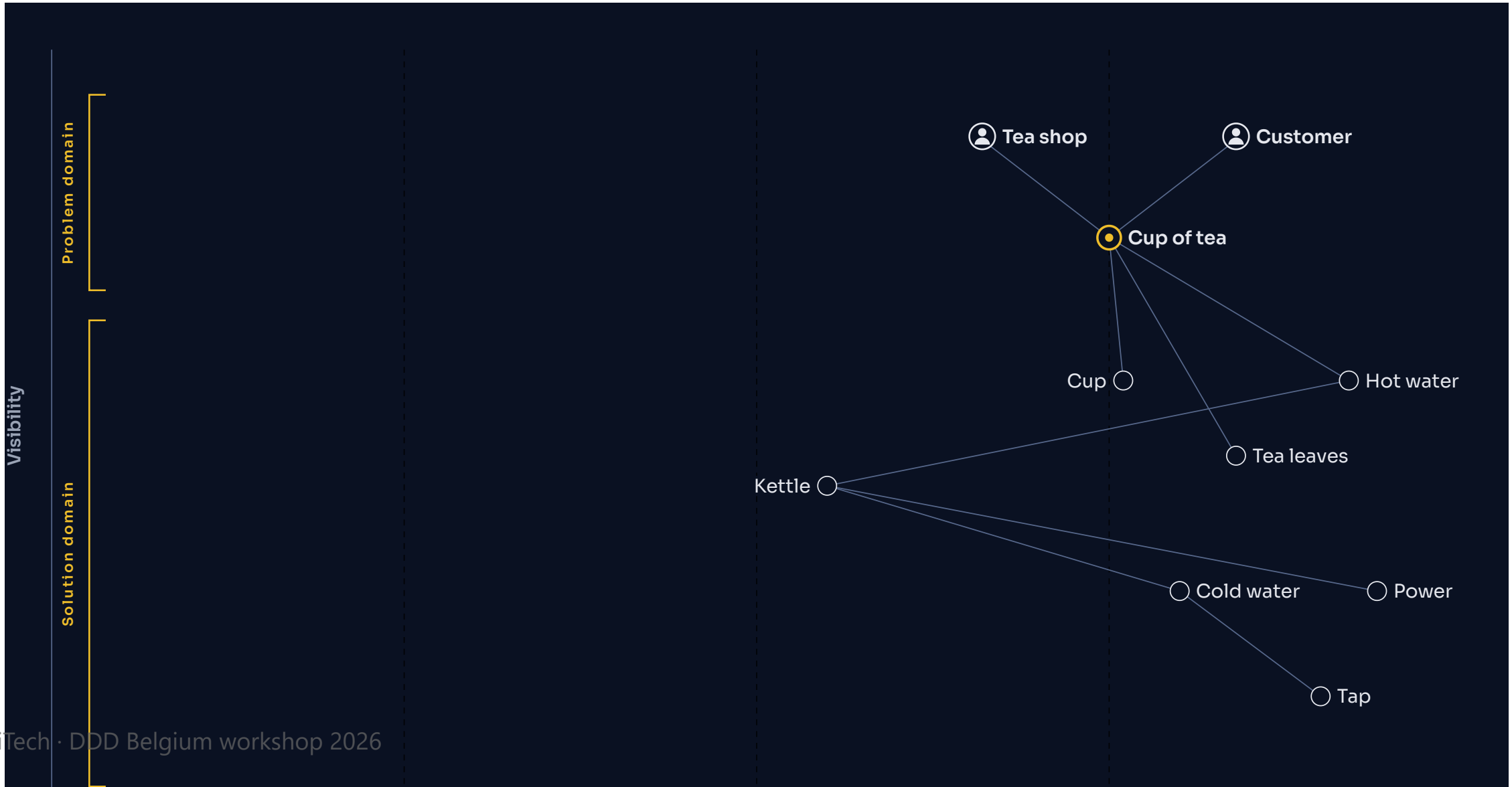


Before Exercise 2 · facilitation

# Pick your anchor

- You've identified multiple users - great!
- To build **one map**, we need **one anchor**
- Ask: **what question is this map answering?**
- For Harbour Insurance: "Why is delivery slow?"
- Pick the user whose need matters for that question
- Suggestion: Management + "Time to market"
- The other needs become **components** in your chain

# Cup of tea · the map wakes up: evolution



Evolution · labels after Simon Wardley

# Every kind of component evolves

| Activities | Genesis | Custom built | Product (+ rental) | Commodity (+ utility) |
|---|---|---|---|---|
| Practices | Novel | Emerging | Good | Best |
| Data | Unmodelled | Divergent | Convergent | Modelled |
| Knowledge | Concept | Hypothesis | Theory | Accepted |

Simon Wardley’s names for the four stages of each kind. Knowledge moves from poorly understood to well understood.

Evolution · knowledge

# Circles on circles

Seen from the Earth, Mars draws loops. The models kept adding circles and still could not predict it.



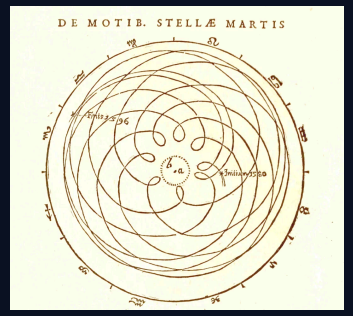


Evolution · knowledge



# Kepler removed the accidental complexity

# Evolution · knowledge evolves too




Solar system model



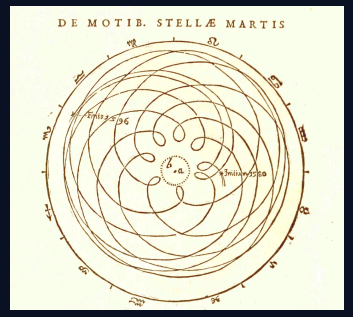
$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity

# Evolution · knowledge evolves too · 1 of 4 — uncharted




Solar system model



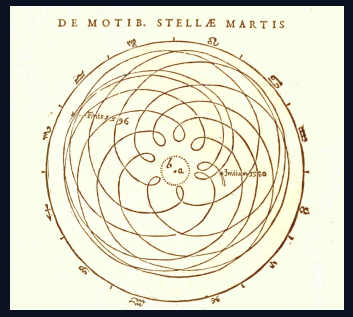
$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity

# Evolution · knowledge evolves too · 2 of 4 — emerging




Solar system model



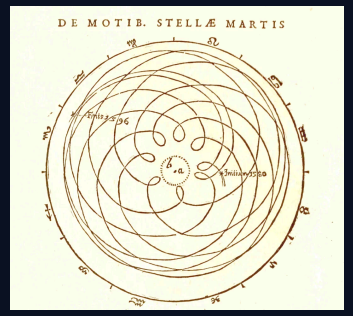
$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity

# Evolution · knowledge evolves too · 3 of 4 — established




Solar system model



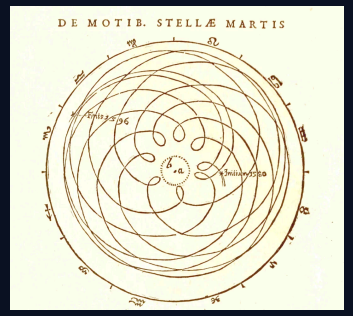
$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity

# Evolution · knowledge evolves too · 4 of 4 — well known




Solar system model



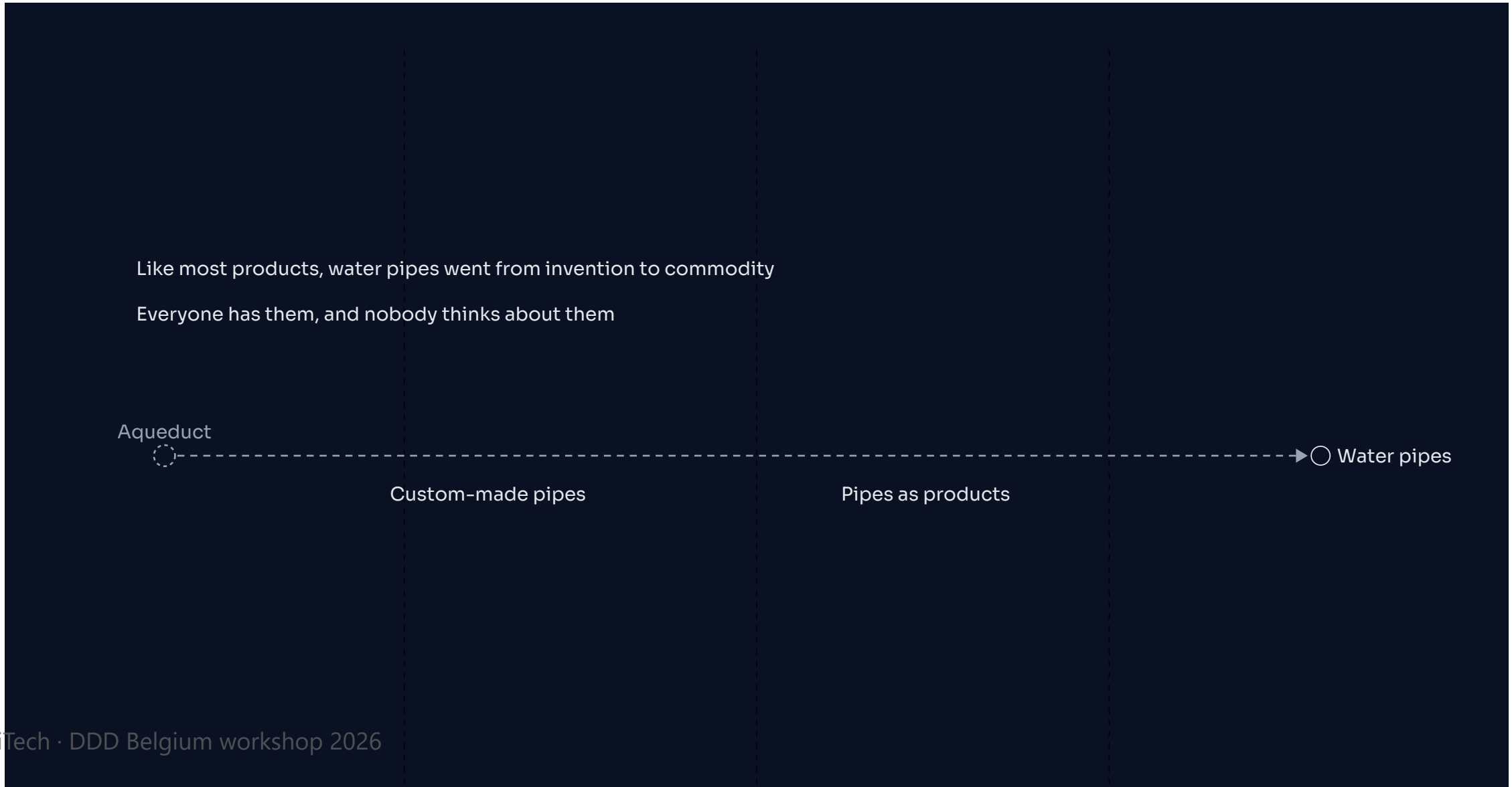
$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity

# Climate · everything evolves



02

# Strategy: how we use the map



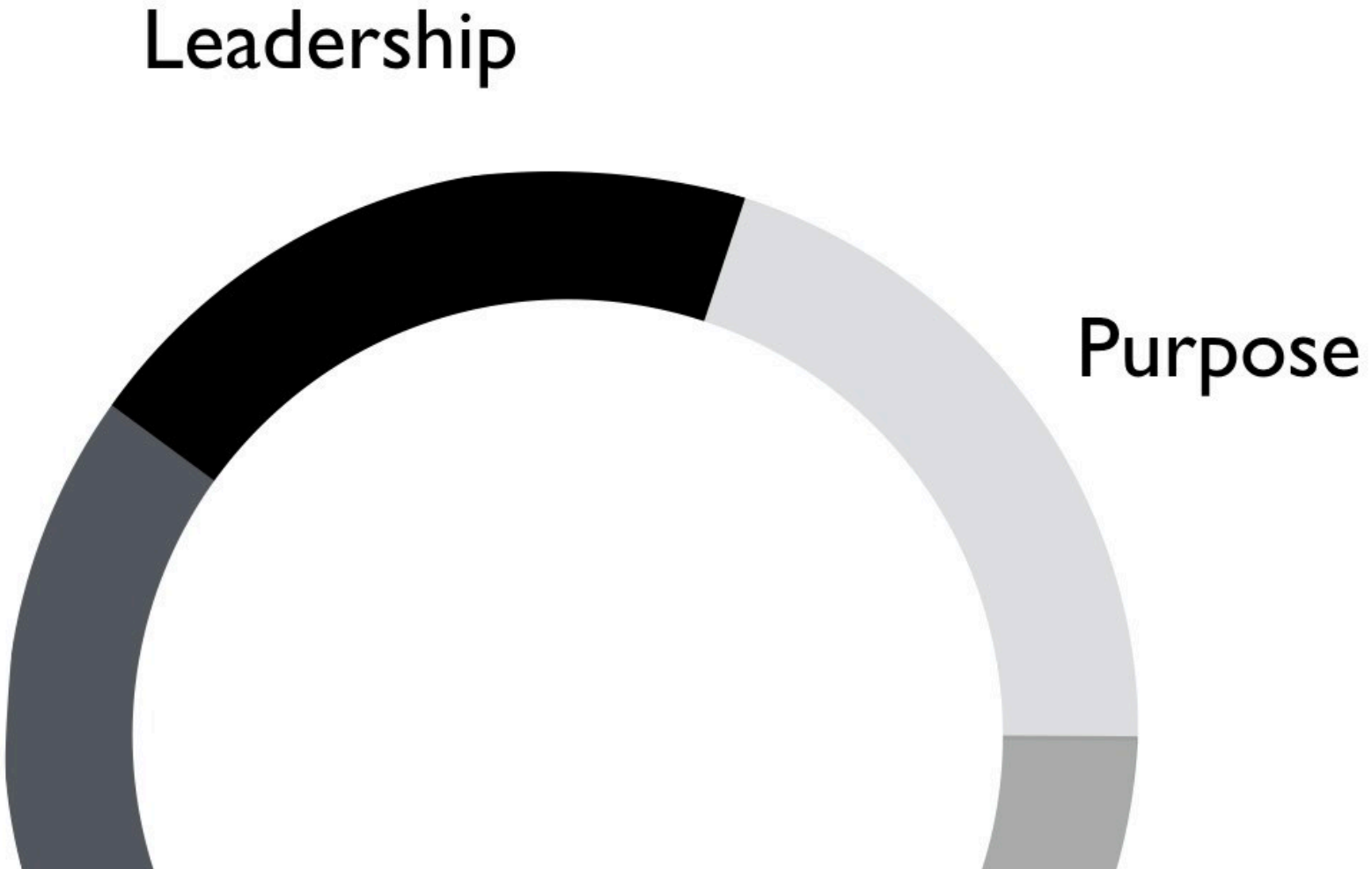
Photo: Martin Makaryan, Unsplash

Sun Tzu

# Sun Tzu's five factors

- Purpose — why we compete
- Landscape — where we stand
- Climate — what changes around us
- Doctrine — training and standard ways of operating
- Leadership — context-dependent strategic decisions

## Sun Tzu · the five factors as Simon Wardley draws them, figure 1, CC BY-SA 4.0

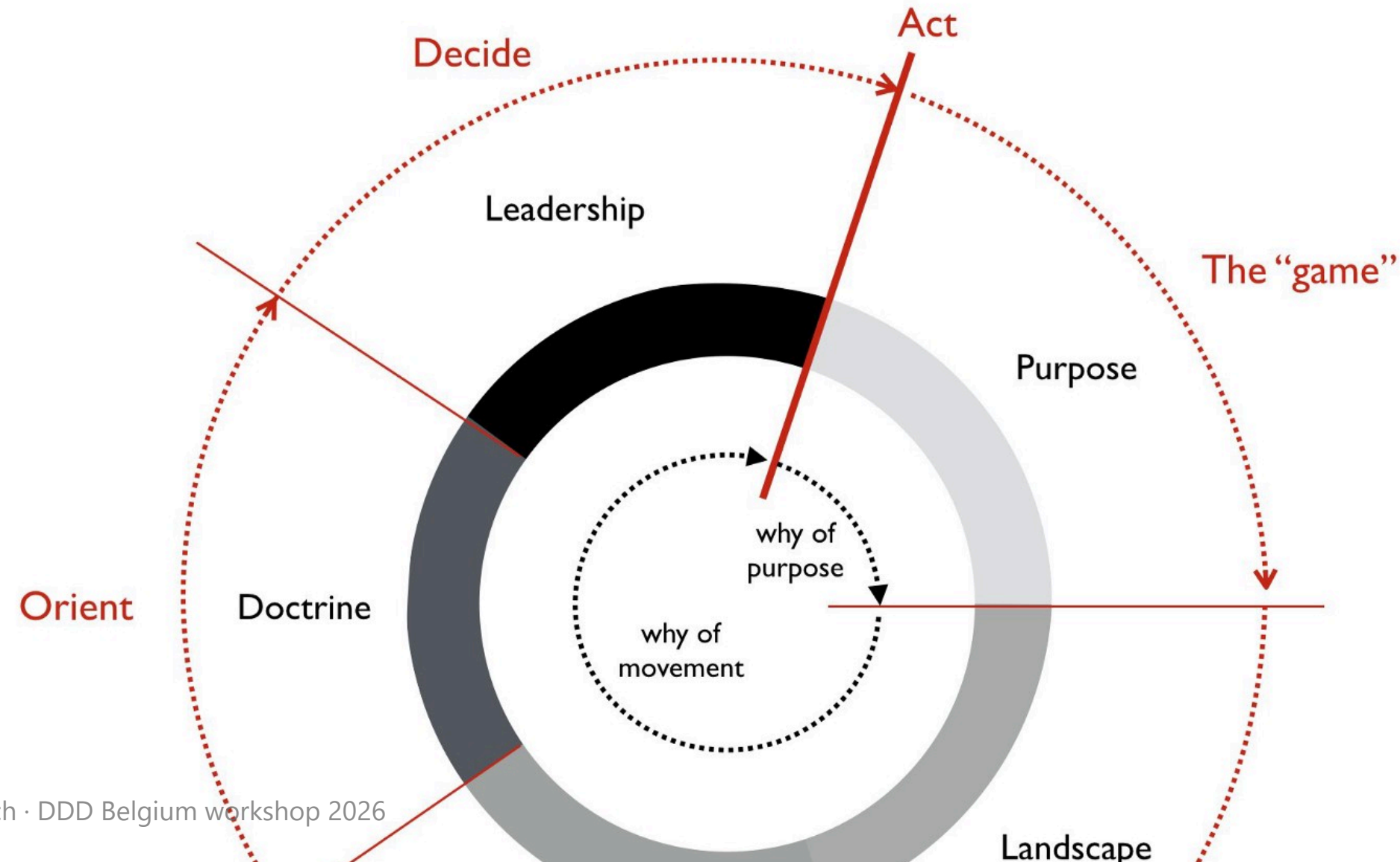


John Boyd · the OODA loop

# John Boyd's OODA loop: observe, orient, decide, act

- Observe: look at the landscape and at what is changing around it, the climate
- Orient: make sense of it with the principles you trust, the doctrine
- Decide: choose the move, which is leadership
- Act: make the move, then observe again, because the landscape has changed
- Boyd was a fighter pilot. The side that goes round the loop faster wins, even with less
- Wardley puts the five factors on this loop. That is his strategy cycle, next slide

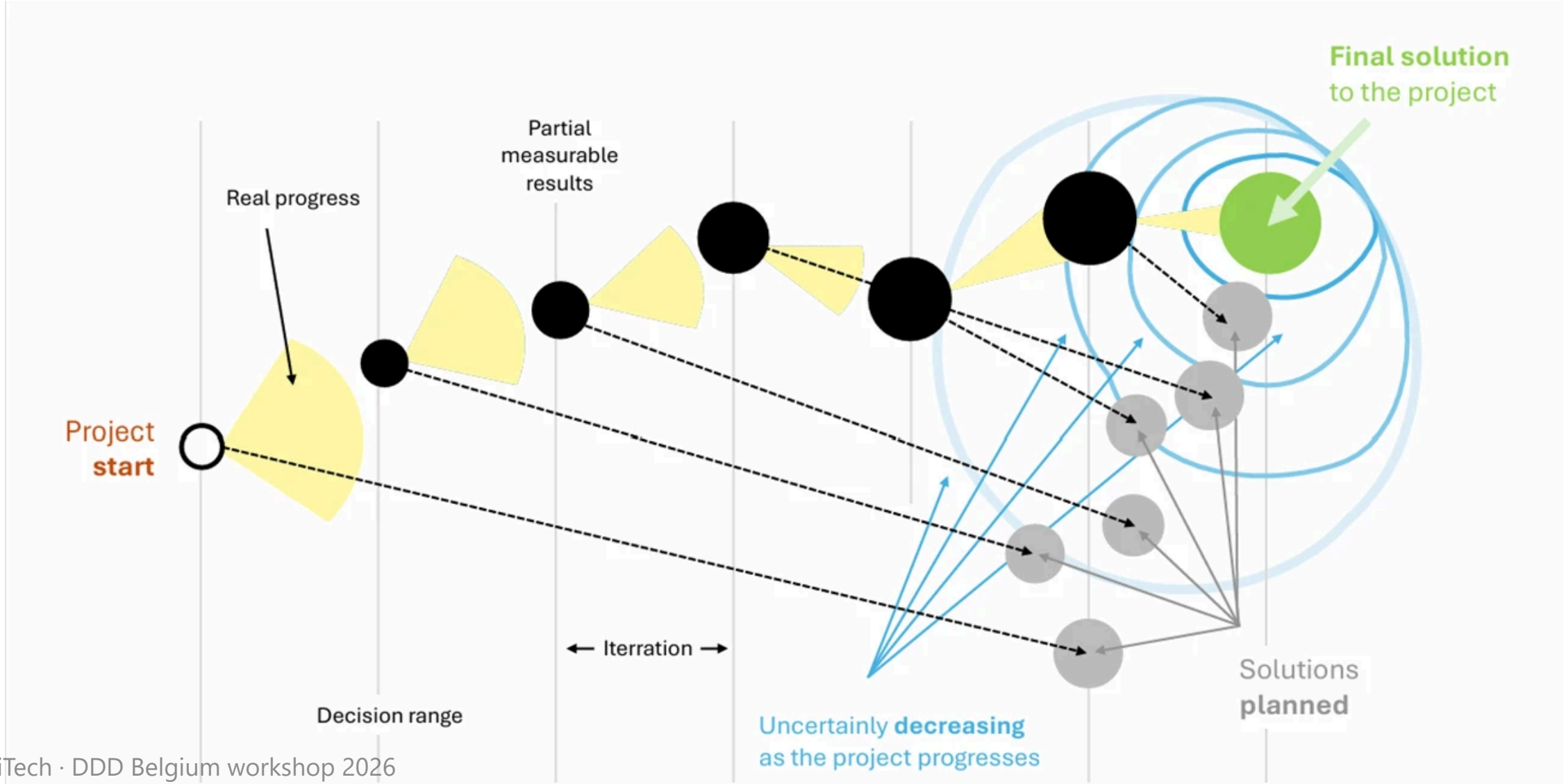
# The strategy cycle · Simon Wardley's figure 7, CC BY-SA 4.0



# Doctrine · Simon Wardley's table, figure 236 of his book, CC BY-SA 4.0

| Category | Wardley's Doctrine (universally useful patterns that a user can apply) | | | |
|---|---|---|---|---|
| *Communication* | Be transparent *(a bias towards open)* | Focus on high situational awareness *(understand what is being considered)* | Use a common language *(necessary for collaboration)* | Challenge assumptions *(speak up and question)* |
| *Development* | Know your users *(e.g. customers, shareholders, regulators, staff)* | Focus on user needs | Think fast, inexpensive, restrained and elegant (FIRE, formerly FIST) | Remove bias and duplication |
| | Use appropriate methods *(e.g. agile vs lean vs six sigma)* | Focus on the outcome not a contract *(e.g. worth based development)* | Be pragmatic *(it doesn't matter if the cat is black or white as long as it catches mice)* | Use standards where appropriate |
| | Use appropriate tools *(e.g. mapping, financial models)* | | | |
| *Operation* | Manage inertia *(e.g. existing practice, political capital, previous investment)* | Optimise flow *(remove bottlenecks)* | Think small *(as in know the details)* | Effectiveness over efficiency |
| | Do better with less *(continual improvement)* | Set exceptional standards *(great is just not good enough)* | Manage failure | |
| *Structure* | Provide purpose, mastery & autonomy | Think small *(as in teams)* | Distribute power and decision making | Think aptitude and attitude |
| | Design for constant evolution | There is no one culture *(e.g. pioneers, settlers and town planners)* | Seek the best | |
| *Learning* | Use a systematic mechanism of learning *(a bias towards data)* | A bias towards action *(learn by playing the game)* | A bias towards the new *(be curious, take appropriate risks)* | Listen to your ecosystems *(acts as future sensing engines)* |
| *Leading* | Be the owner *(take responsibility)* | Move fast *(an imperfect plan executed today is better than a perfect plan executed tomorrow)* | Think big *(inspire others, provide direction)* | Strategy is iterative not linear *(fast reactive cycles)* |
| Phase I<br>Phase II<br>Phase III | Strategy is complex *(there will be uncertainty)* | Commit to the direction, be adaptive along the path *(crossing the river by feeling the stones)* | There is no core *(everything is transient)* | Be humble *(listen, be selfless, have* |

# Strategy · each loop narrows the uncertainty

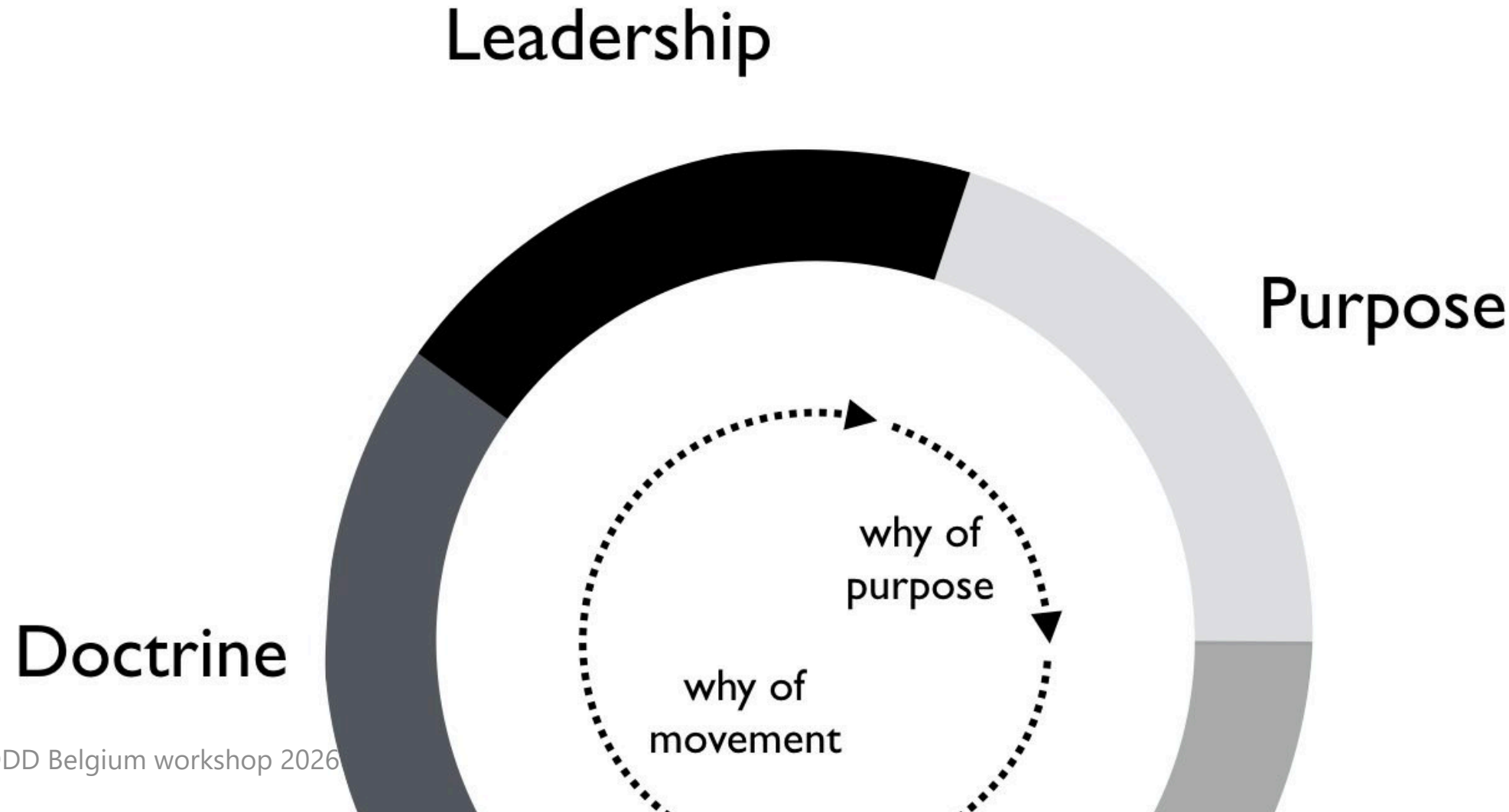


Strategy · after Simon Wardley

# Wardley’s two kinds of why

- Why of purpose: why we play. In chess: to win
- Why of movement: why this move over that one
- Leaders often give the first why and skip the second
- A map lets us argue the why of movement

## Strategy · the two types of why, Simon Wardley's figure 2, CC BY-SA 4.0

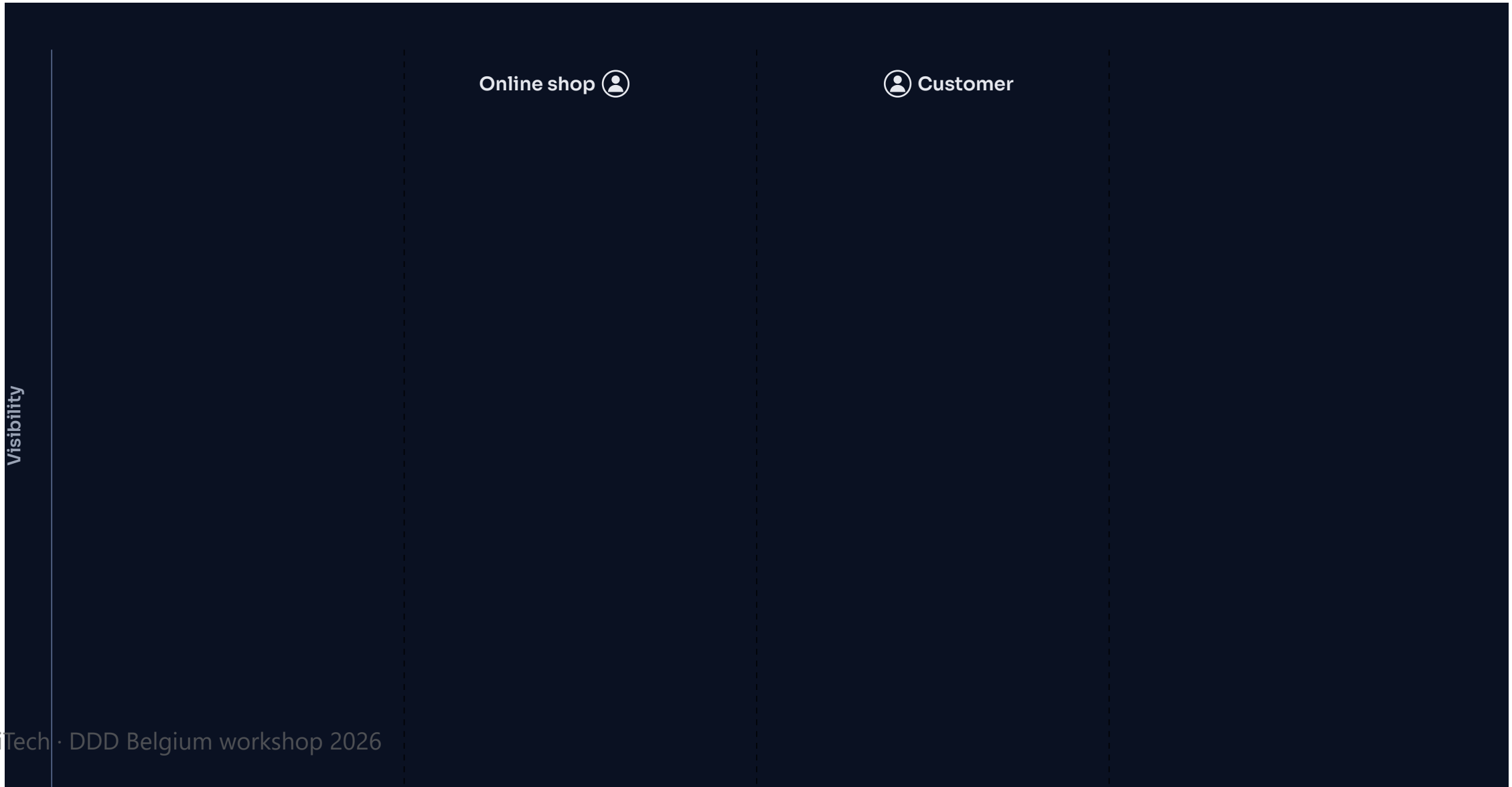


Climate

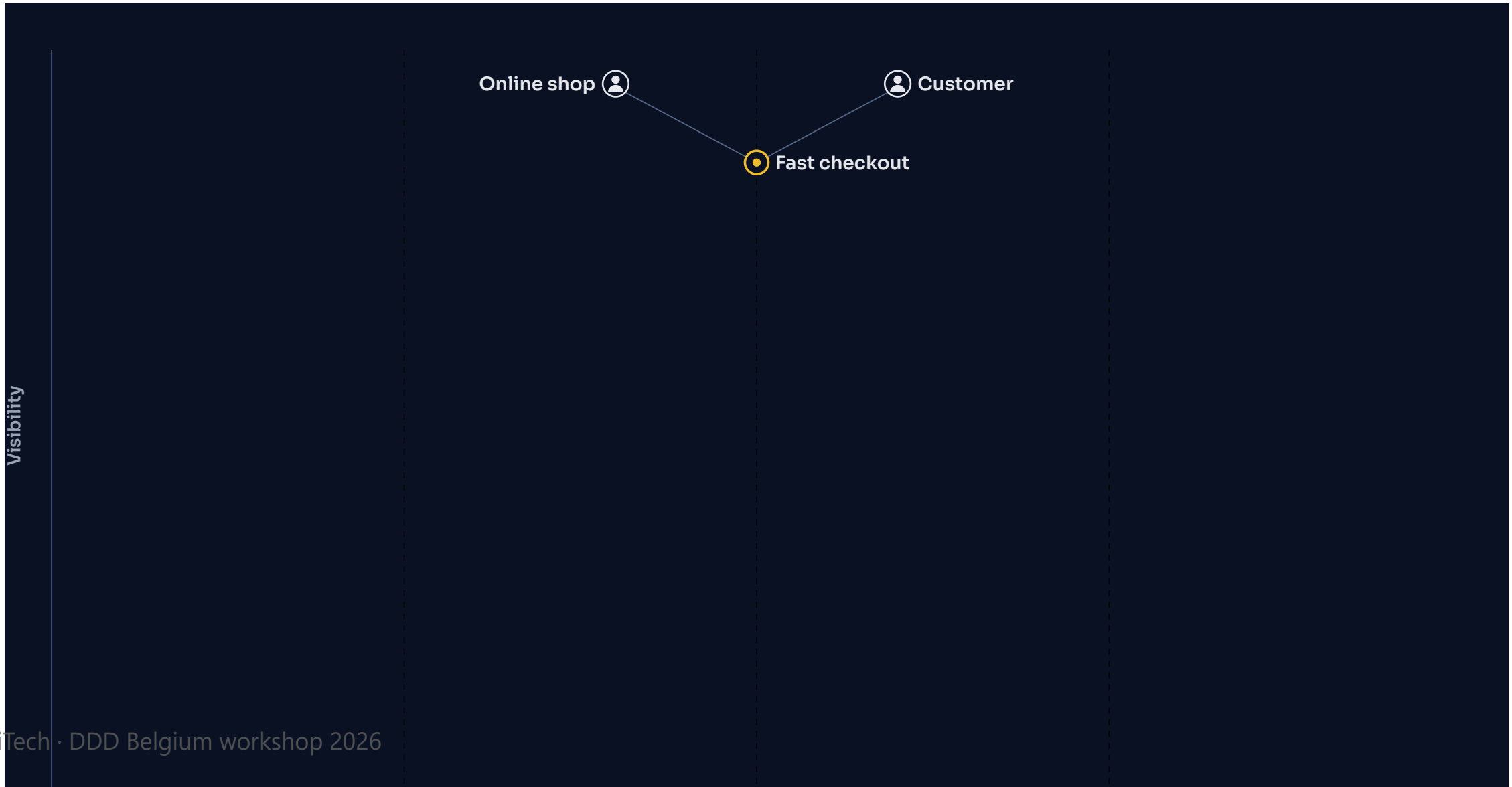
# Things you can't stop

- Everything evolves: genesis to commodity
- Success breeds inertia
- Efficiency enables innovation
- You can anticipate these, not avoid them

# Co-evolution · 1 of 8 — the users



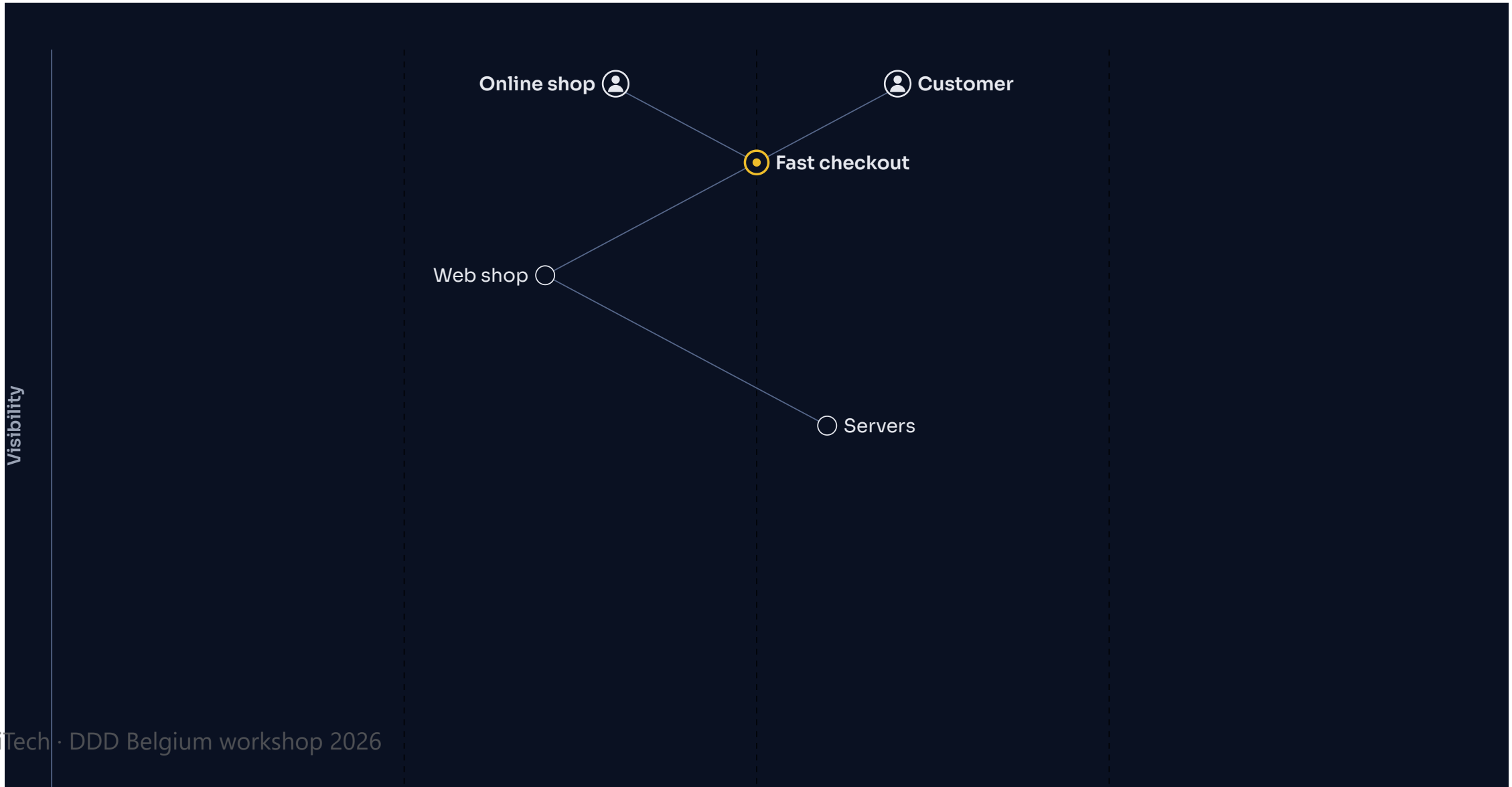
# Co-evolution · 2 of 8 — the need



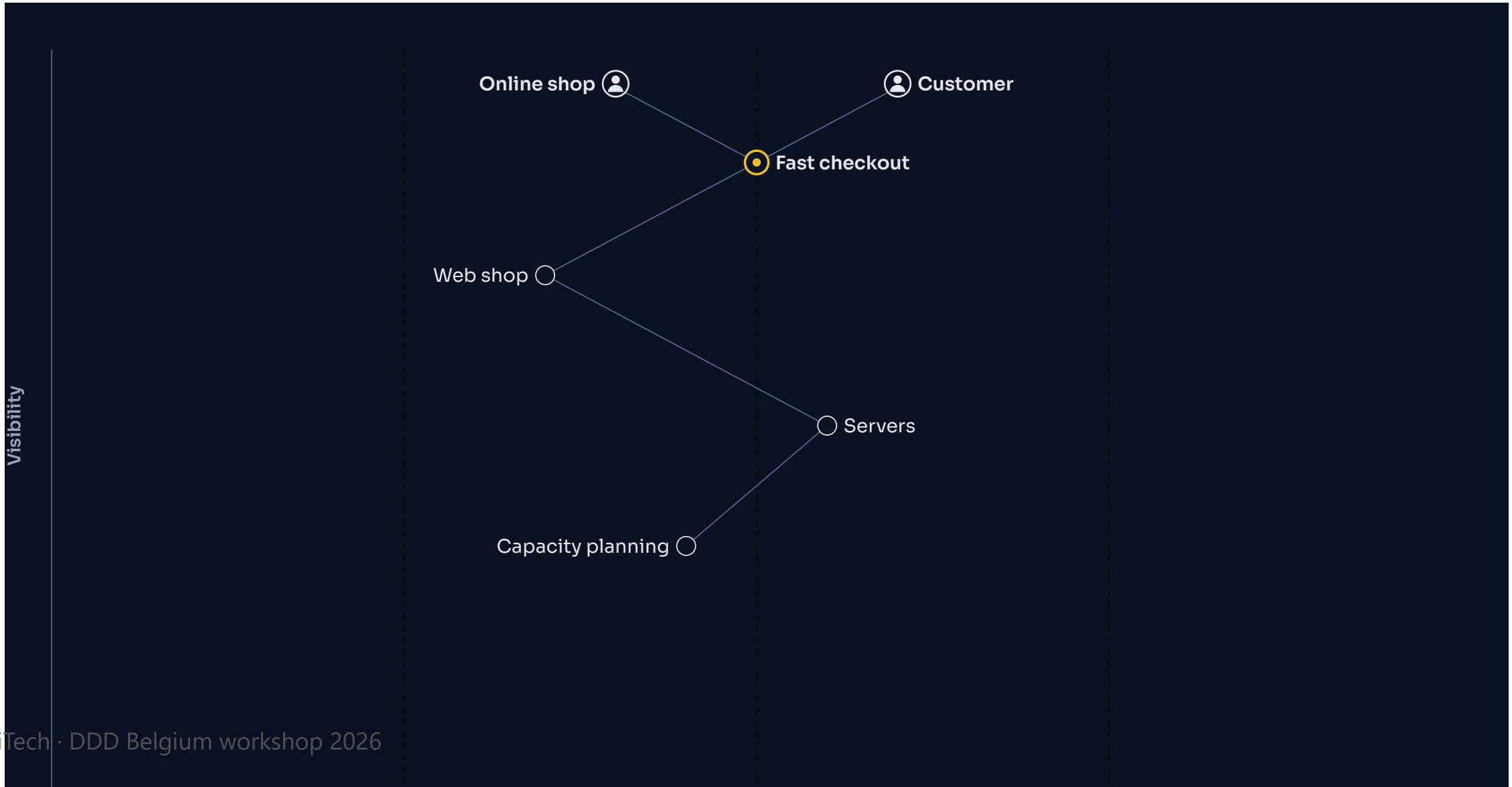
# Co-evolution · 3 of 8 — the web shop



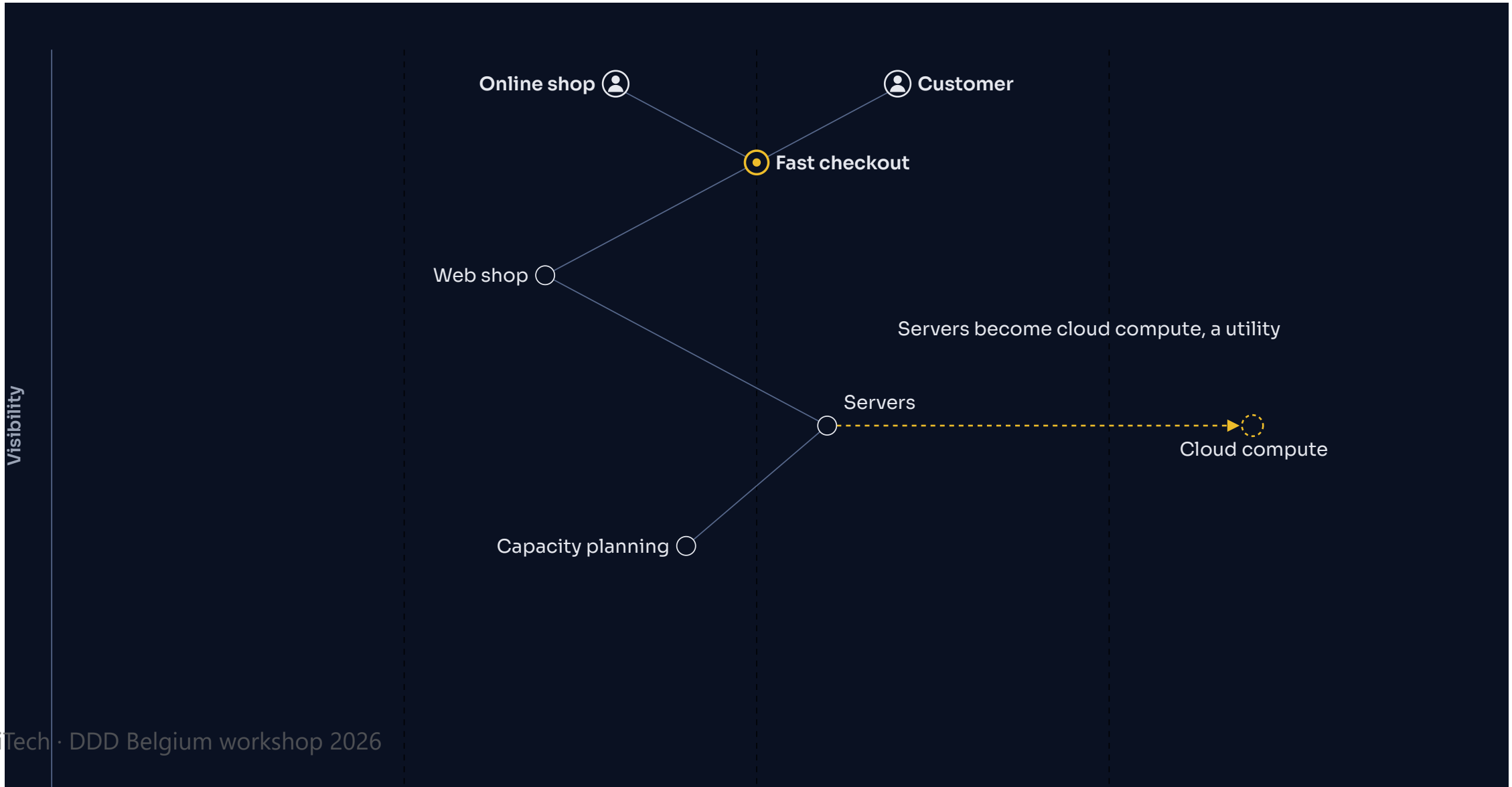
# Co-evolution · 4 of 8 — servers



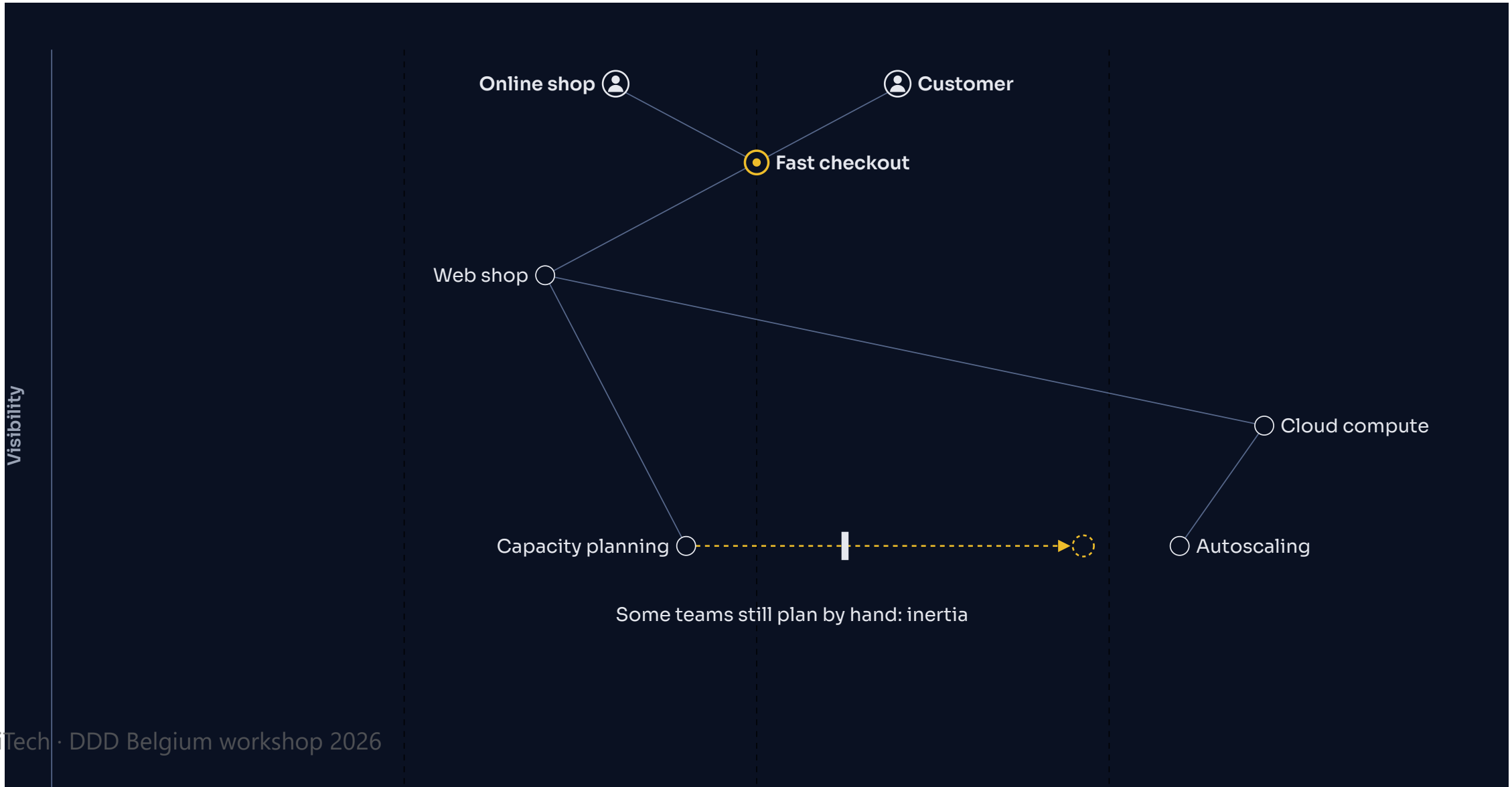
# Co-evolution · 5 of 8 — the practice



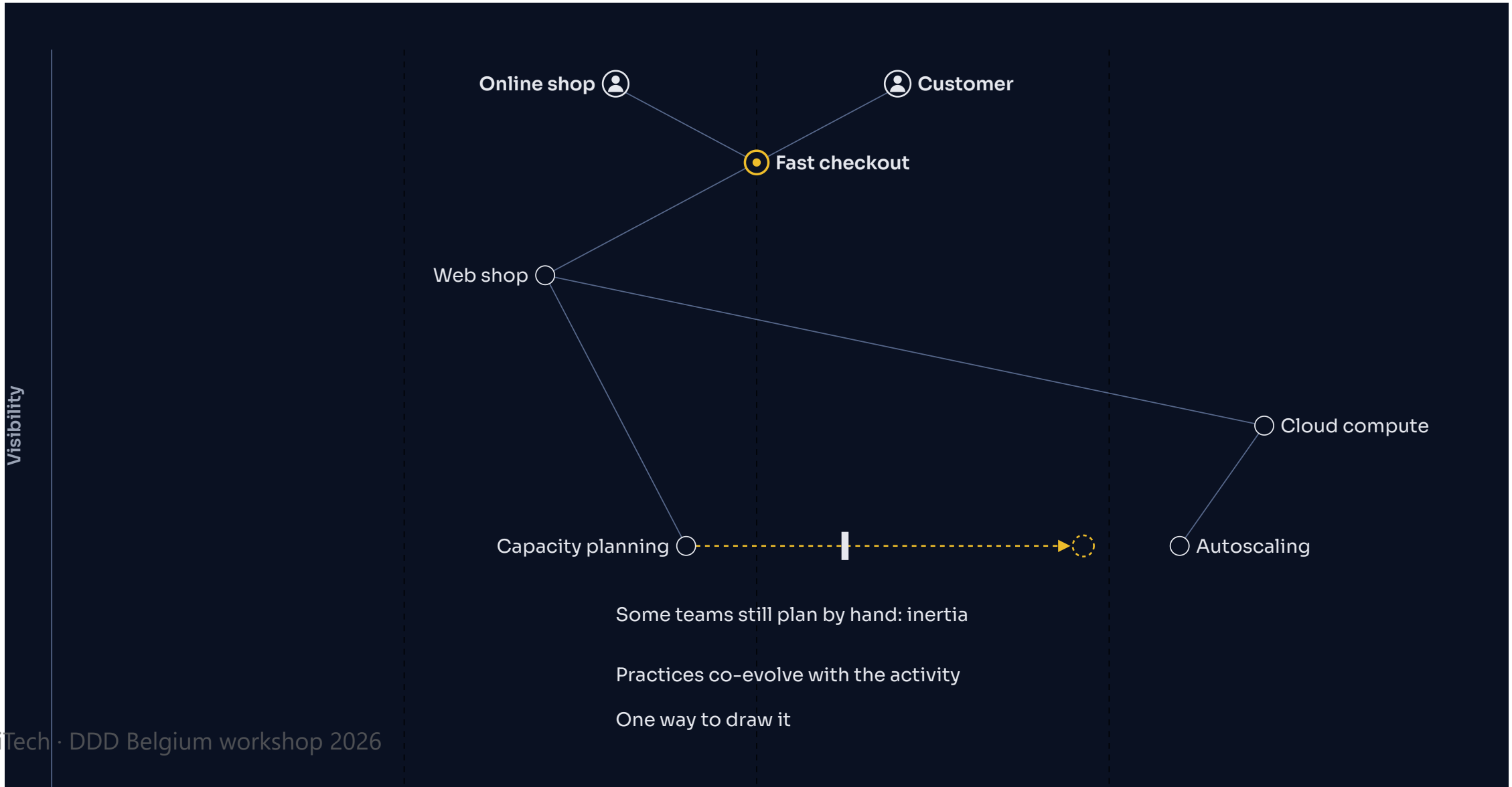
# Co-evolution · 6 of 8 — compute evolves



# Co-evolution · 7 of 8 — a new practice



# Co-evolution · 8 of 8 — co-evolution



Climate · co-evolution

# Practices evolve with the activity

- Servers as a product: you need capacity planning
- Compute as a utility: autoscaling replaces it
- Some teams still plan capacity by hand. That is inertia
- Review is old: punch cards made mistakes costly
- Open source made review a daily practice
- Code is now a commodity. Review will evolve too

Exercise 2 · group work

# Exercise 2: build the chain

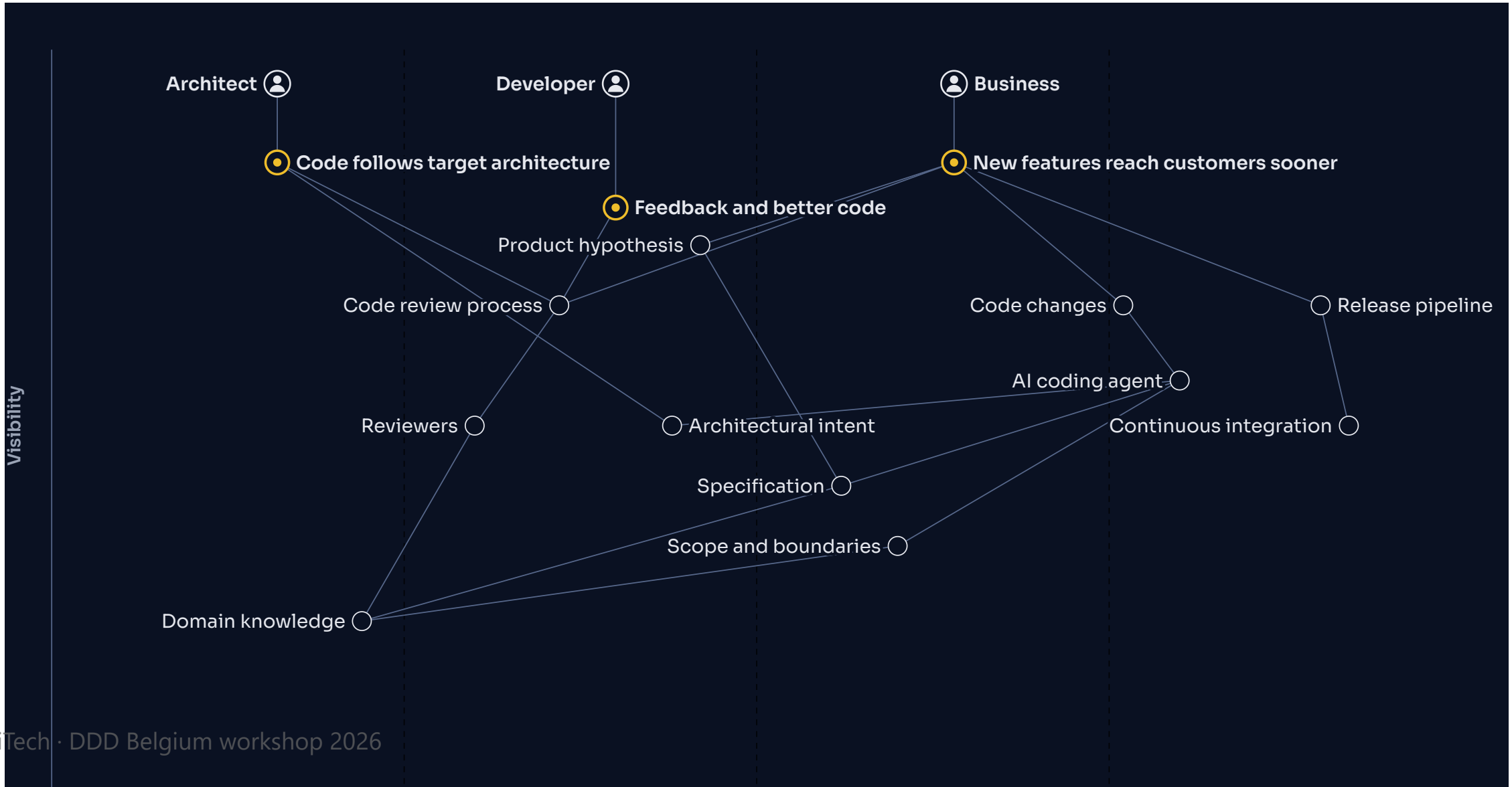
- Start from your chosen anchor (e.g., Management + Time to market)
- Build the value chain: what does that need depend on?
- Keep asking "what does X depend on?" until you reach infrastructure
- Stack components by visibility: visible to users at top, hidden at bottom
- **No evolution axis yet** - that comes in Exercise 3

Exercise 2 · back together

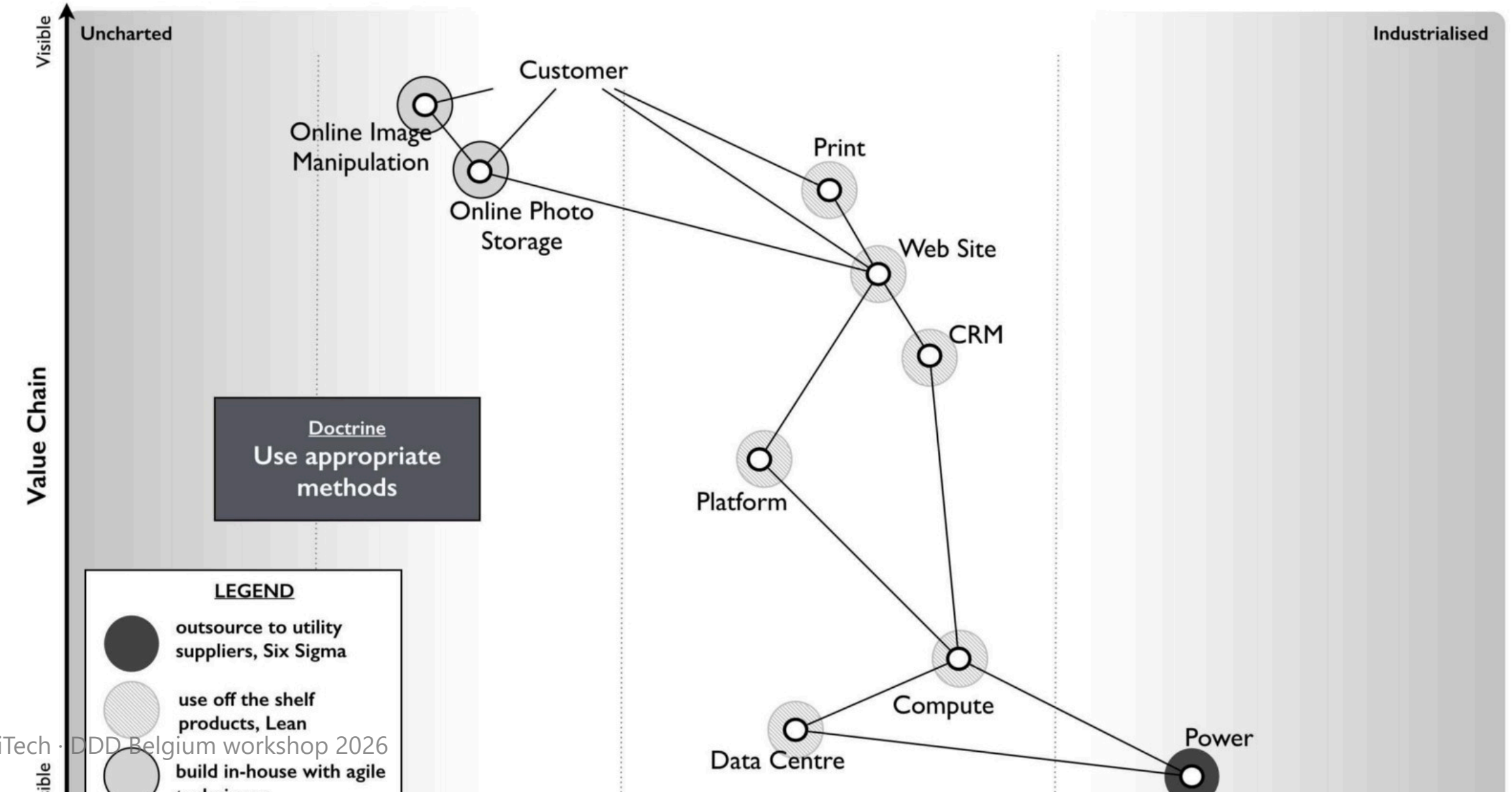
# Exercise 2: discuss your chain

- Where is the line between problem domain and solution domain?
- Which dependency was hardest to identify?
- Did you reach infrastructure, or stop somewhere in the middle?

# Exercise 2 · one way to map it



# Doctrine · use appropriate methods, Simon Wardley's figure 35, CC BY-SA 4.0



Doctrine · use appropriate methods, after Simon Wardley

# One way of working does not fit the whole map

| | Genesis, custom built | Product | Commodity, utility |
|---|---|---|---|
| The goal | Learn fast | Get it right | Cut cost and variation |
| How we work | Agile: small experiments | Lean: improve the flow | Six Sigma: standardise, automate |
| How we get it | Build it ourselves | Buy it and fit it in | Rent it as a utility |
| At Harbour Insurance | The product hypothesis | Event model tooling | AI coding agents, CI |

Simon Wardley's doctrine use appropriate methods: pick the way of working by the

Exercise 3 · group work

# Exercise 3: spread it out!

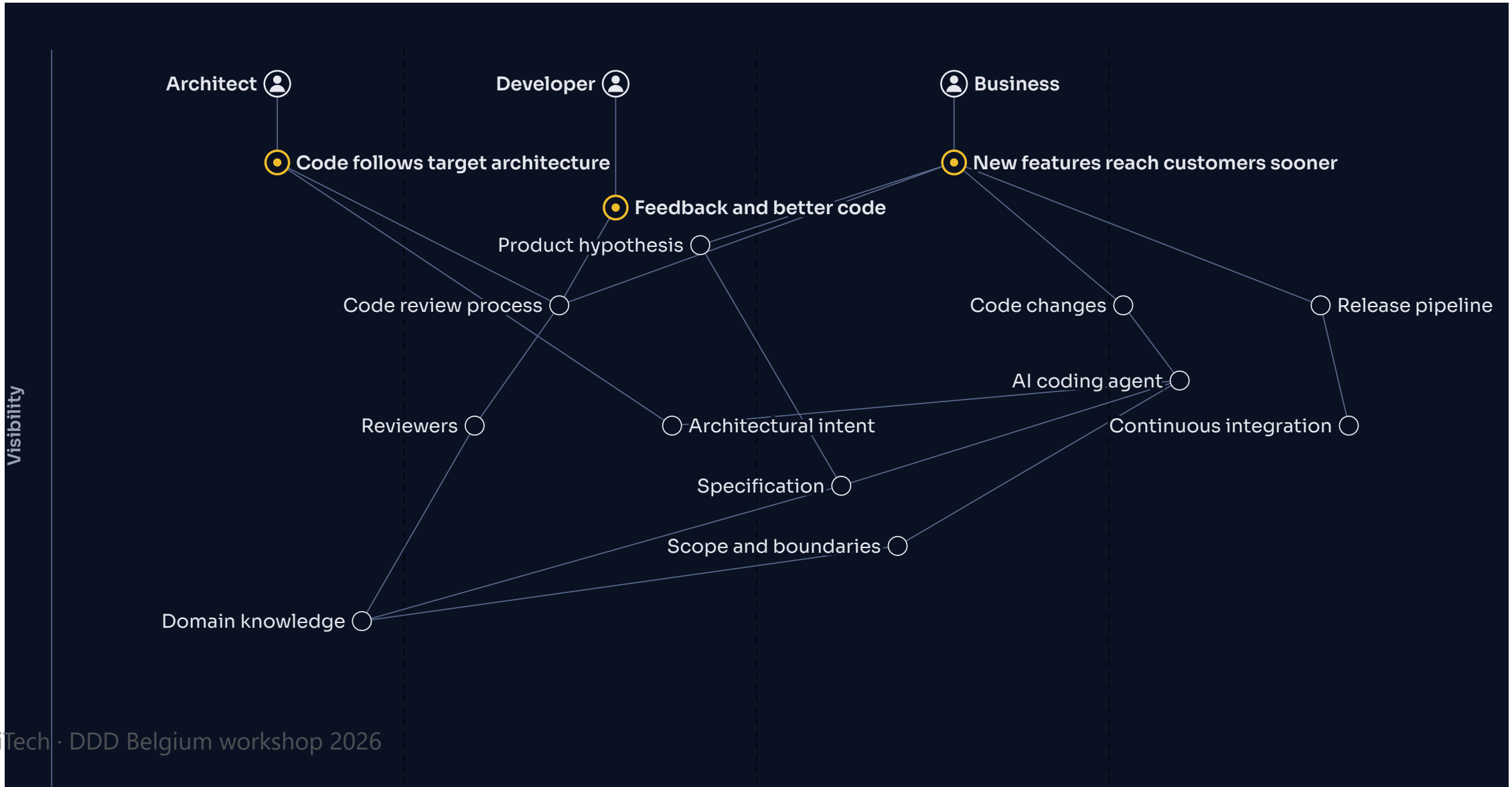
- Take your value chain from Exercise 2
- Now add the evolution axis
- Place every component: Genesis → Custom → Product → Commodity
- Users and needs stay at the top
- Stuck on a placement? The domain expert decides

Exercise 3 · back together

# Exercise 3: where did your group disagree?

- What is a commodity now that was not two years ago?
- Which placement surprised you?
- Next round: carry on with your map, or start from ours

# Exercise 3 · one way to map it



03

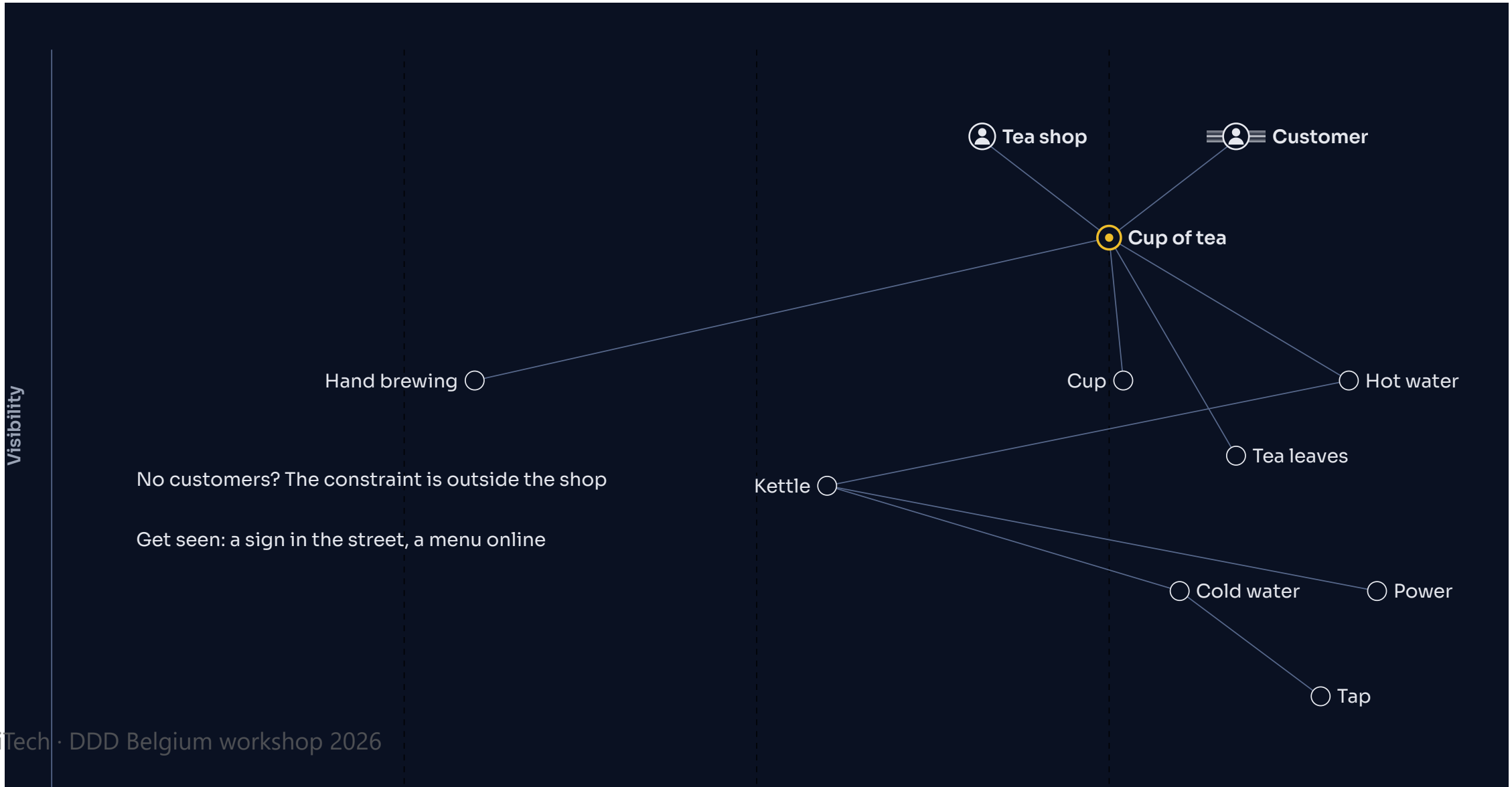
# Tea shop at rush hour

Scoping · after Tristan Slominski

# How much should we map?

- Maps grow until they stop saying anything
- Ask: what is the primary constraint here?
- Map the user, the need, the path to it
- Map around the constraint, as Tristan Slominski teaches it

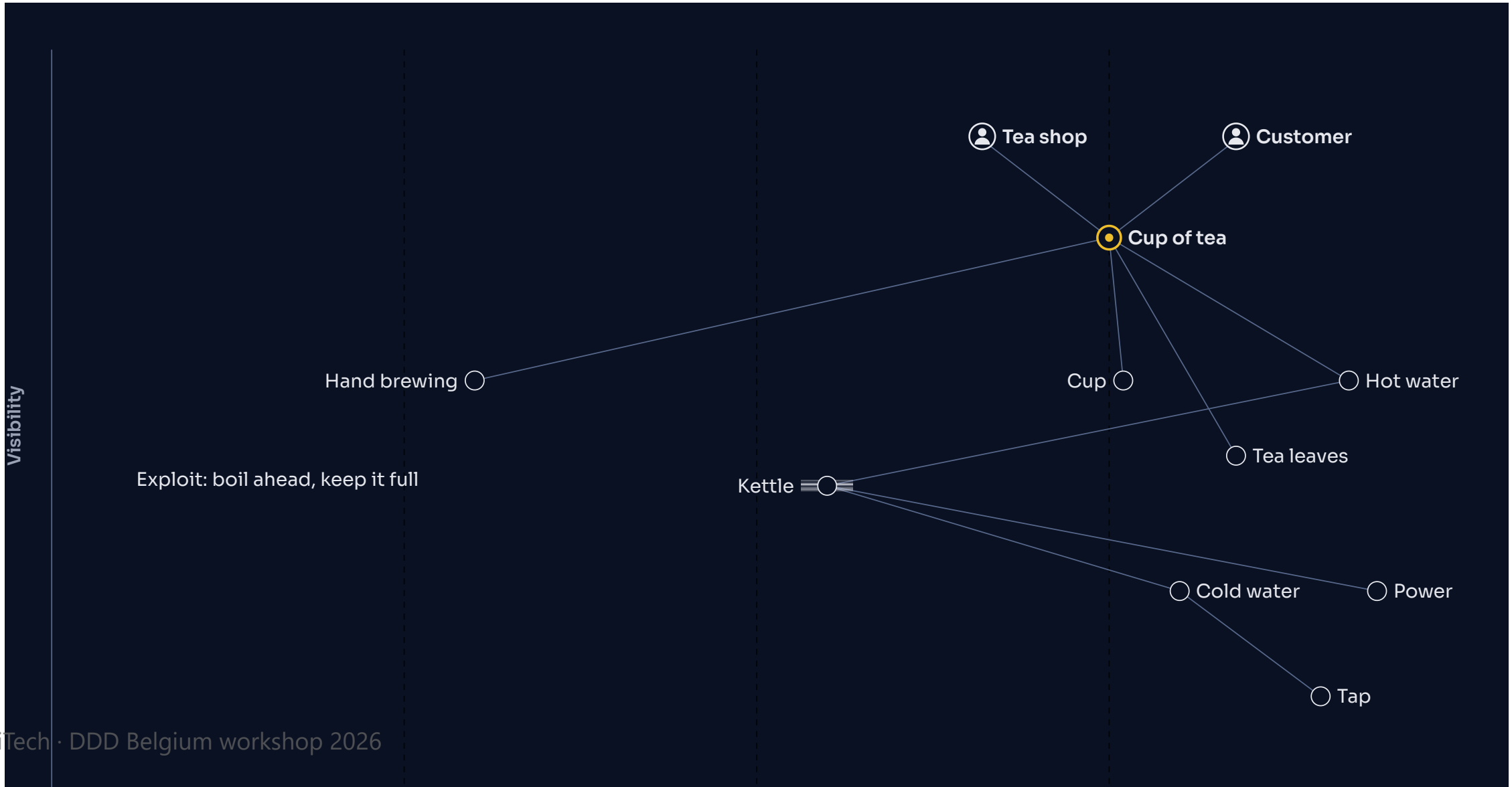
# Tea shop at rush hour · 1 of 7 — outside or inside?



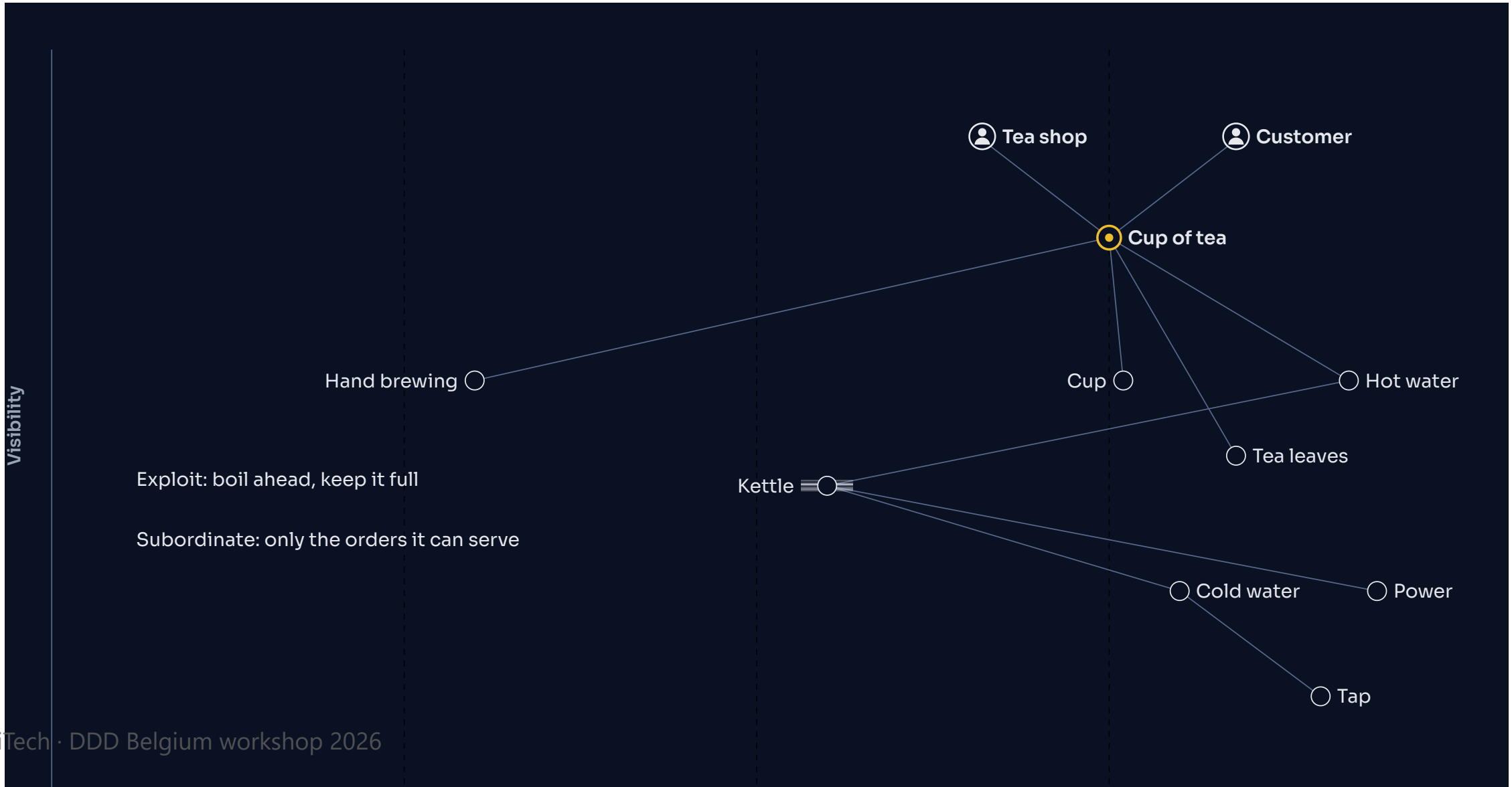
# Tea shop at rush hour · 2 of 7 — the constraint



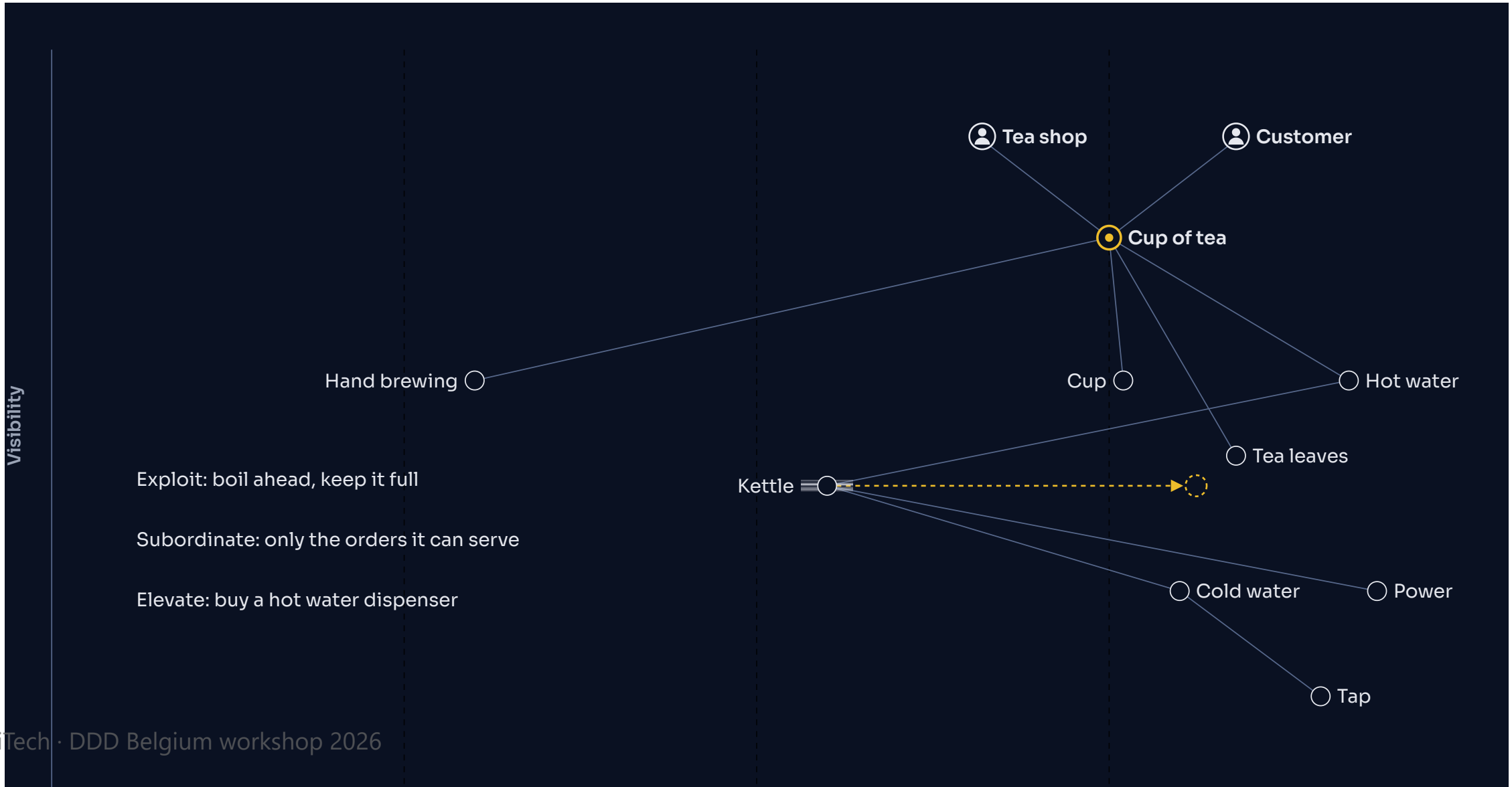
# Tea shop at rush hour · 3 of 7 — exploit



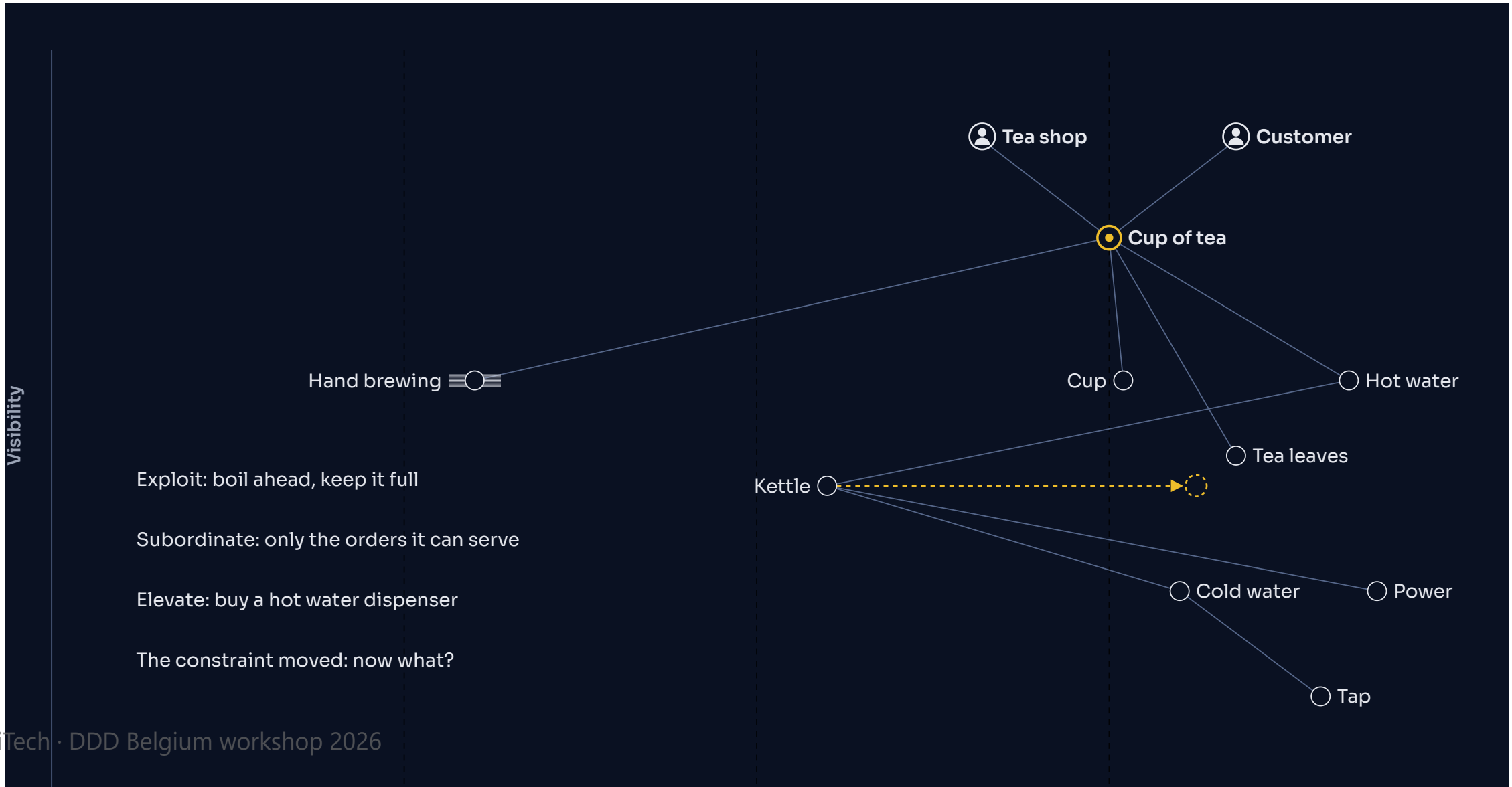
# Tea shop at rush hour · 4 of 7 — subordinate



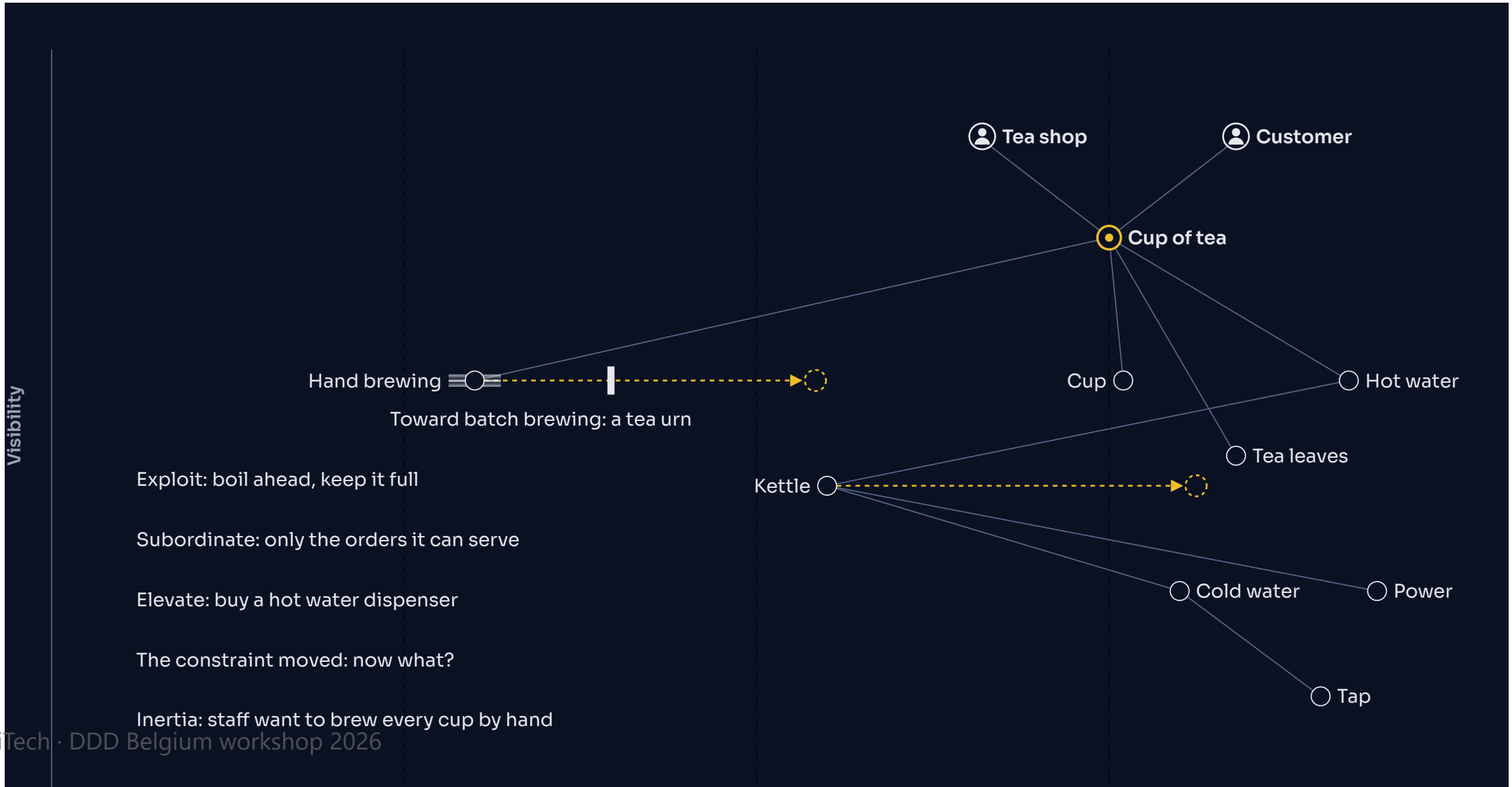
# Tea shop at rush hour · 5 of 7 — elevate



# Tea shop at rush hour · 6 of 7 — the constraint moves



# Tea shop at rush hour · 7 of 7 — the inertia



Tea shop at rush hour

# Two kinds of friction

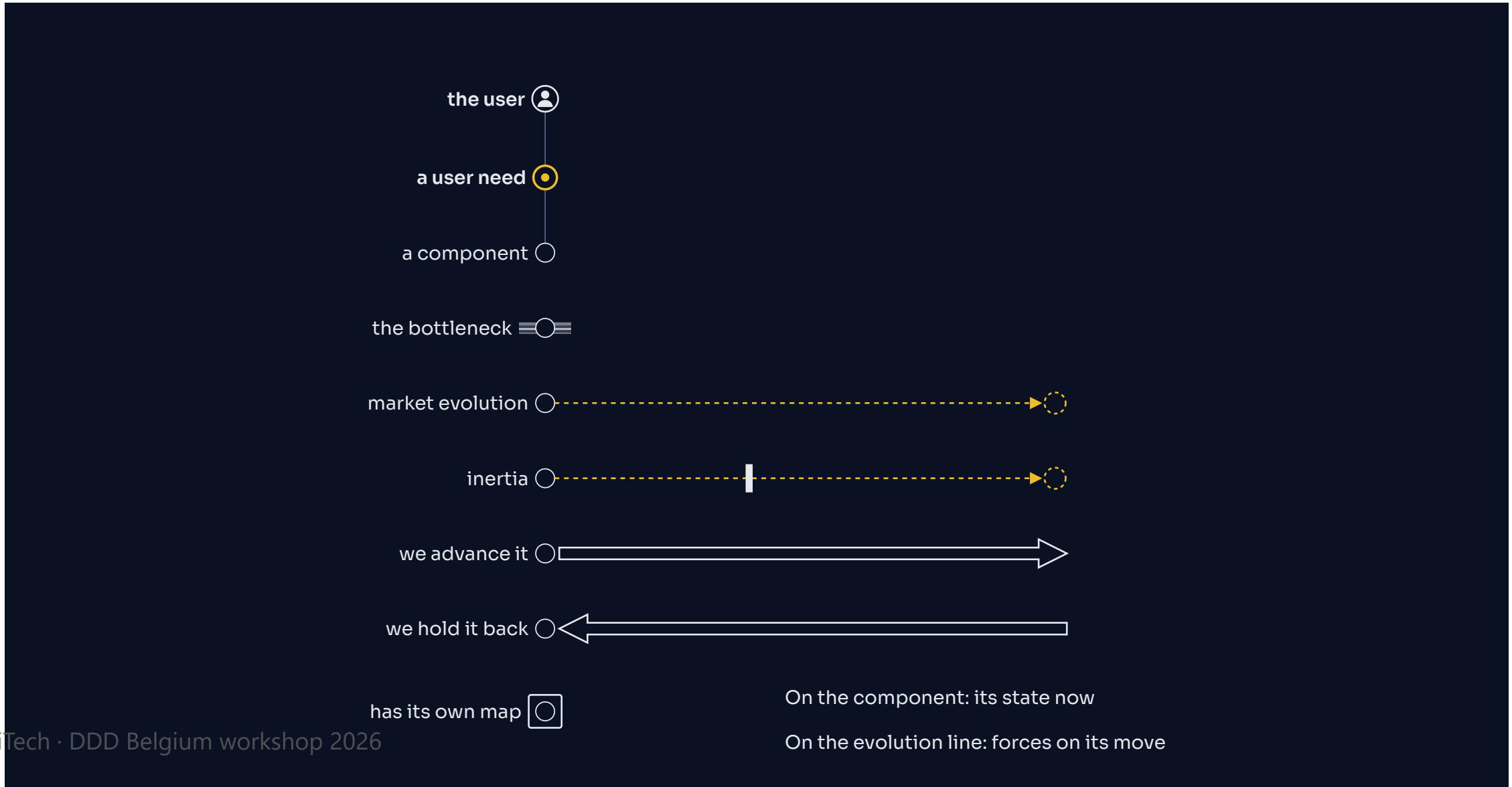
- Constraint: a capacity limit. One kettle, a queue
- Inertia: resistance to change. "We brew by hand"
- A constraint caps the flow; inertia blocks the move
- Work on the constraint first. The rest waits on it

Theory of Constraints · Goldratt

# Goldratt: exploit before you elevate

- First ask: is the constraint outside the shop or inside?
- Exploit: get the most out of the constraint as it is
- Subordinate: pace everything else to it
- Elevate: invest only when that is not enough
- Repeat: the constraint moves, often into a practice

# Legend


the user
a user need
a component
the bottleneck
market evolution
inertia
we advance it
we hold it back
has its own map
On the component: its state now
On the evolution line: forces on its move

Exercise 4 · group work

# Exercise 4: find the constraint and the pipelines

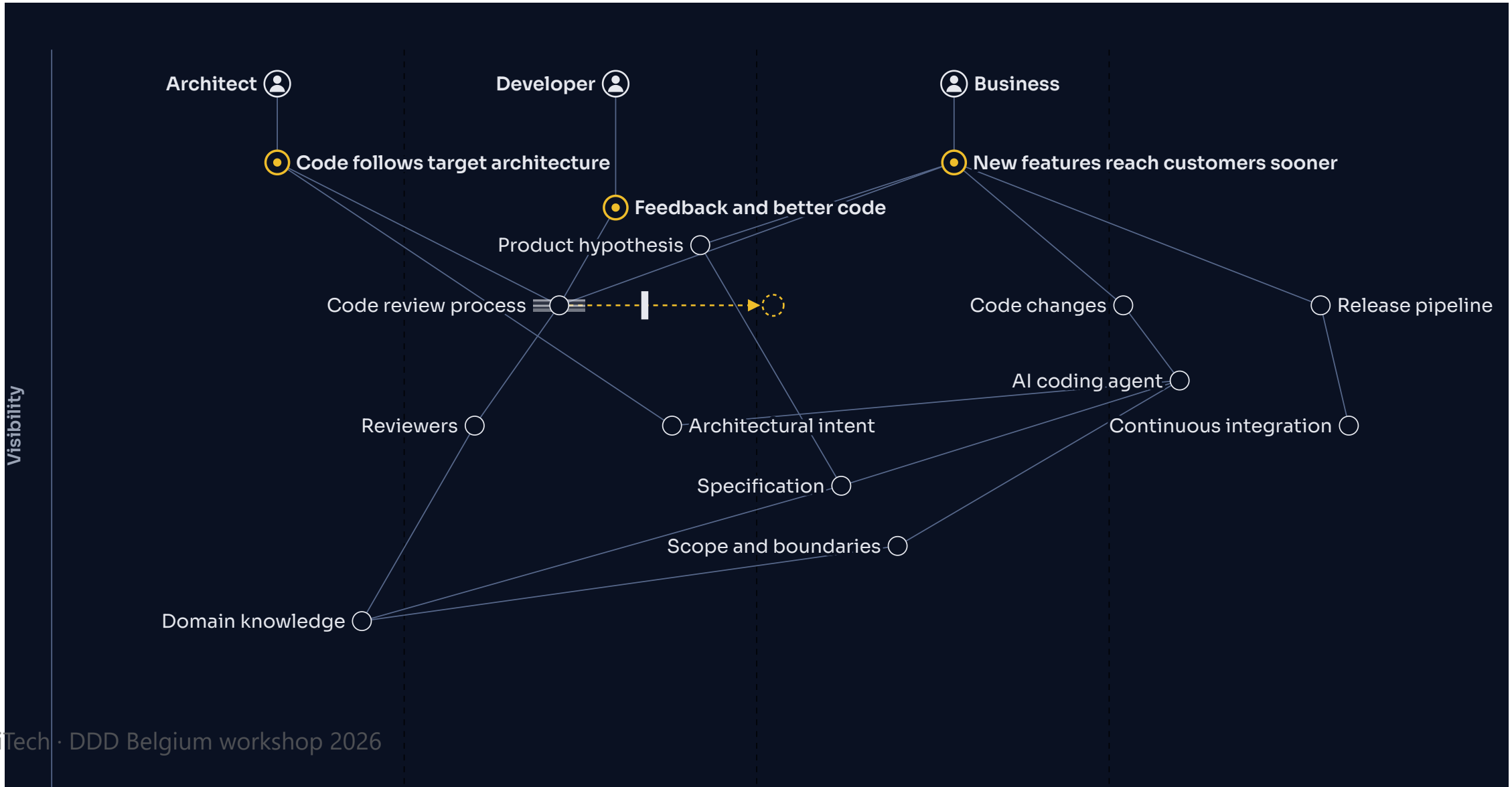
- Keep working on your map
- **Identify the 3 pipelines:** code, quality, release
- Mark the constraint: where does work wait? (Hint: quality pipeline)
- Mark one inertia: what resists change?
- **Notice:** AI sped up code pipeline, but constraint is in quality
- Exploit, subordinate, then elevate the constraint

Exercise 4 · back together

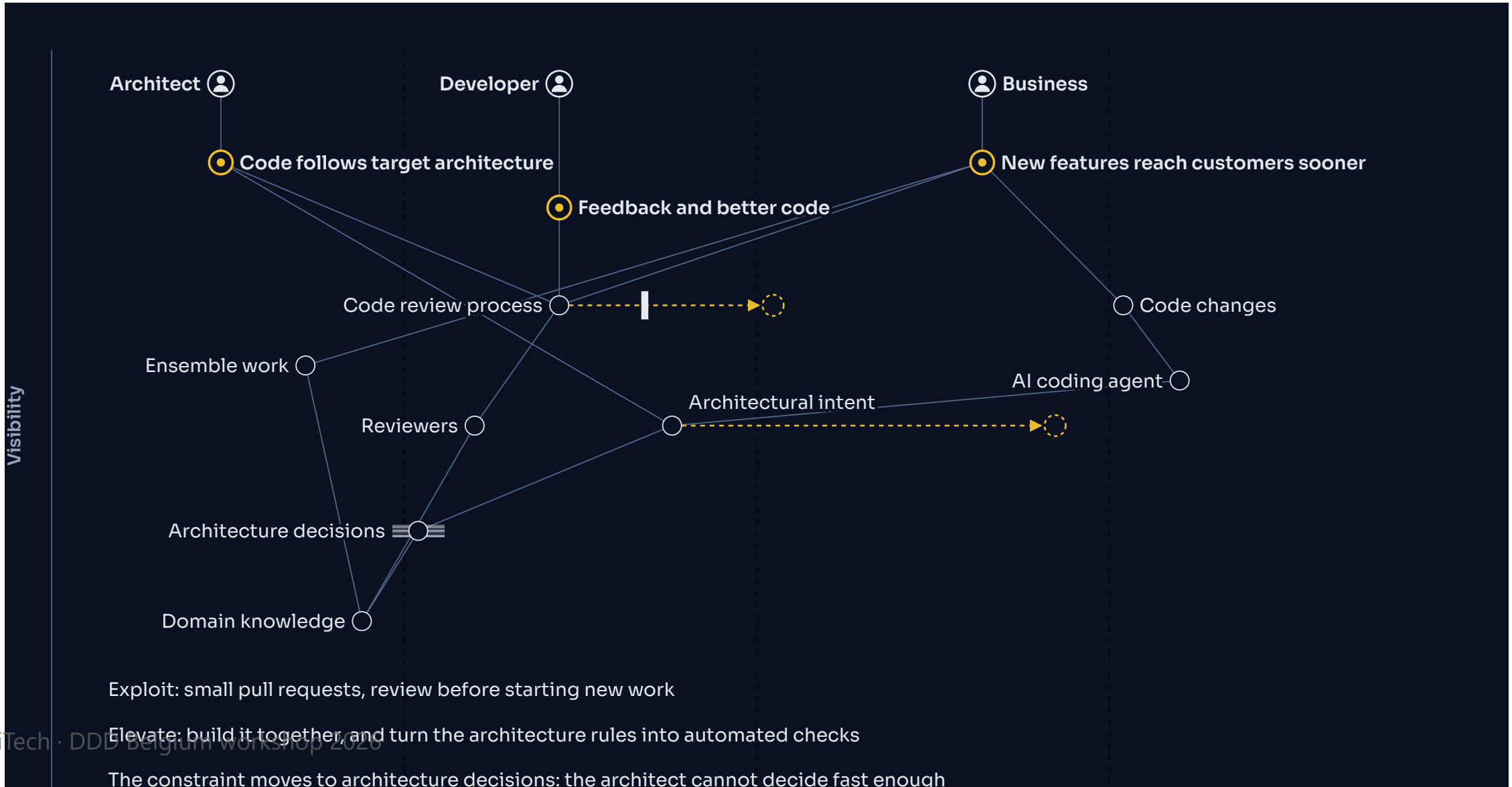
# Exercise 4: discuss constraints and pipelines

- Which pipeline moved fastest? (Code)
- Which pipeline has the constraint? (Quality - review)
- Is the constraint a practice or technology?
- Which of your exploit moves needs no budget?
- Where does the constraint move after you elevate it?

# Exercise 4 · one way to find the constraint



# Exercise 4 · one way to work the constraint



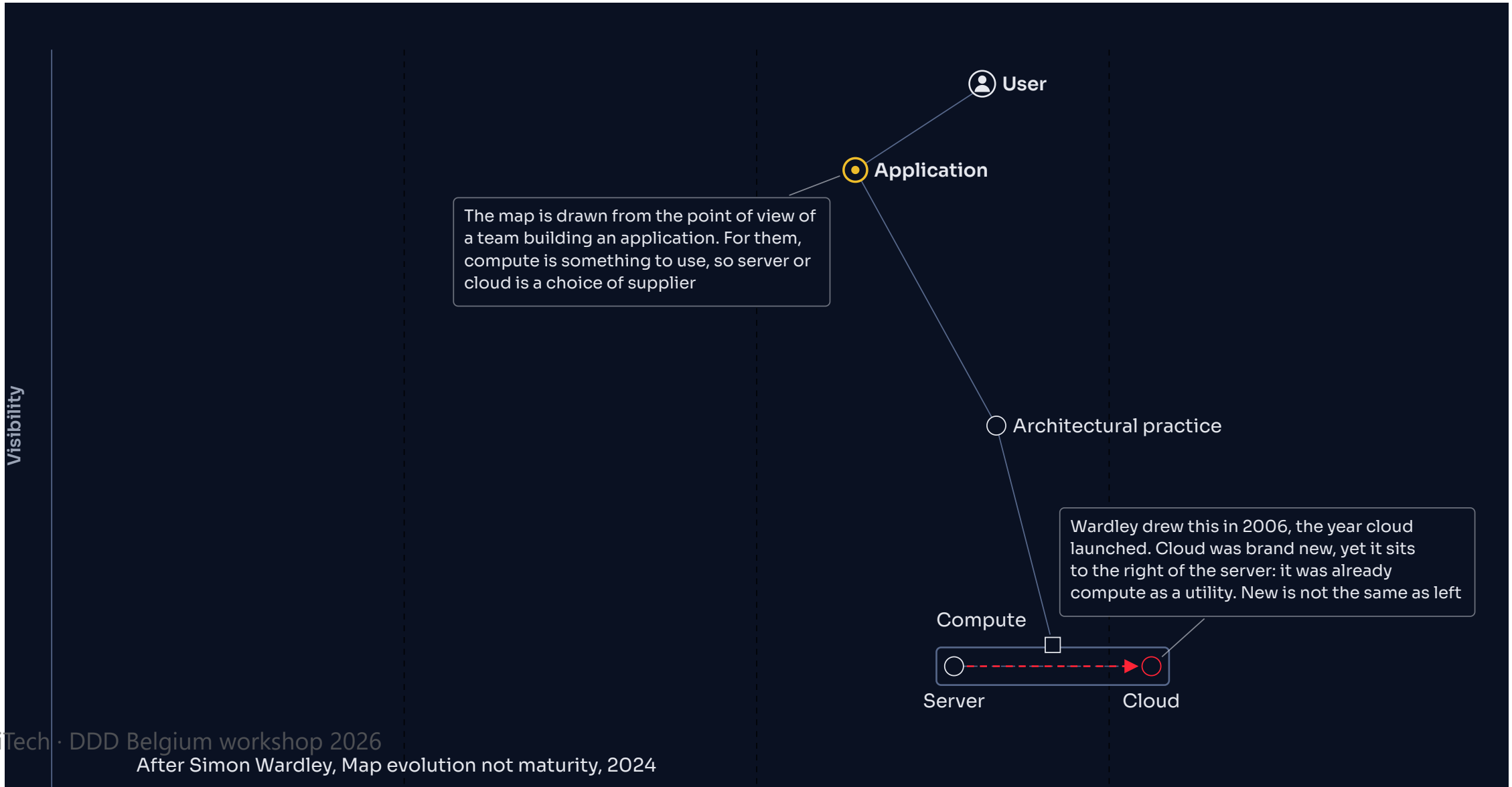
04

# Pipelines: several ways to do one thing

# A pipeline: several ways to do one thing

- One component, several ways to do it, side by side in a box
- Each way sits at its own stage of evolution
- A new invention starts on the left
- The industrialised form of an old activity goes on the right, however new it is
- If people move from one way to another, draw an arrow

# Pipelines · after Simon Wardley, "Map evolution not maturity"

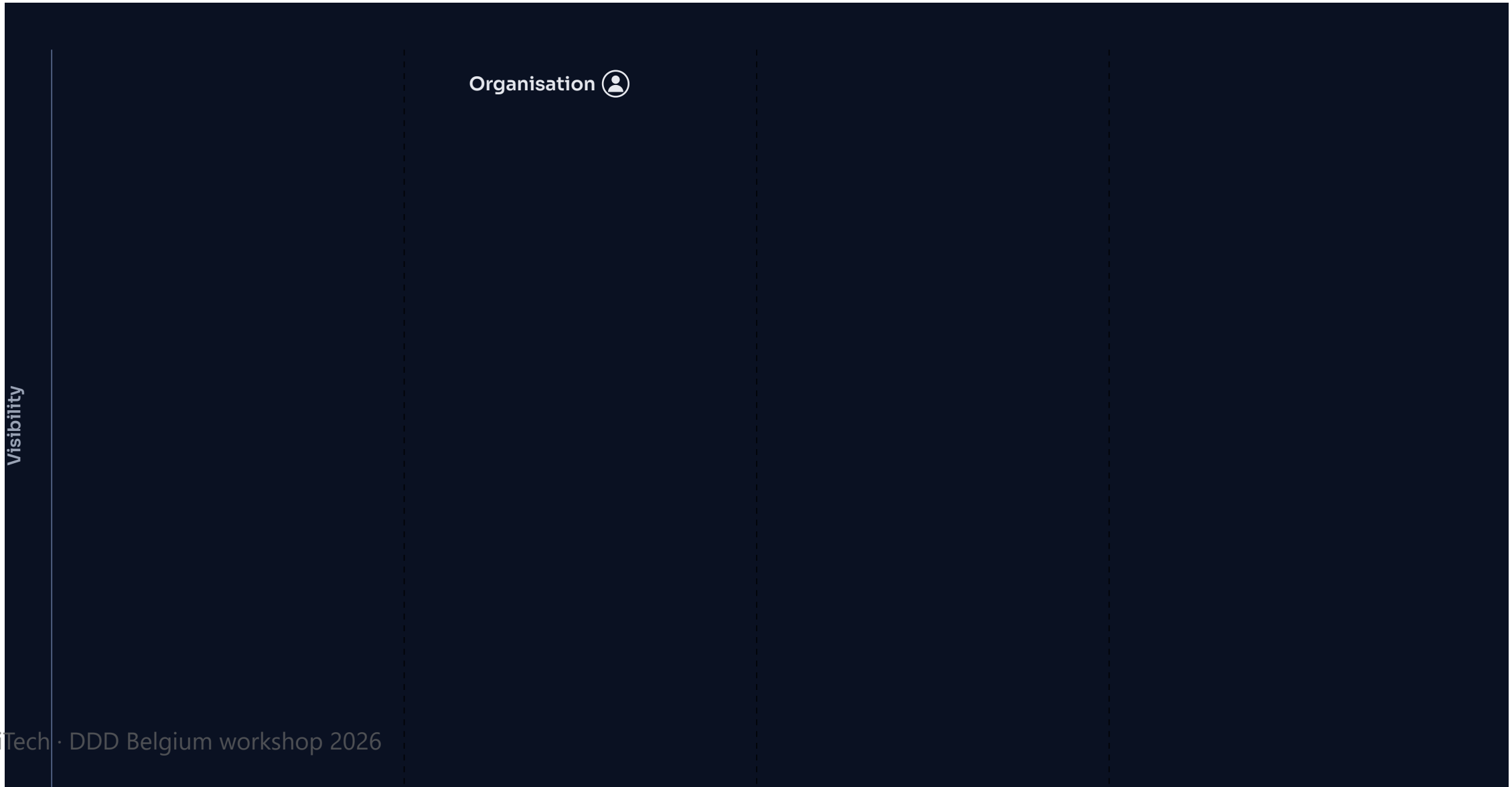



After Simon Wardley, Map evolution not maturity, 2024

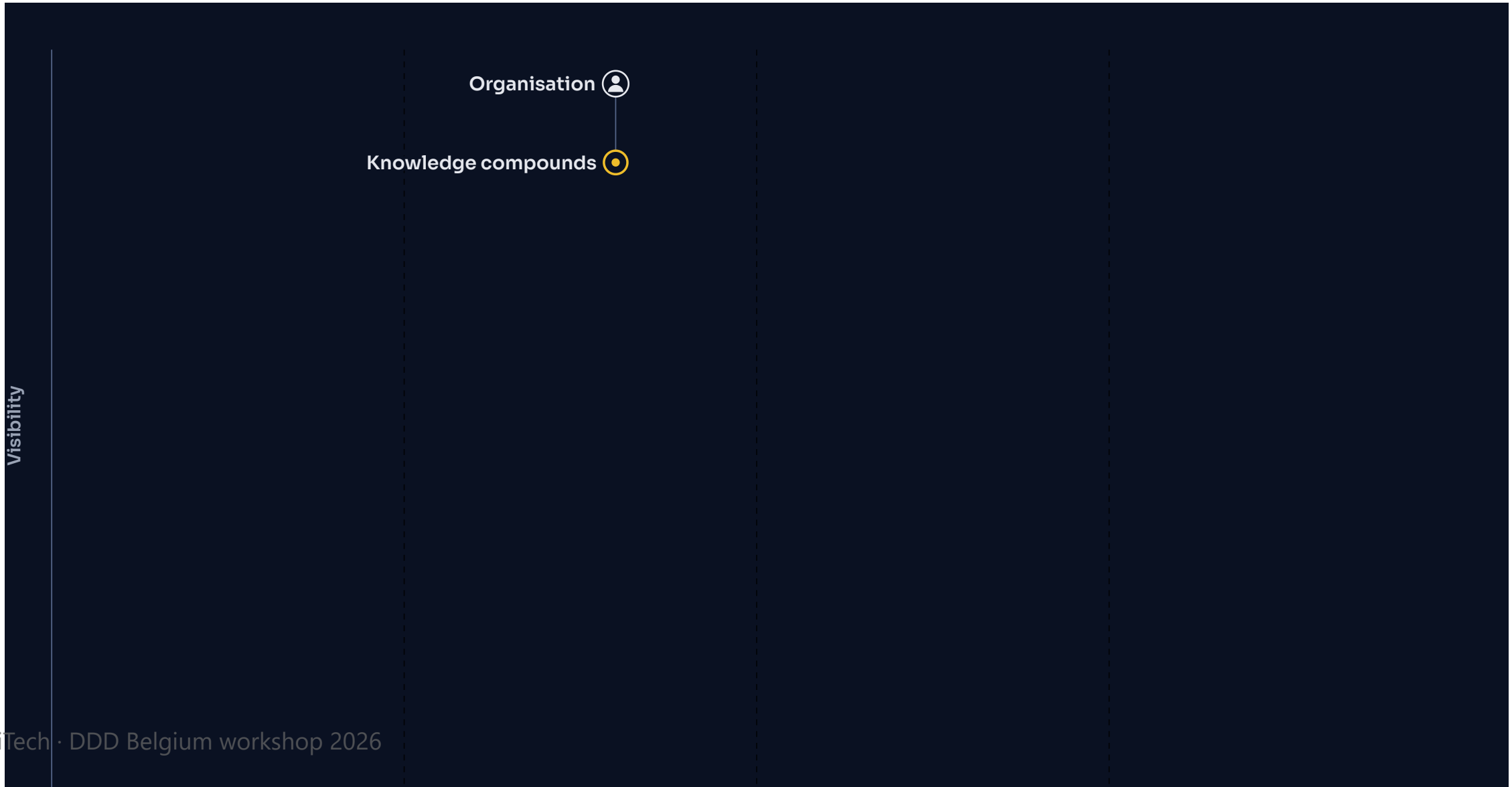
05

# Zoom in: knowledge architecture

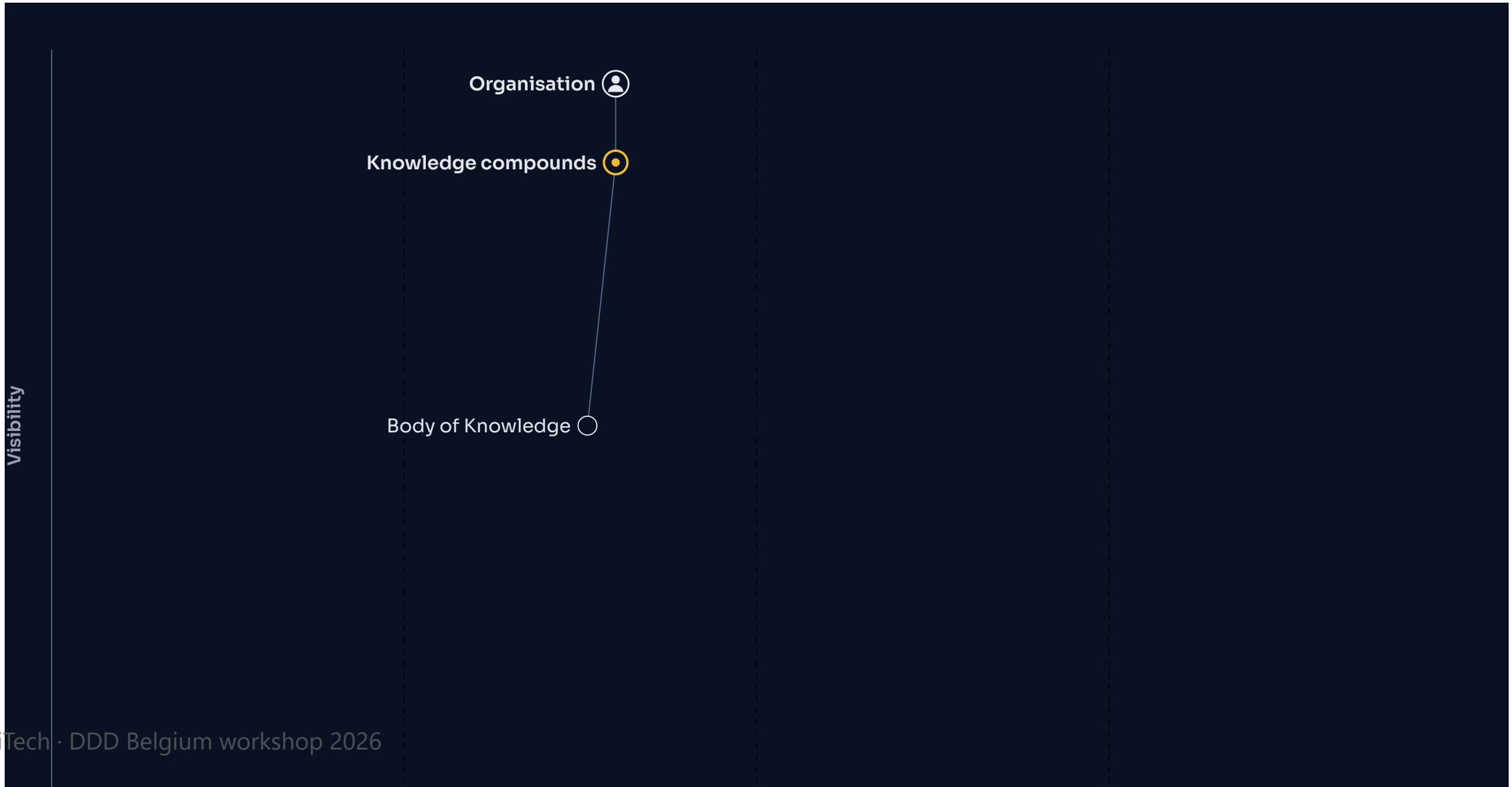
# Knowledge architecture · 1 of 12 — the organisation



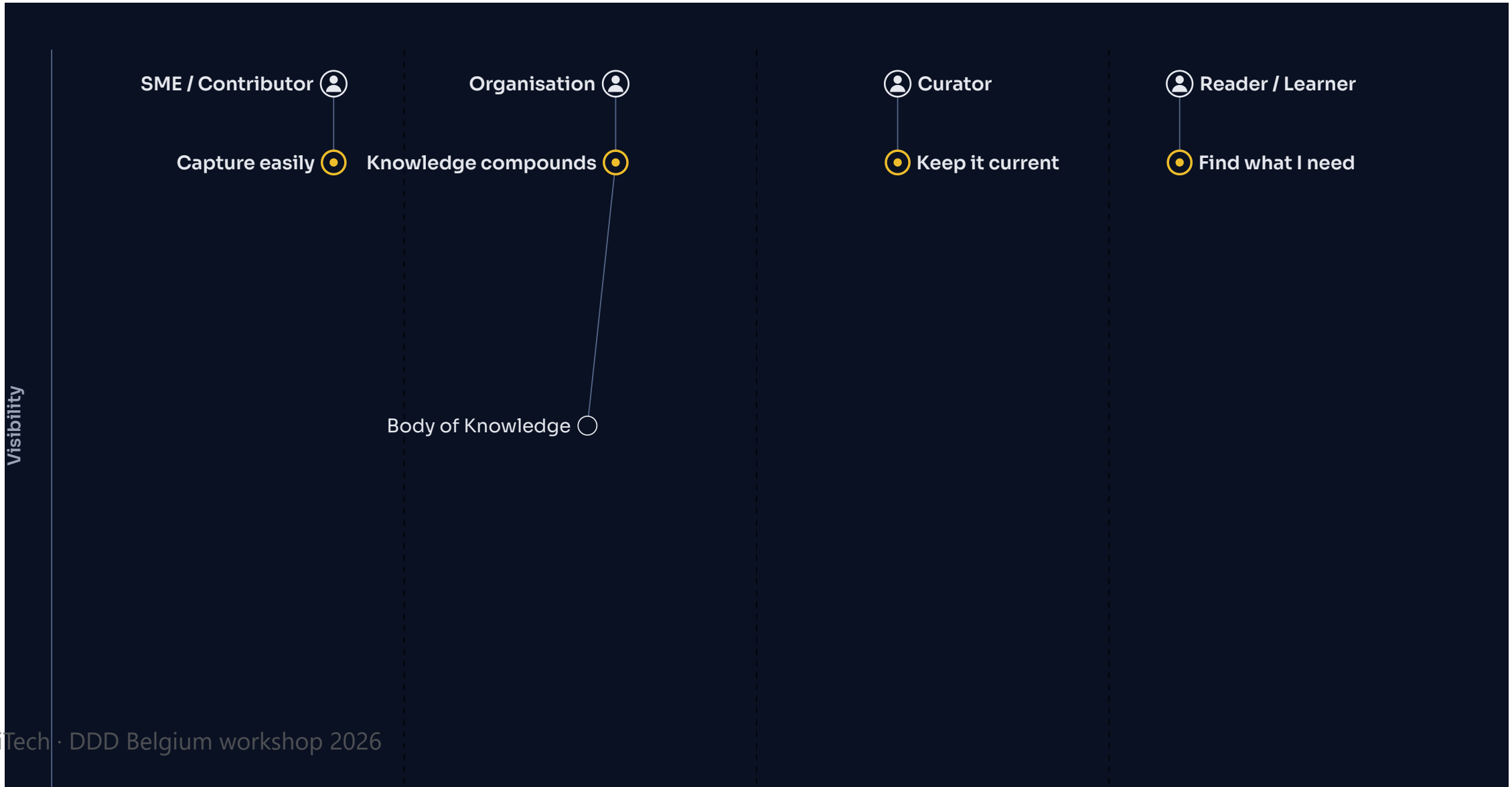
# Knowledge architecture · 2 of 12 — its need



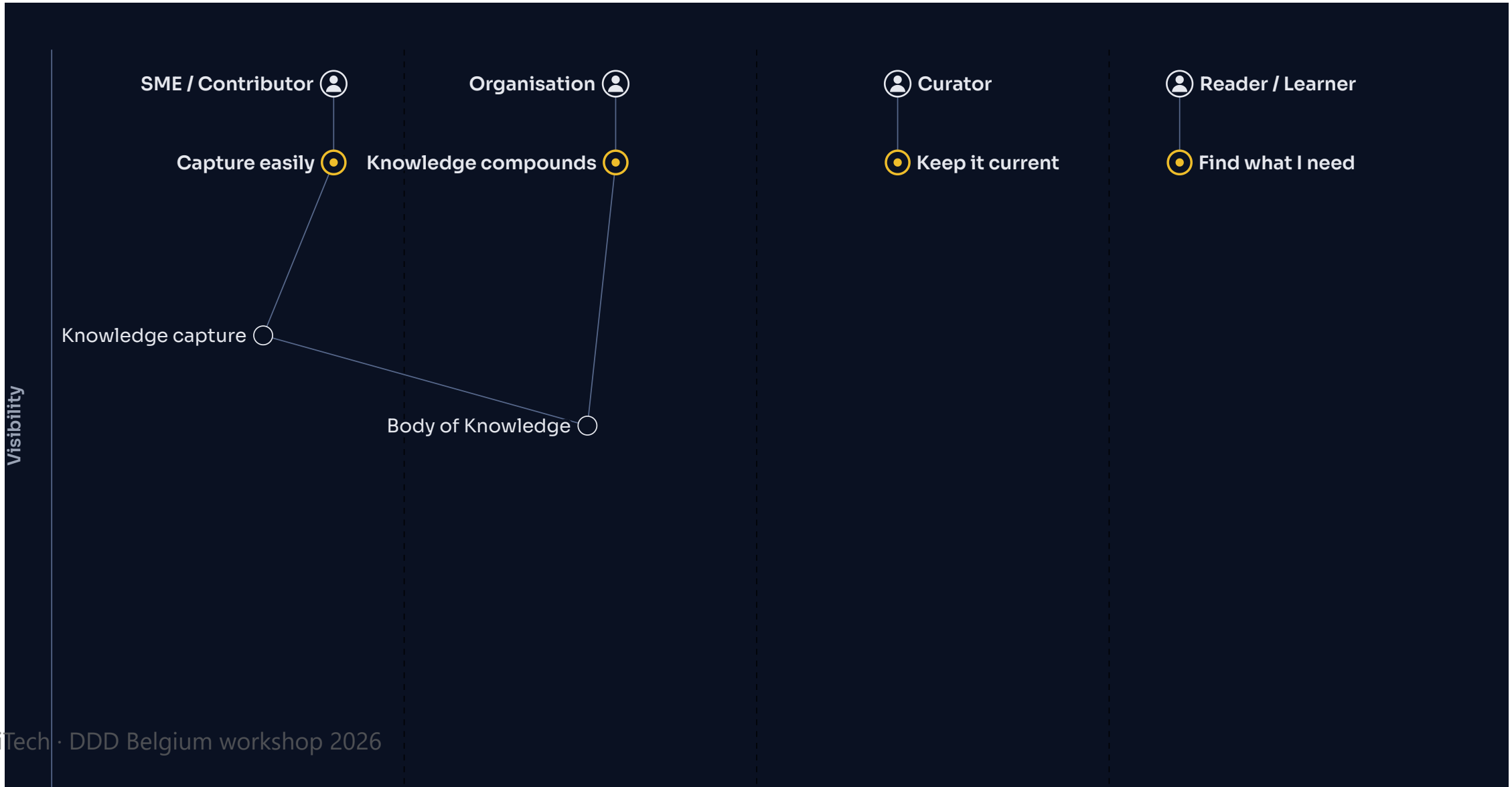
# Knowledge architecture · 3 of 12 — the hub



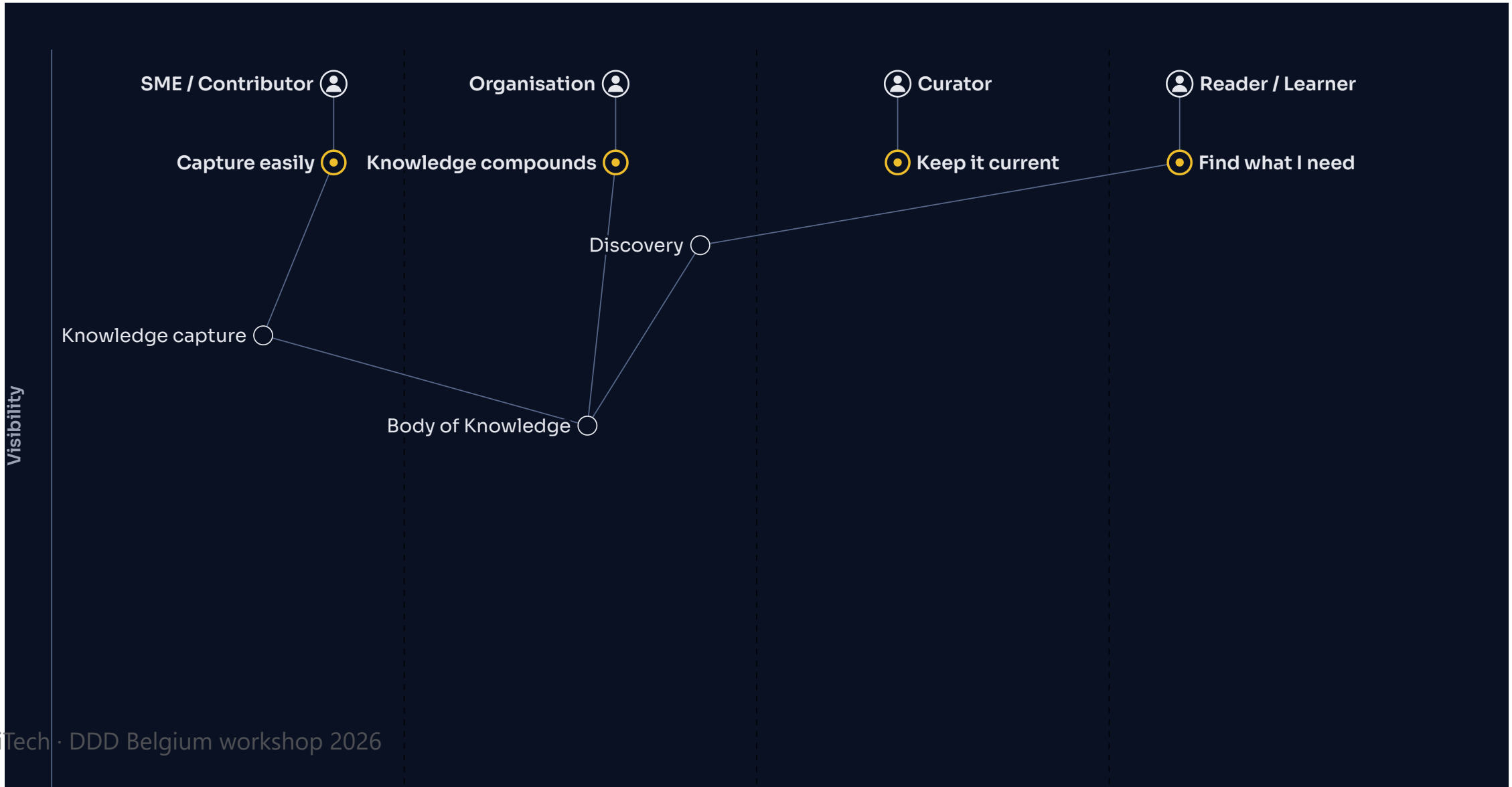
# Knowledge architecture · 4 of 12 — the roles



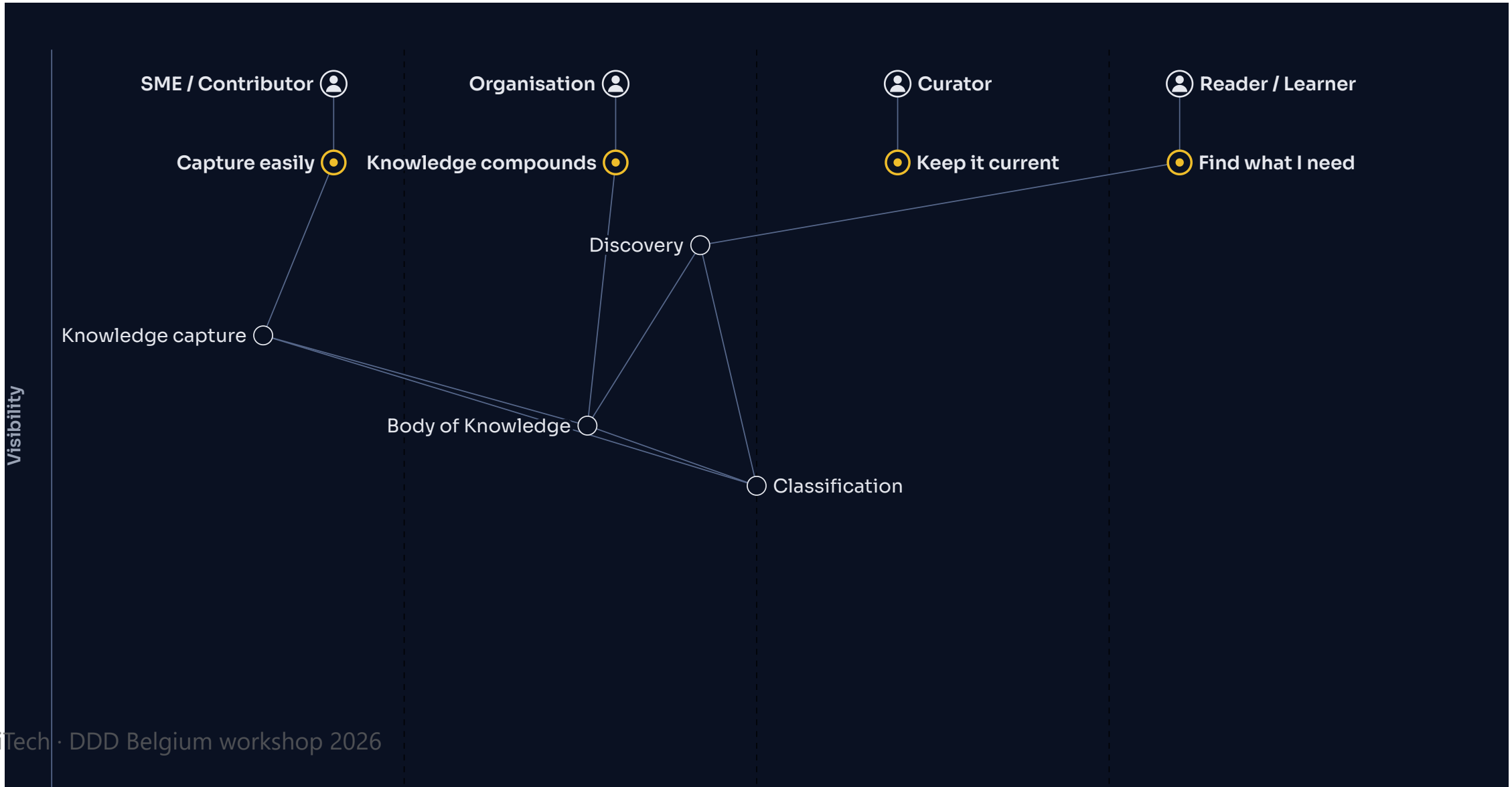
# Knowledge architecture · 5 of 12 — capture



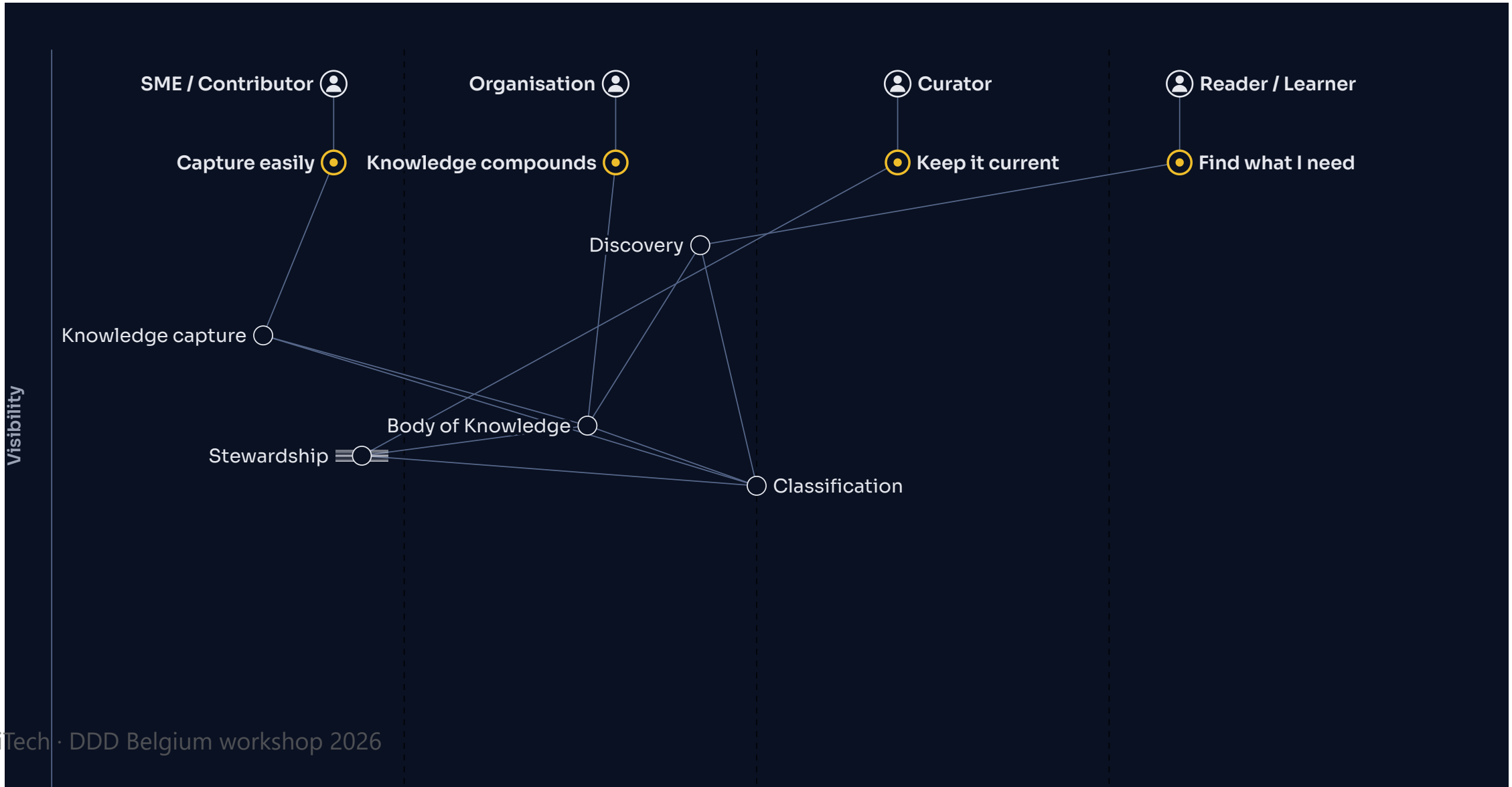
# Knowledge architecture · 6 of 12 — discovery



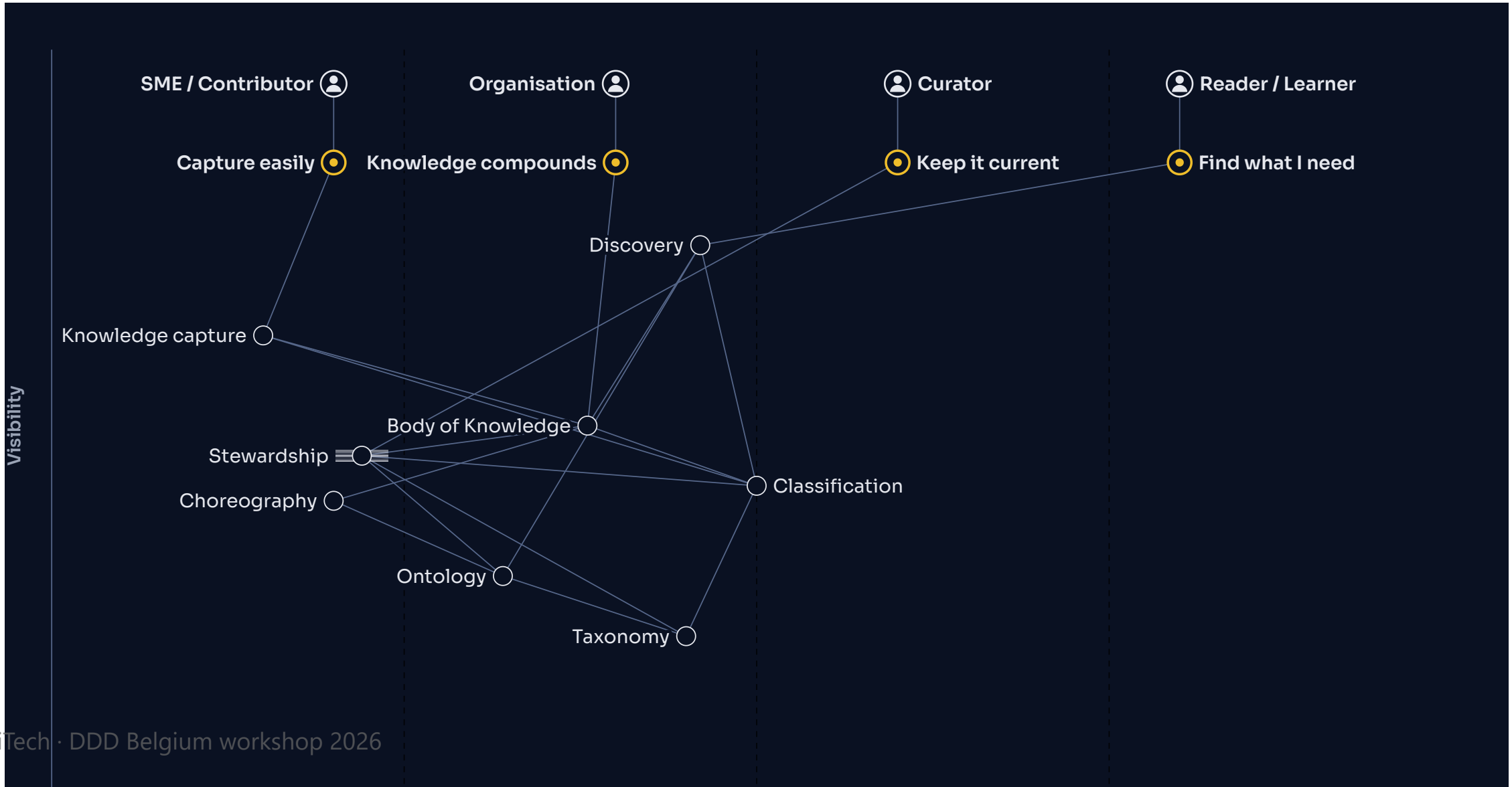
# Knowledge architecture · 7 of 12 — classification



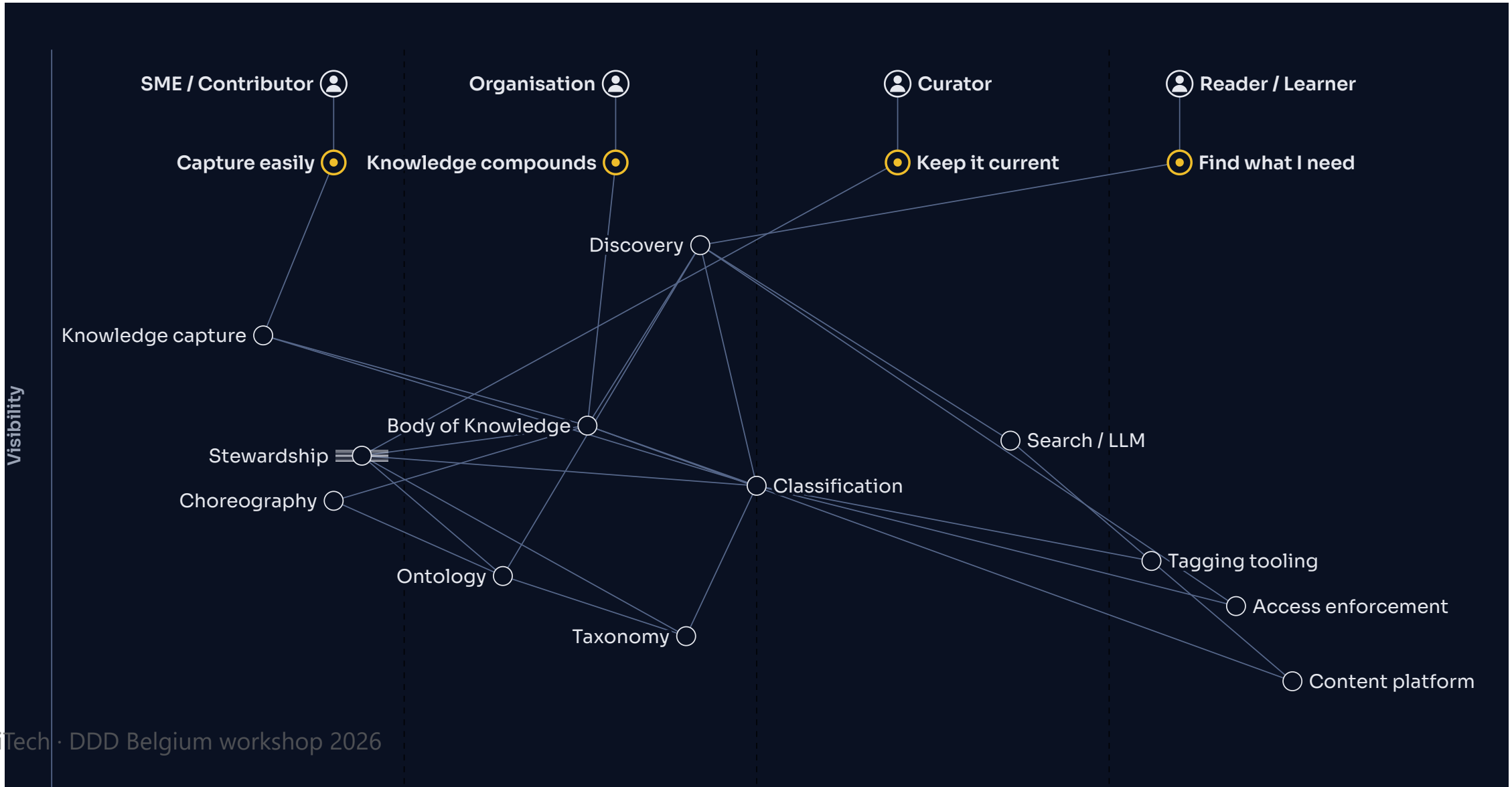
# Knowledge architecture · 8 of 12 — stewardship



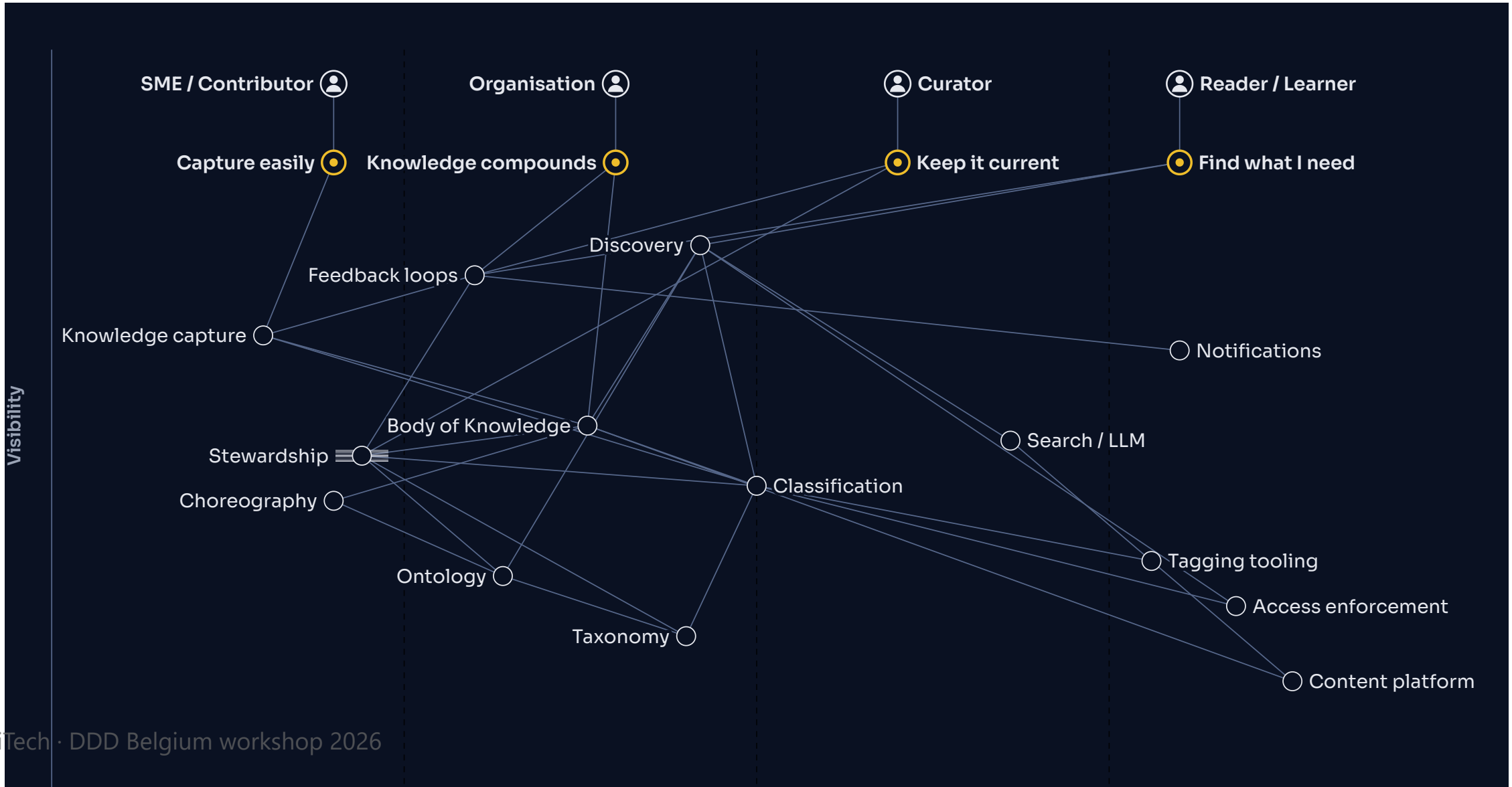
# Knowledge architecture · 9 of 12 — the stack



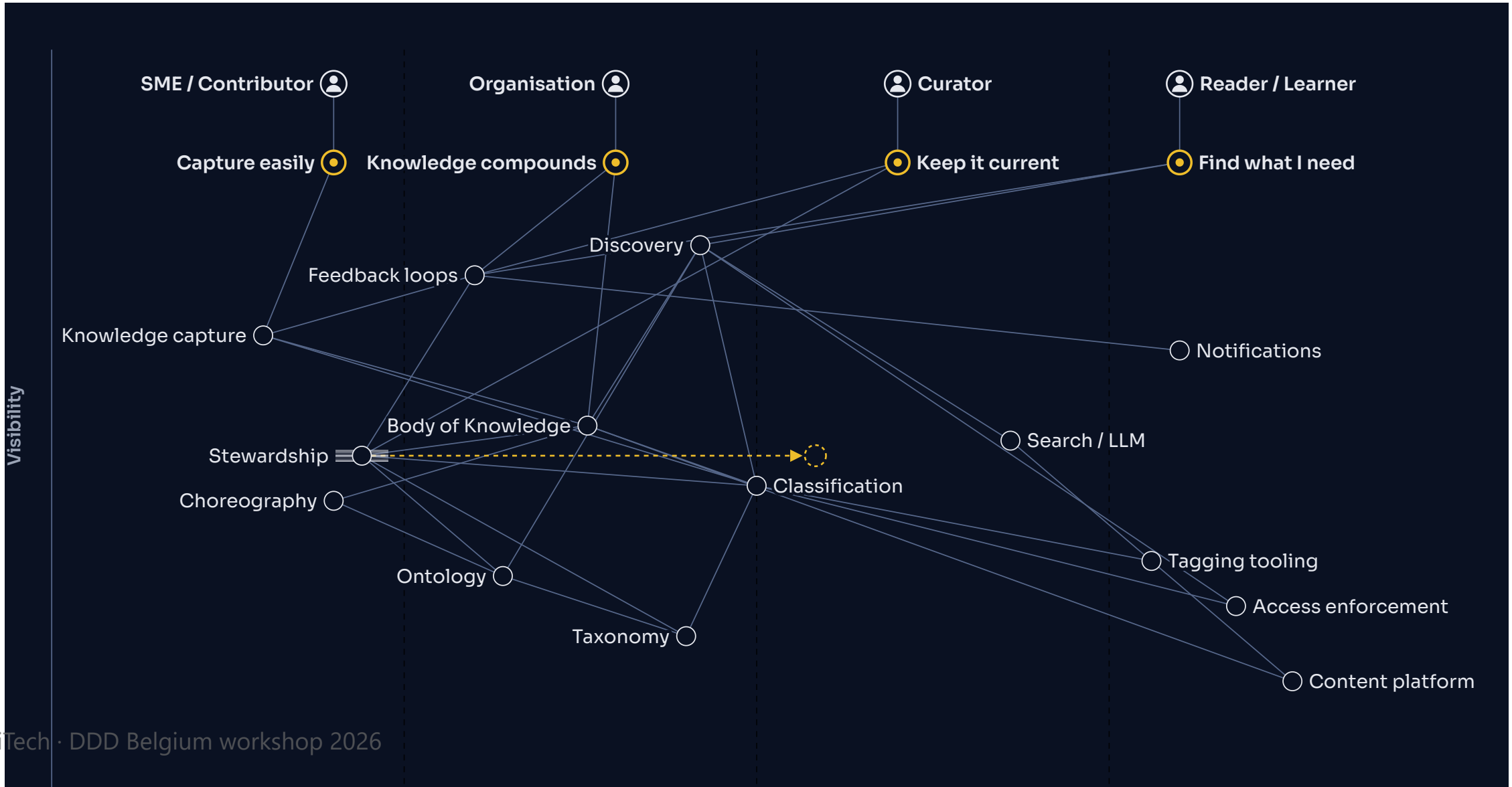
# Knowledge architecture · 10 of 12 — the platform



# Knowledge architecture · 11 of 12 — the loop



# Knowledge architecture · 12 of 12 — the move



Knowledge architecture

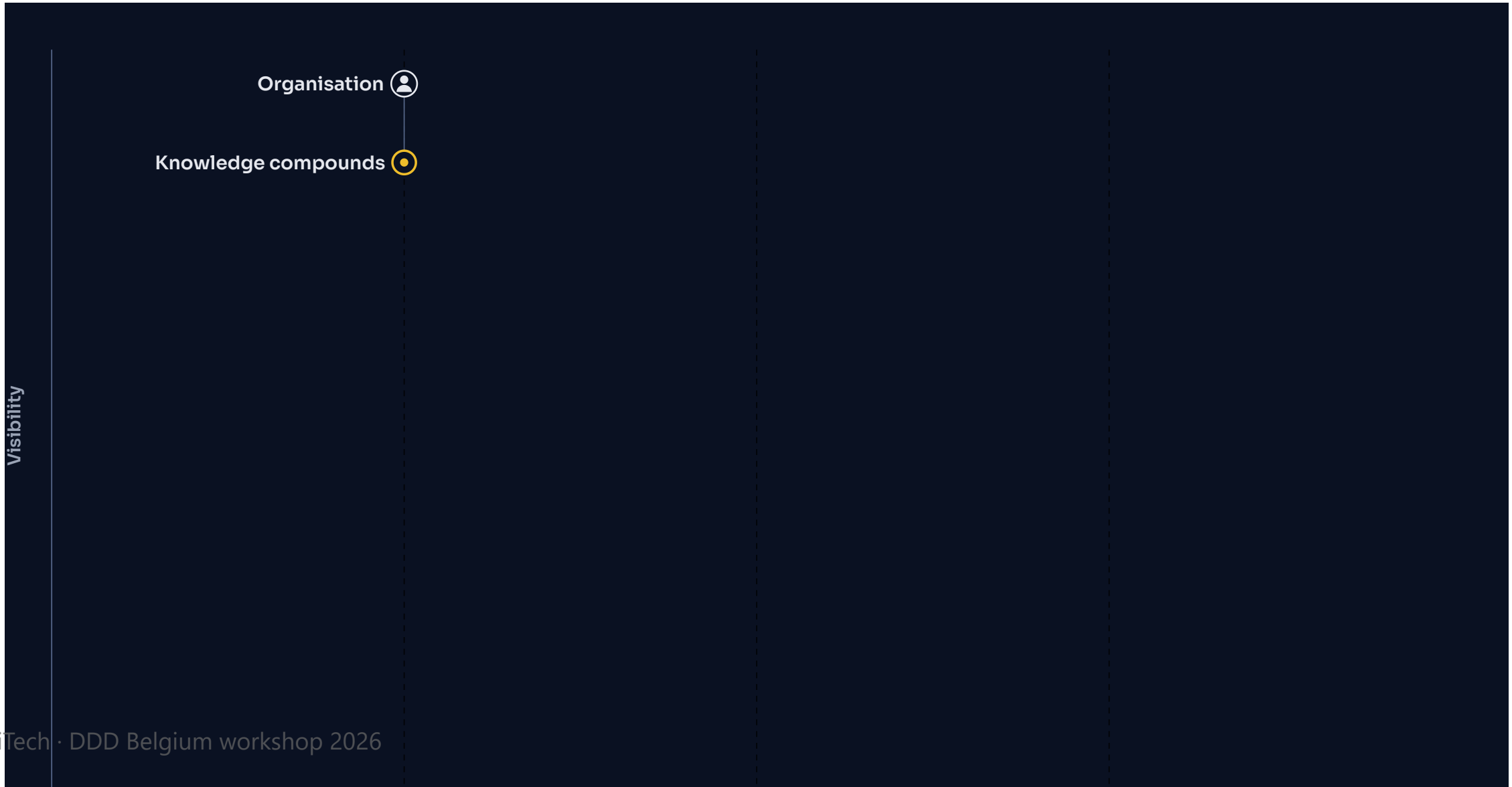
# A map with no question

- Twenty components, thirty lines
- Everything is true, nothing is decided
- Ask: which question does this map answer?
- Keep what changes the answer, fold the rest

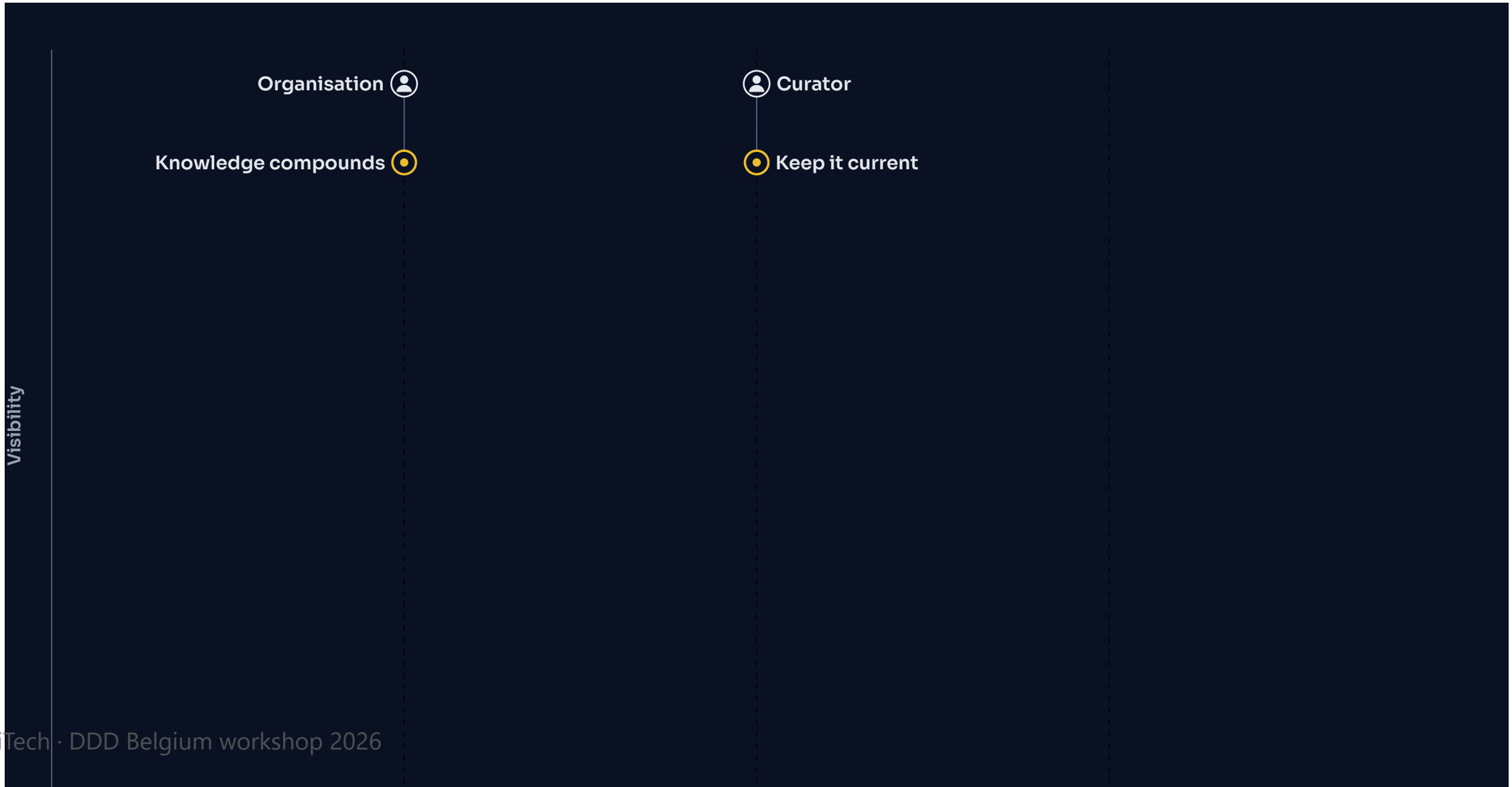
# Four ways to make a map readable

- One question per map: keep what changes the answer
- One map per user, when their needs really differ
- Map around the constraint: the chain to it, nothing else
- Fold a cluster into a submap
- Next: the constraint view, then a submap

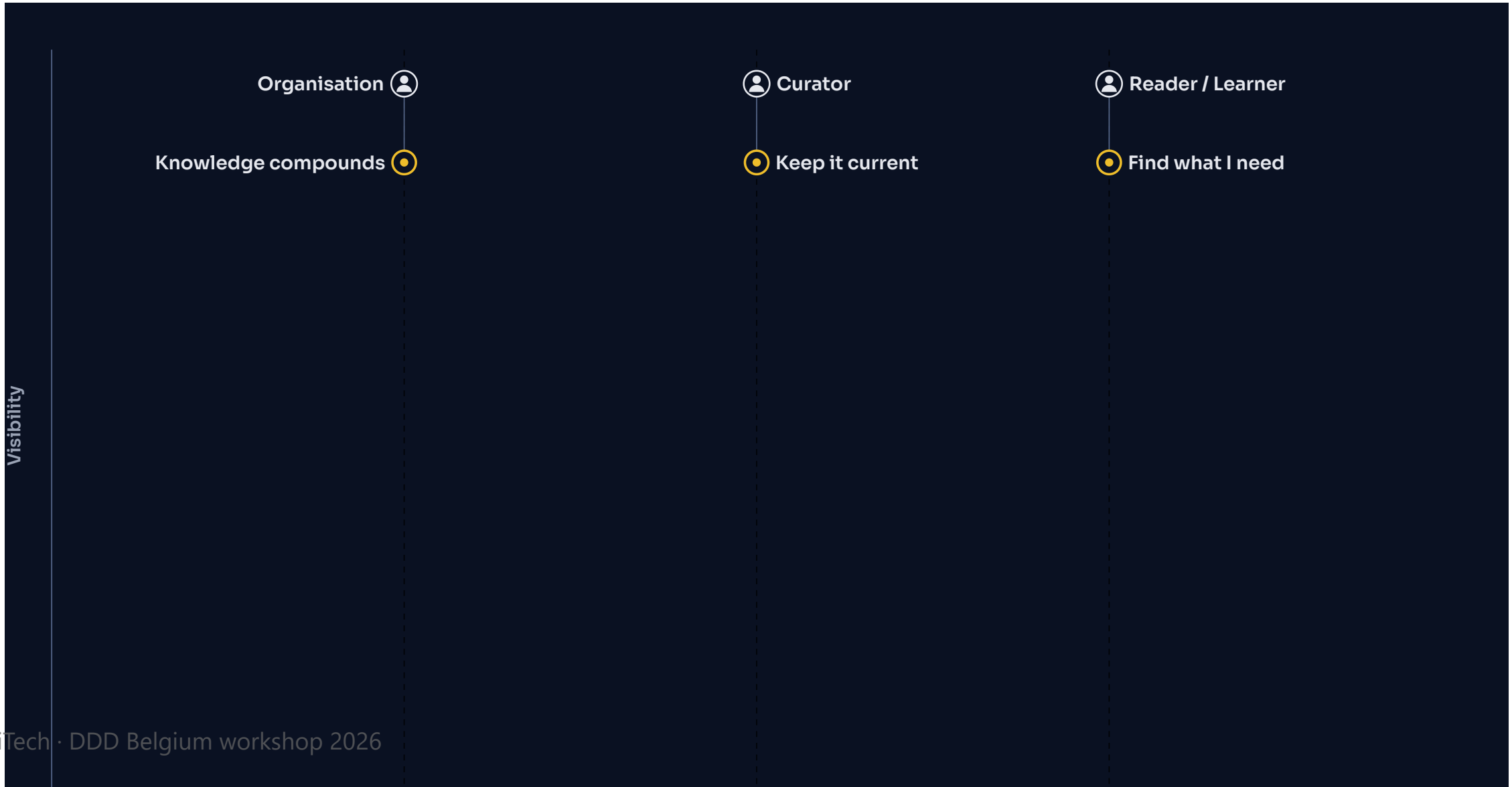
# The constraint view · 1 of 10 — viability



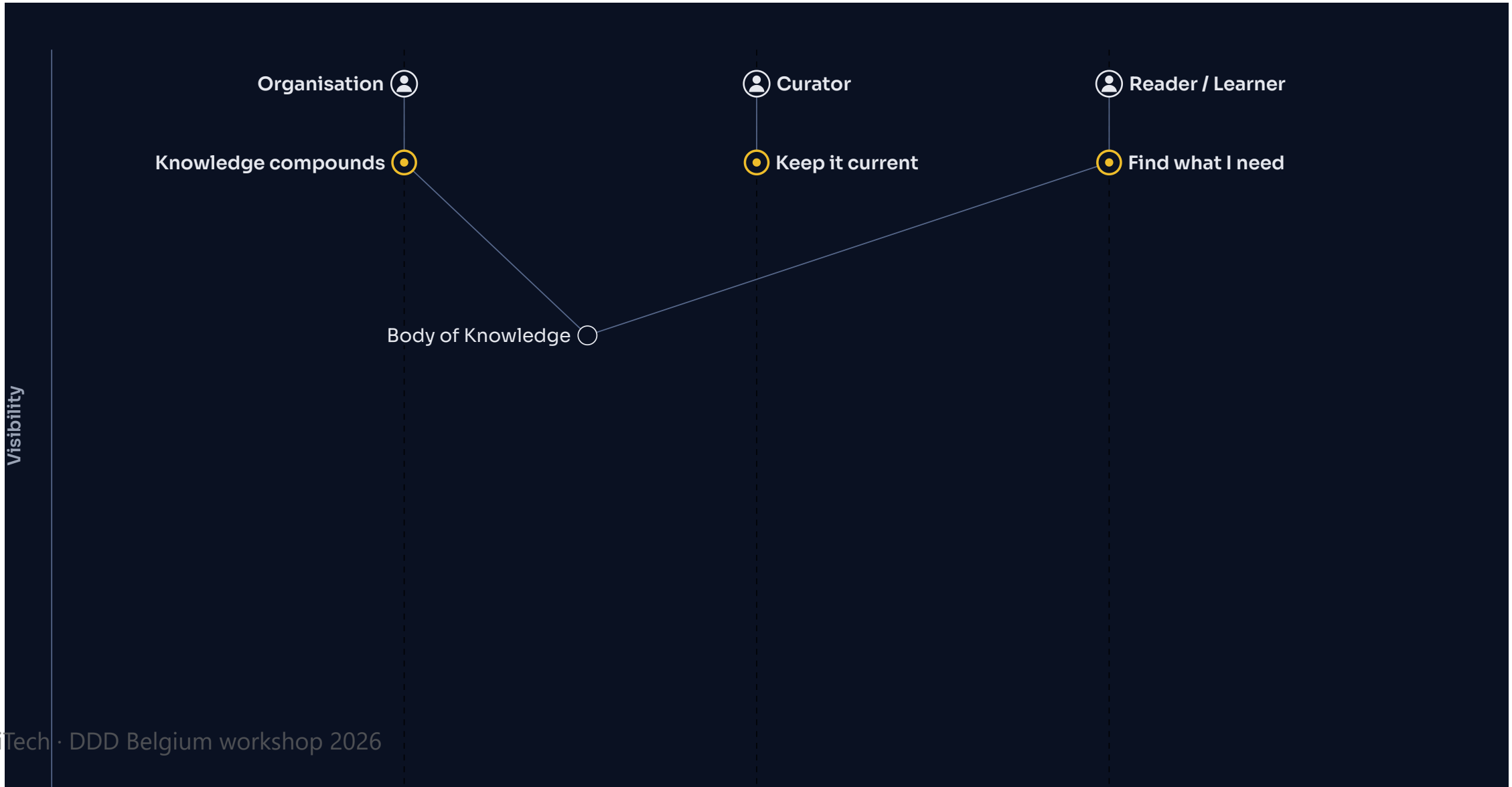
# The constraint view · 2 of 10 — the curator



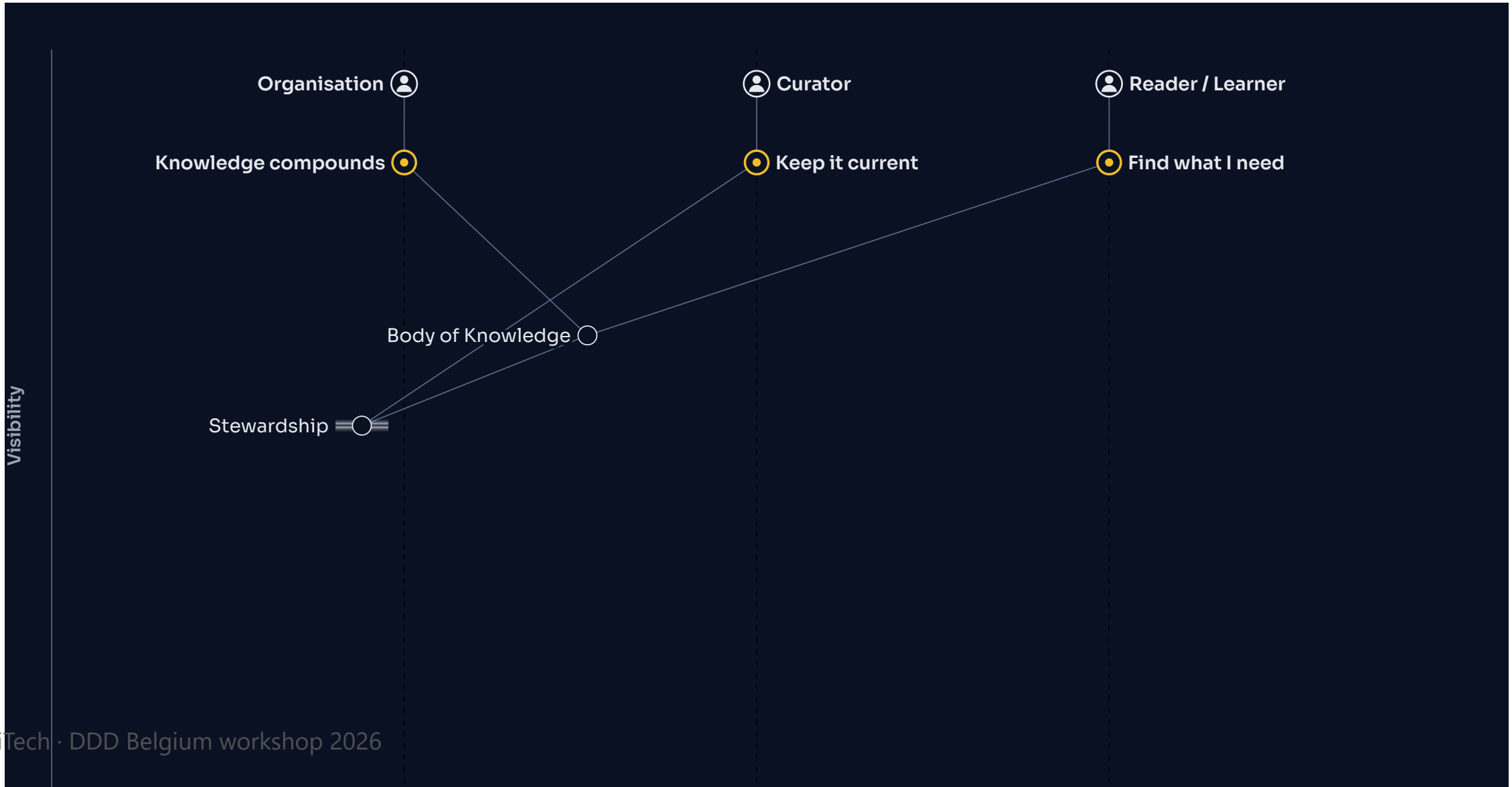
# The constraint view · 3 of 10 — the reader



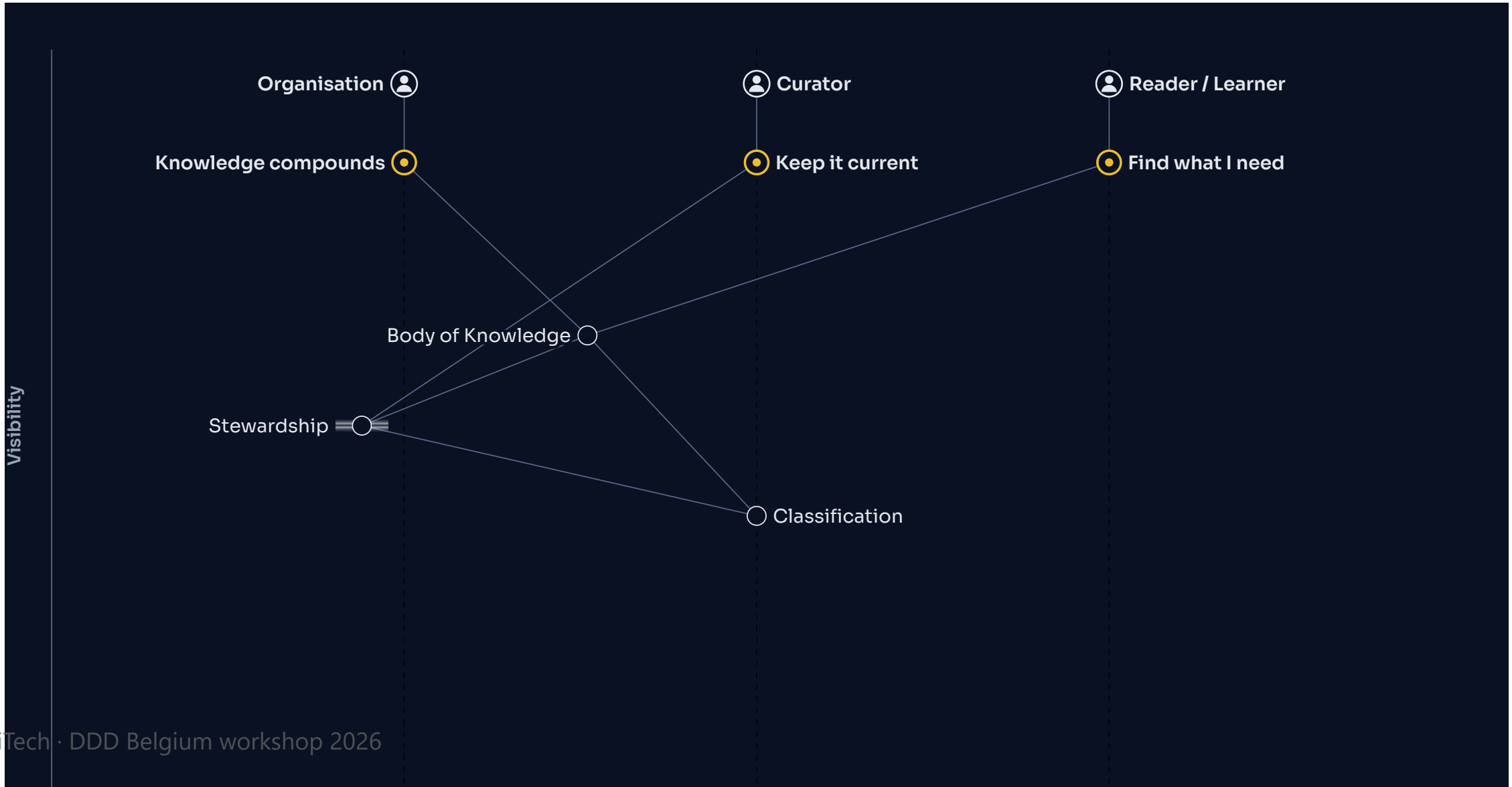
# The constraint view · 4 of 10 — the hub



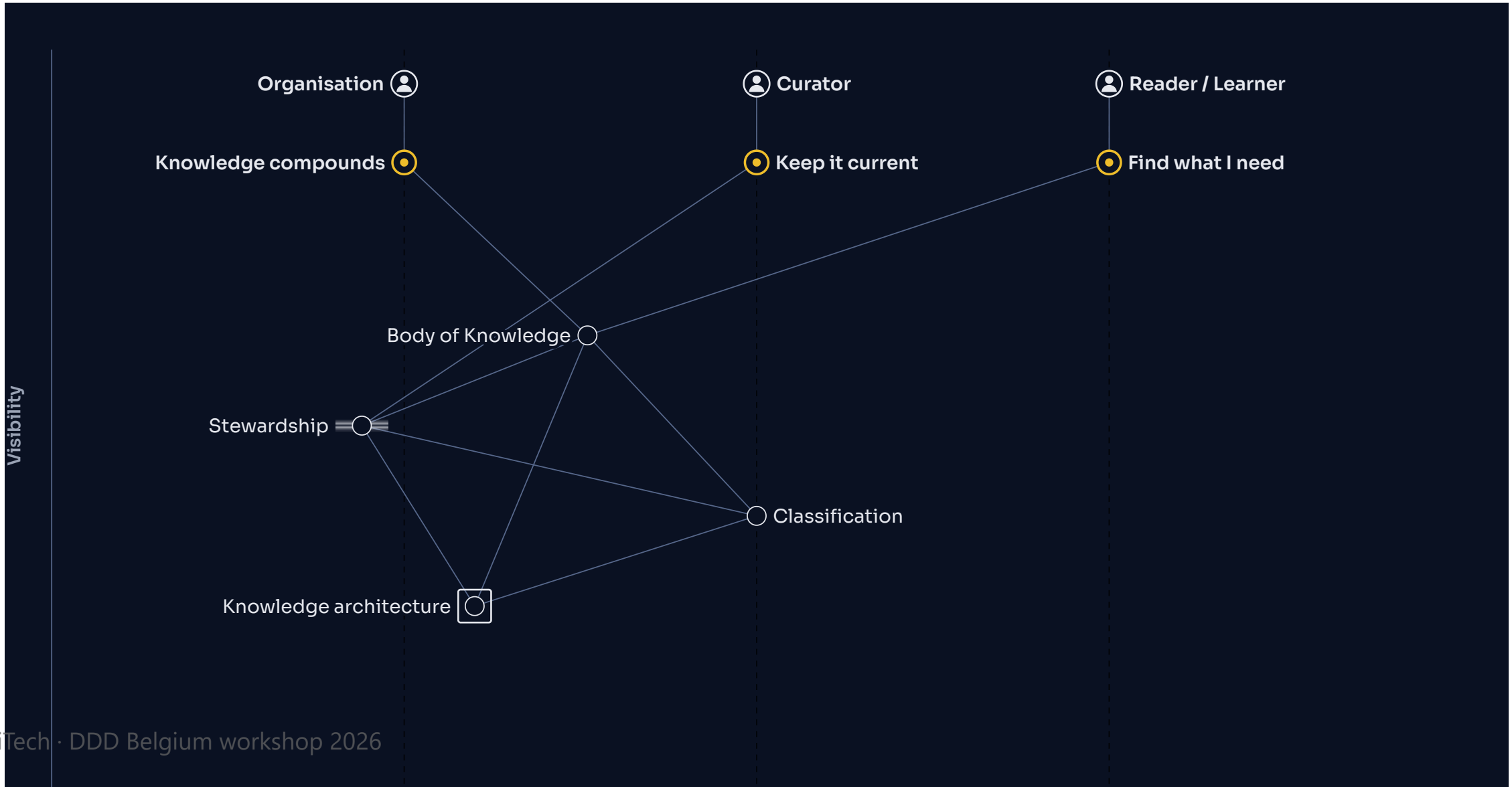
# The constraint view · 5 of 10 — the constraint



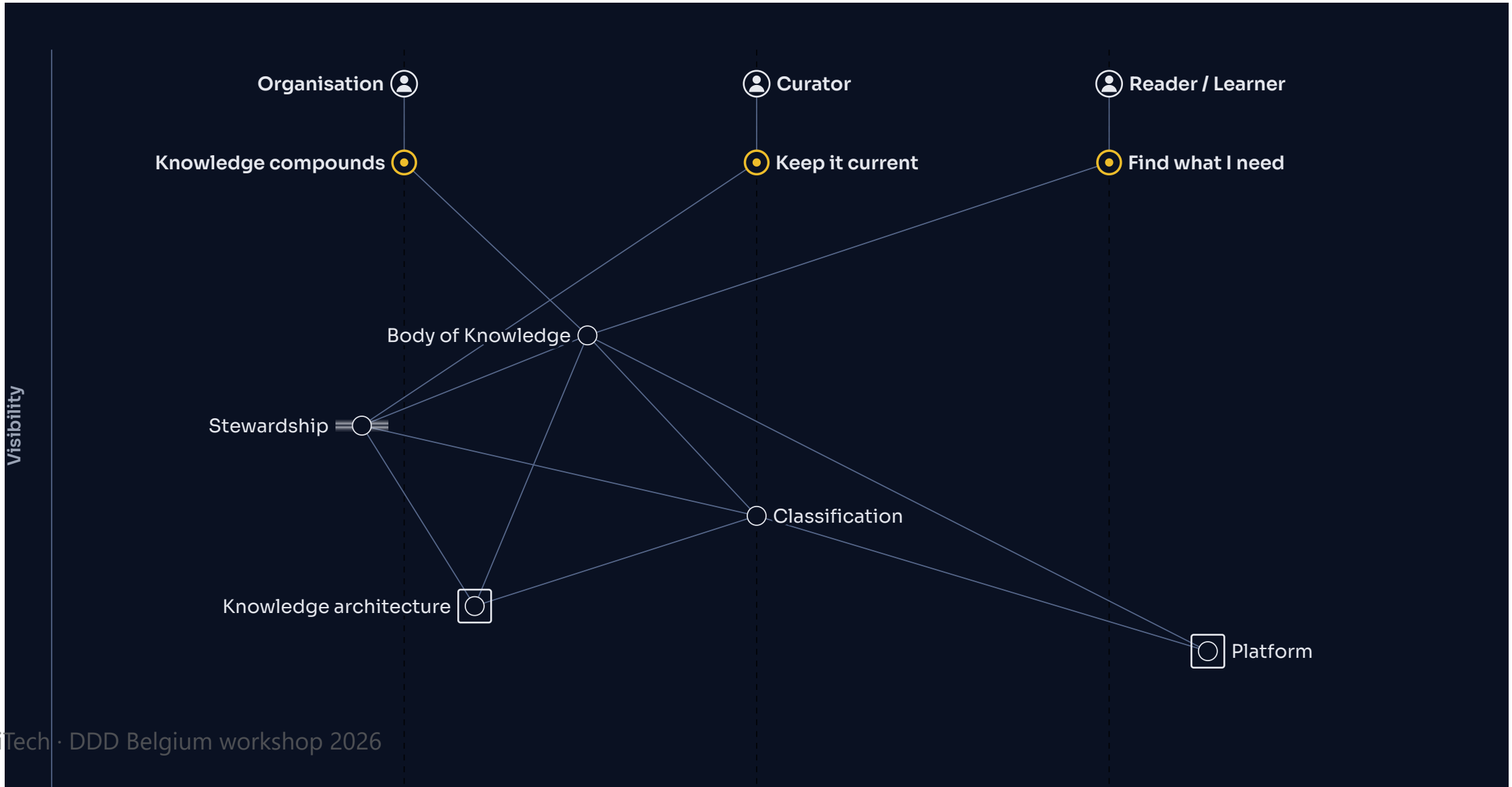
# The constraint view · 6 of 10 — classification



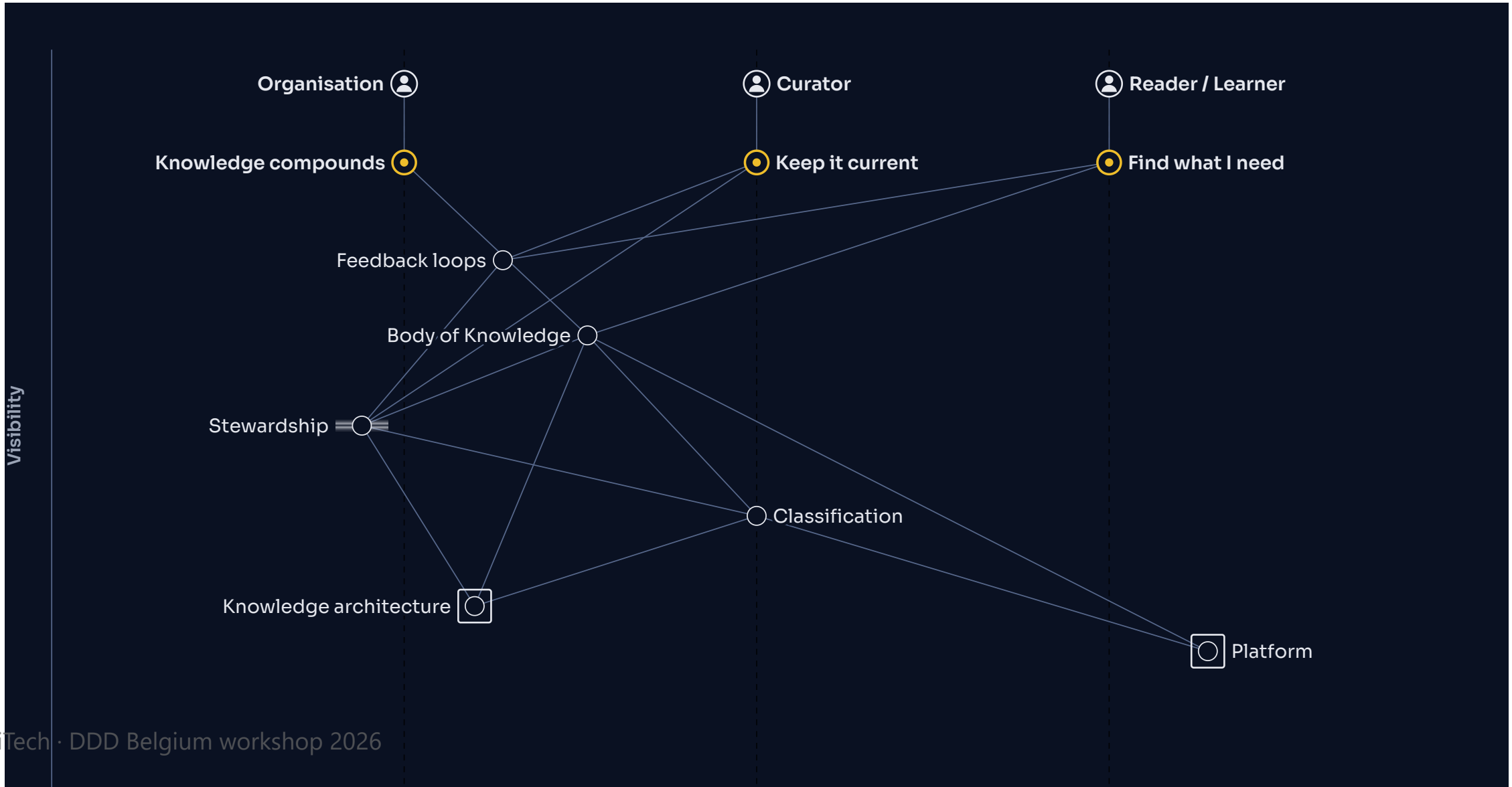
# The constraint view · 7 of 10 — a submap



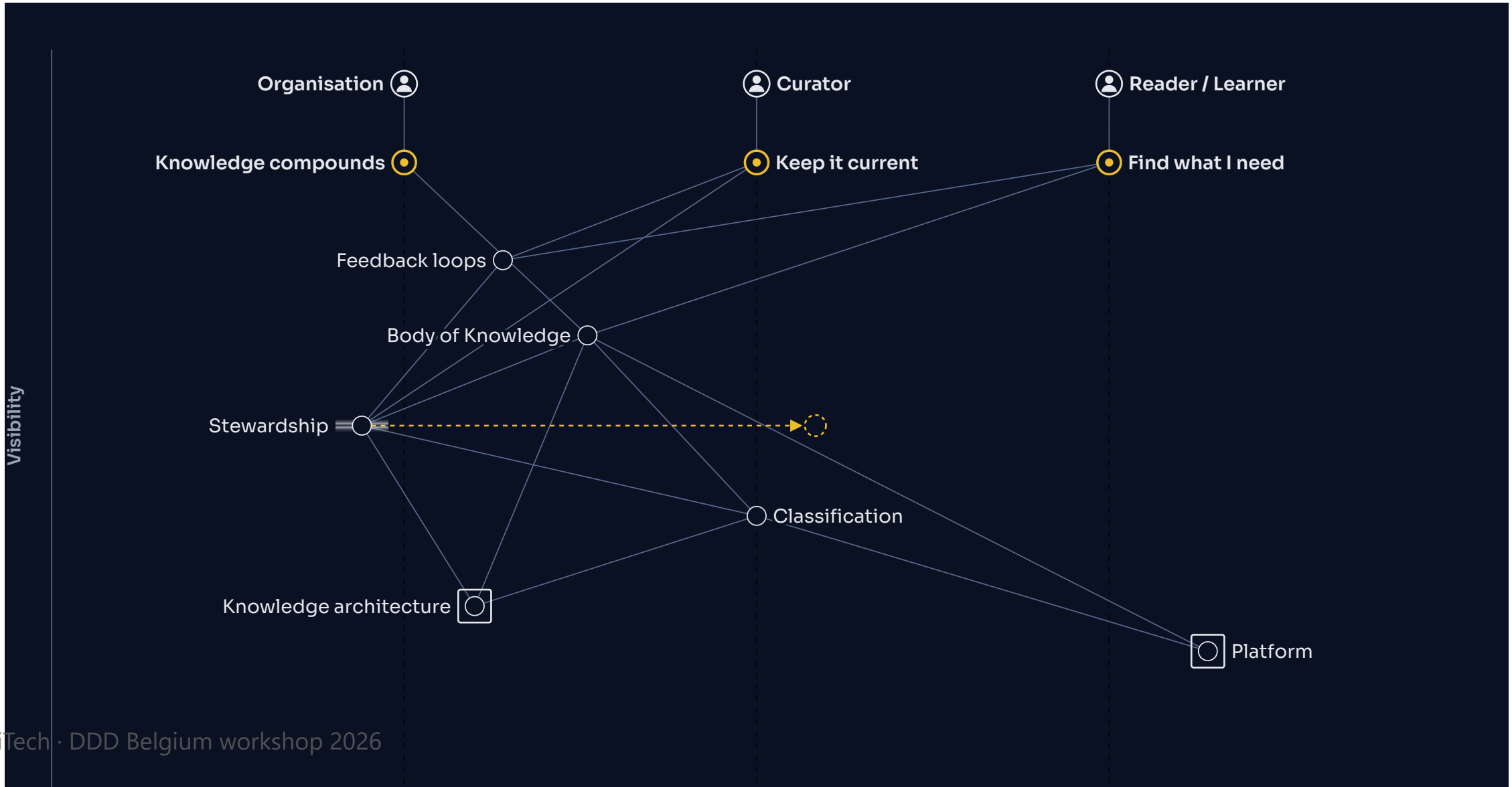
# The constraint view · 8 of 10 — the platform



# The constraint view · 9 of 10 — the loop



# The constraint view · 10 of 10 — elevate

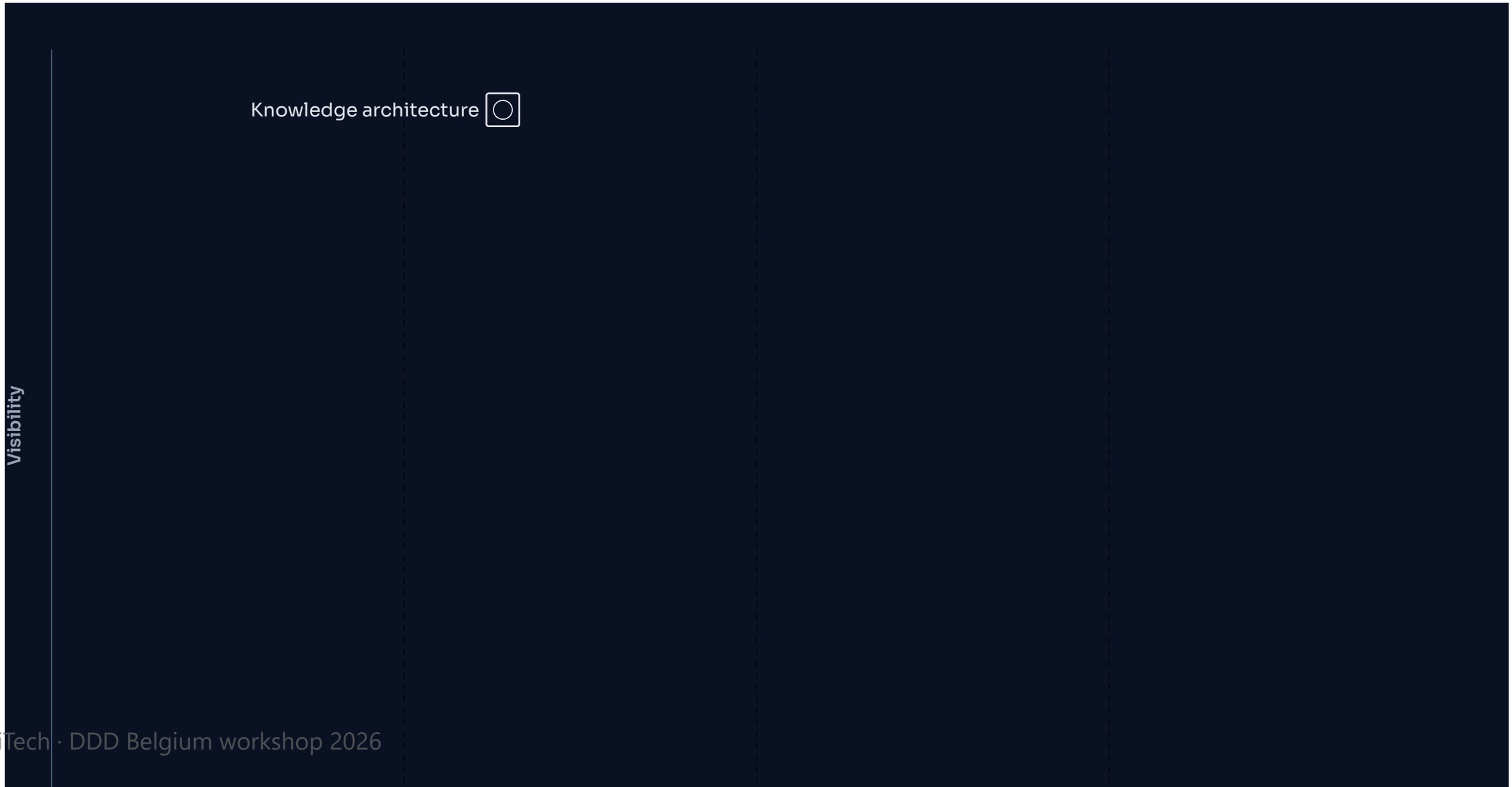


Knowledge architecture · the constraint view

# Stewardship holds the flow

- Every need runs through stewardship
- Exploit: curate what readers ask for; agents pre-tag
- Subordinate: pace capture to curation
- Elevate: shared stewardship, the evolve arrow

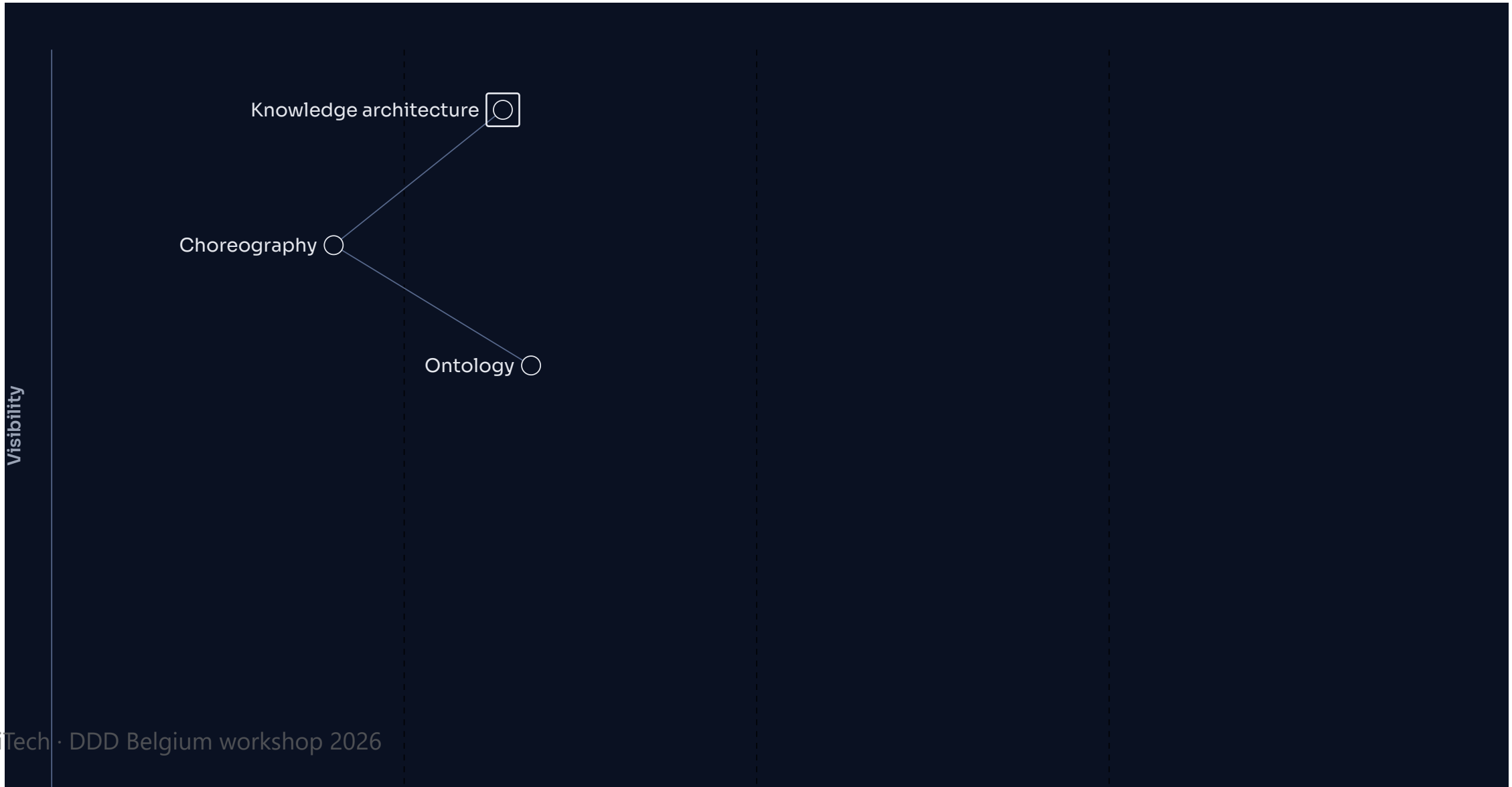
# Submap · knowledge architecture · 1 of 7 — the submap



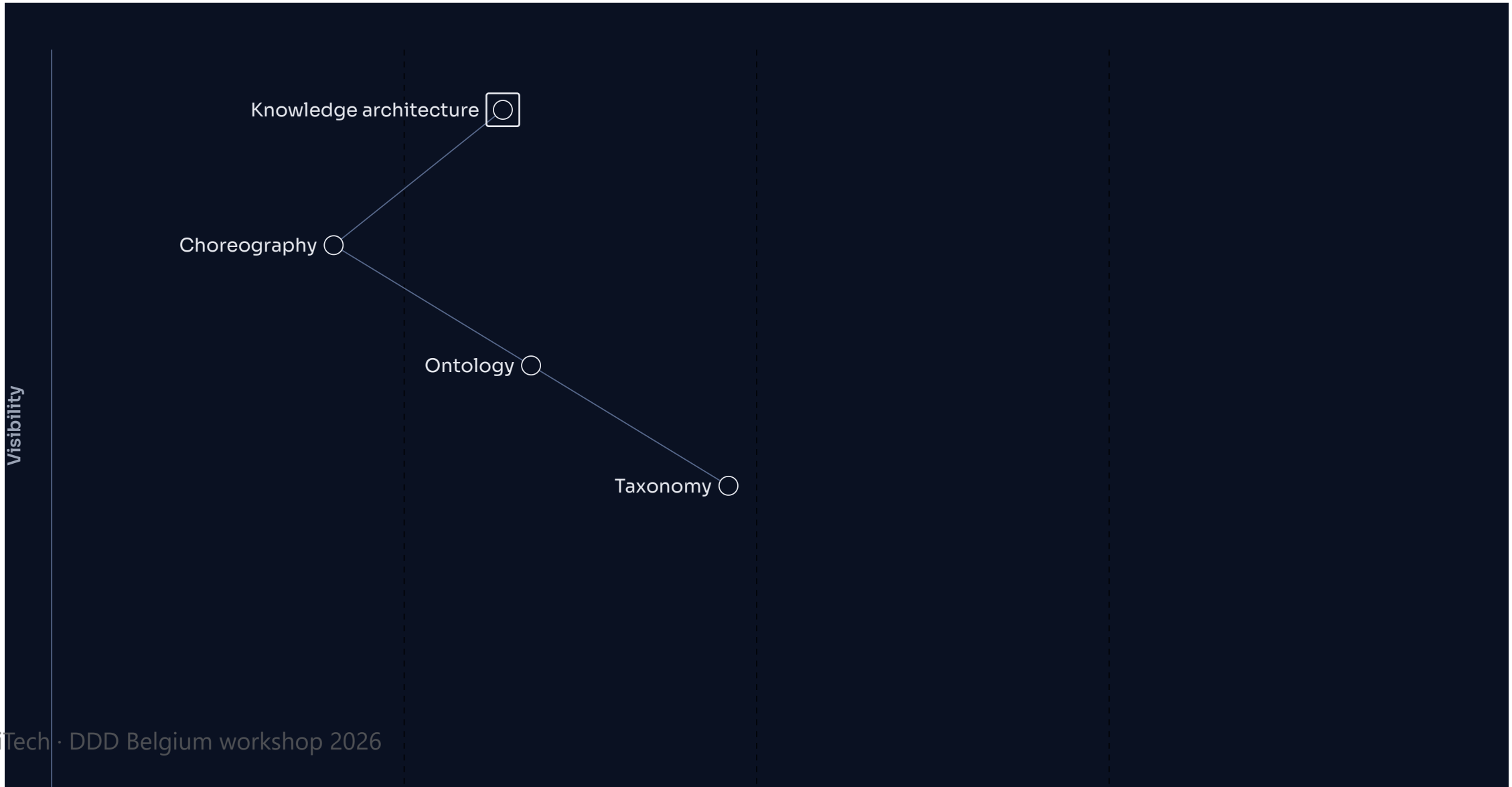
# Submap · knowledge architecture · 2 of 7 — behaviour



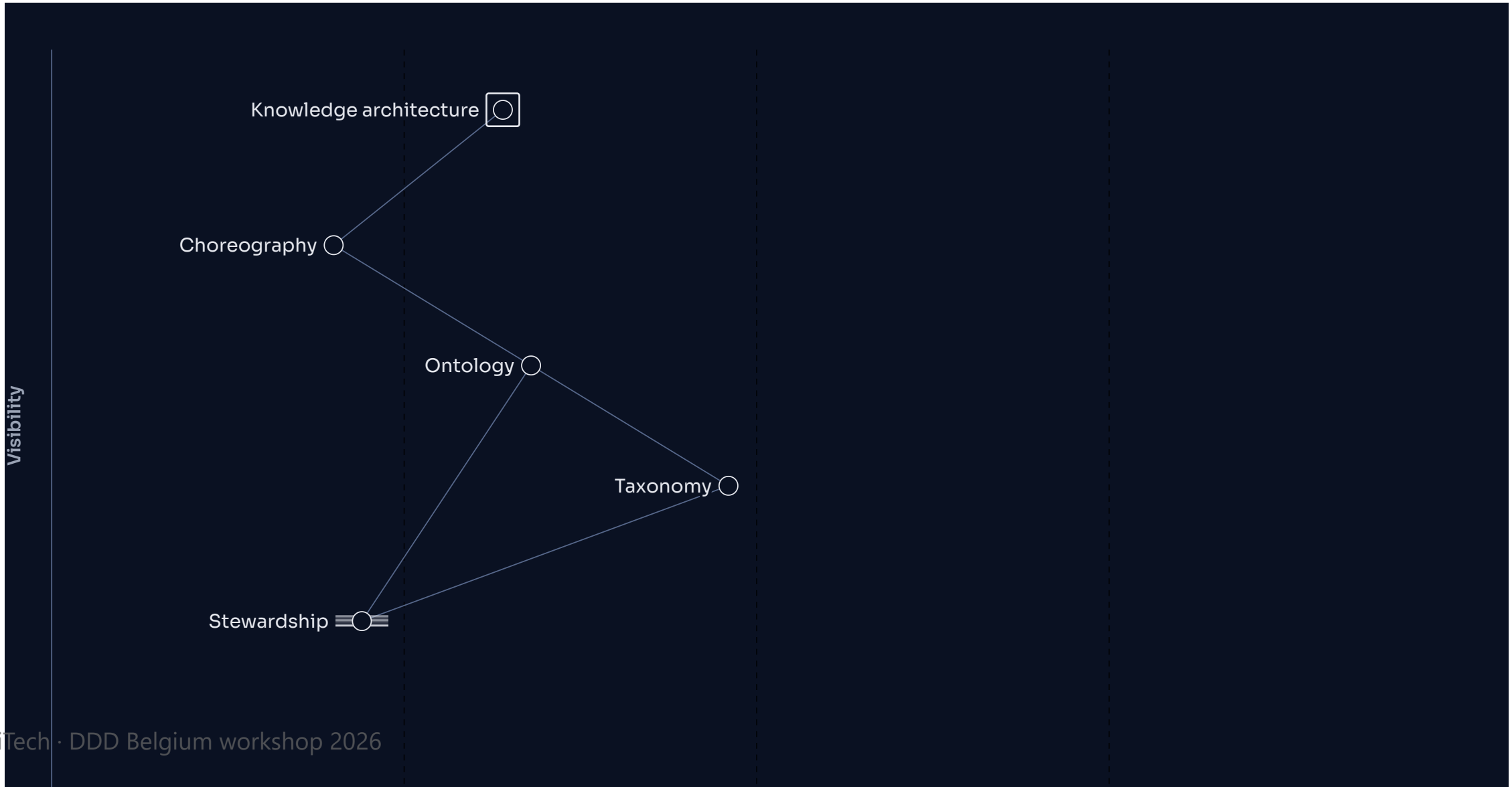
# Submap · knowledge architecture · 3 of 7 — meaning



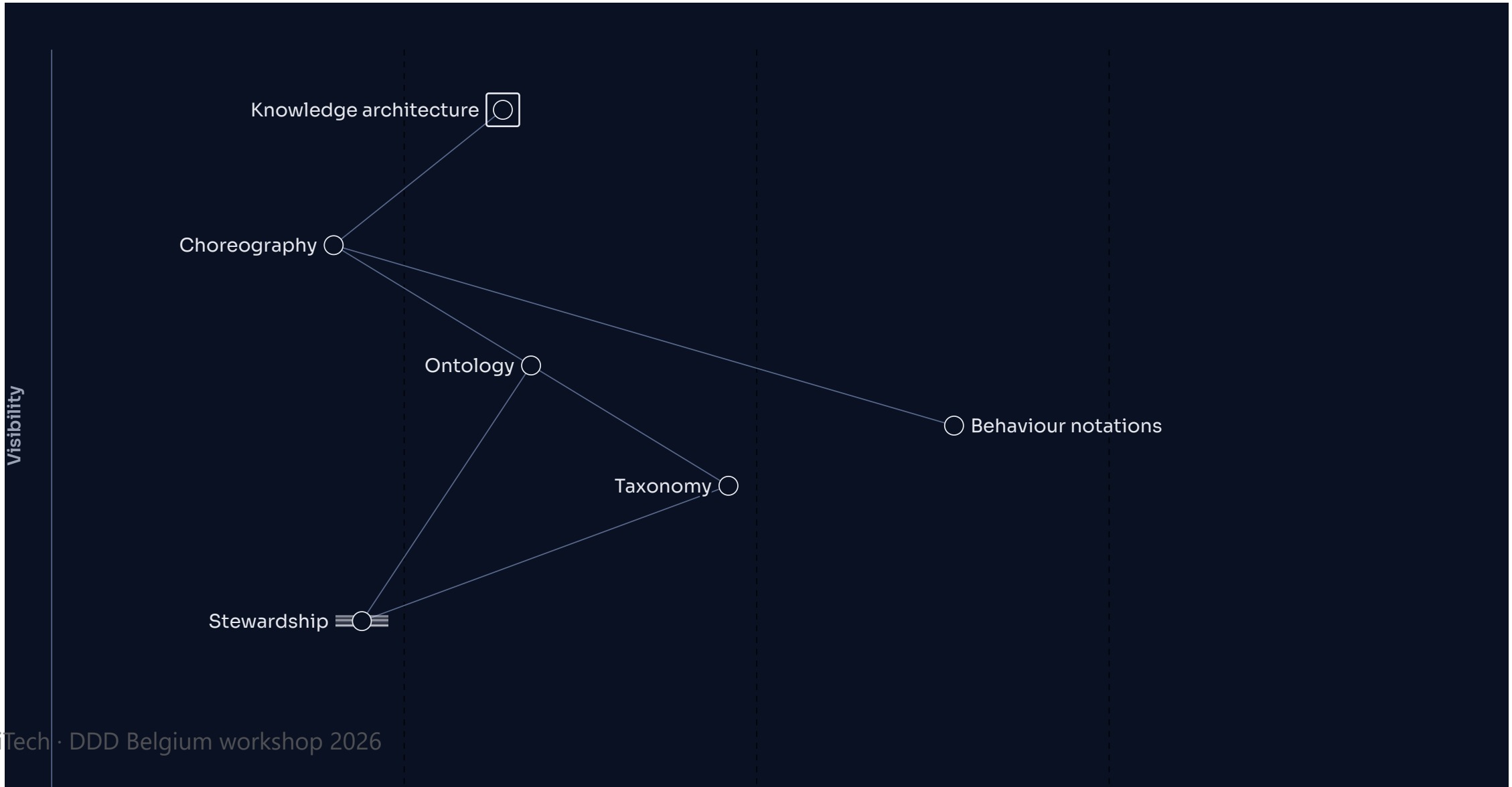
# Submap · knowledge architecture · 4 of 7 — structure



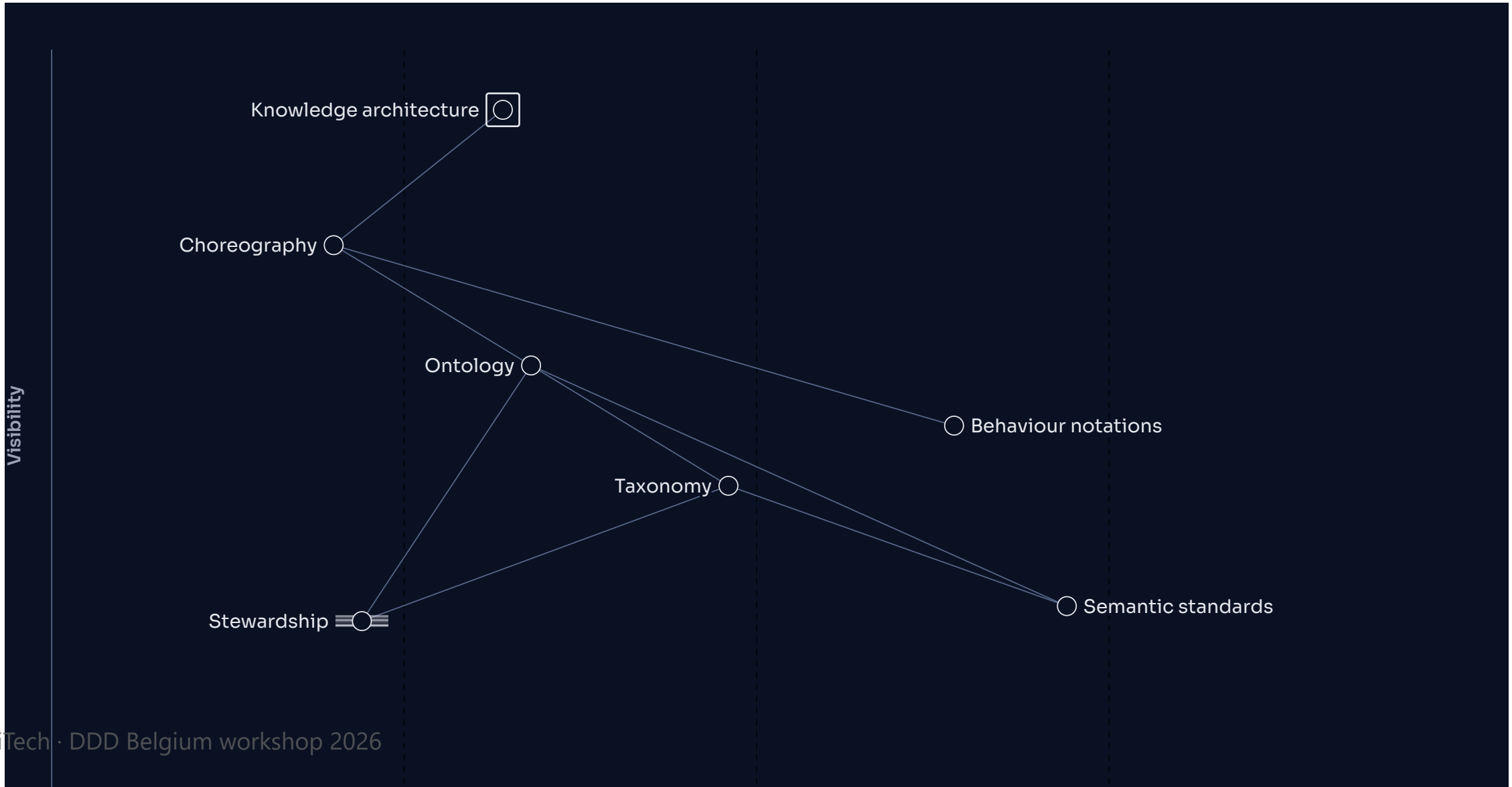
# Submap · knowledge architecture · 5 of 7 — the steward



# Submap · knowledge architecture · 6 of 7 — notations



# Submap · knowledge architecture · 7 of 7 — standards



Knowledge architecture · submap

# Structure, meaning, behaviour

- Taxonomy: what exists and how it is grouped
- Ontology: what things and their relations mean
- Behaviour: what happens over time
- Users feel behaviour, yet it is the least evolved
- We polish taxonomy because it is easy to see
- Stewardship looks after all three layers

## Knowledge architecture · the street light

# Searching where it is easy




"Why look here?" "Because the light is better." Storm P, 1926.

Exercise 5 · group work

# Exercise 5: simplify and move

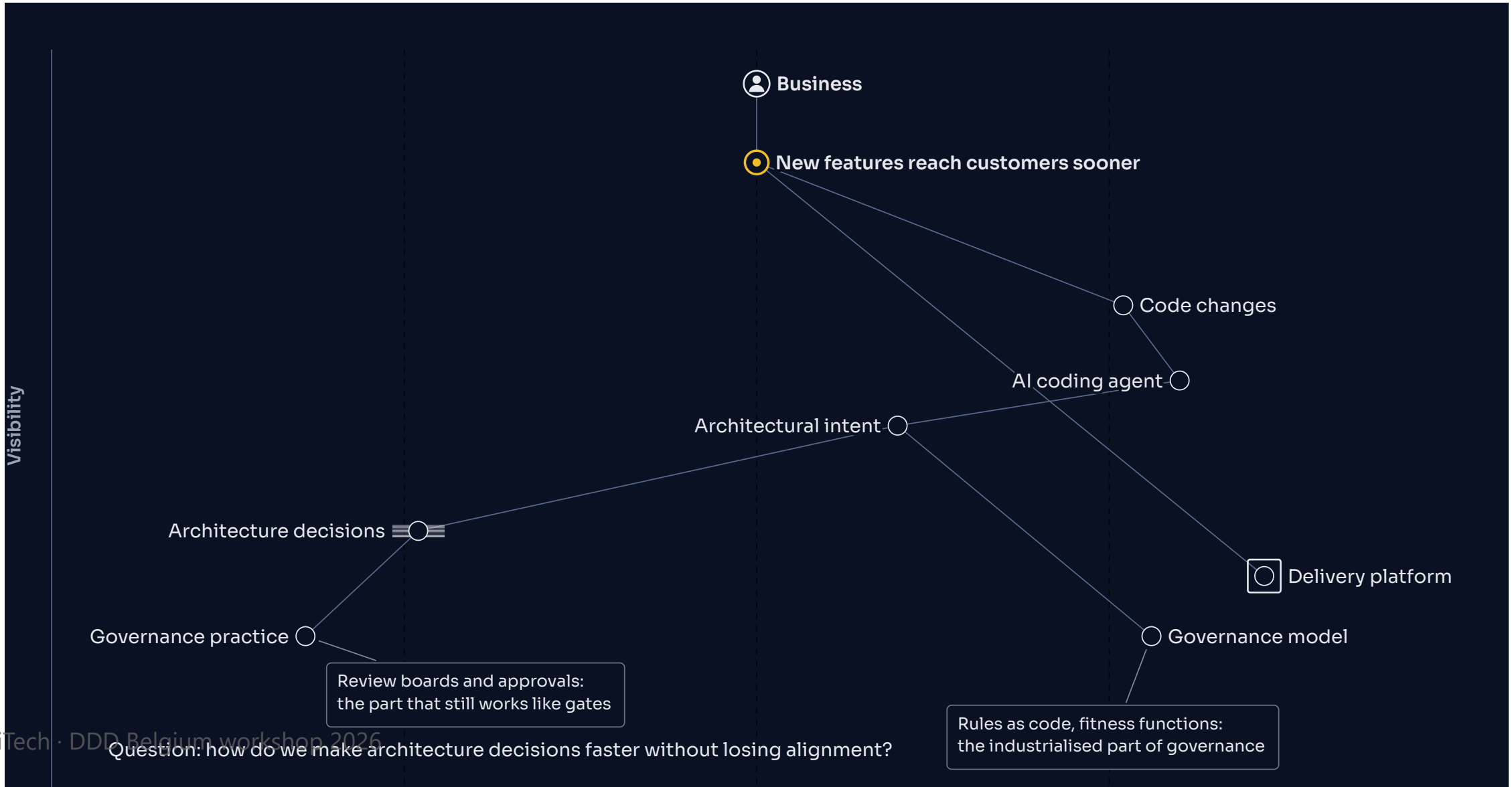
- **First: Cut it down**
  - Write the question your map answers
  - Remove what doesn't change the answer
  - Aim for 12 components or fewer
- **Then: Make a move**
  - Draw one strategic move on your simplified map
  - Natural evolution (dashed) or intended move (solid arrow)
  - Pick a gameplay pattern that speeds it up or holds it back

Exercise 5 · back together

# Exercise 5: discuss your simplification and move

- Is the story clearer after simplifying?
- Which component did you move, and where does it end up?
- Who gains from that move? Who loses?
- Would you dare to make that move this quarter?

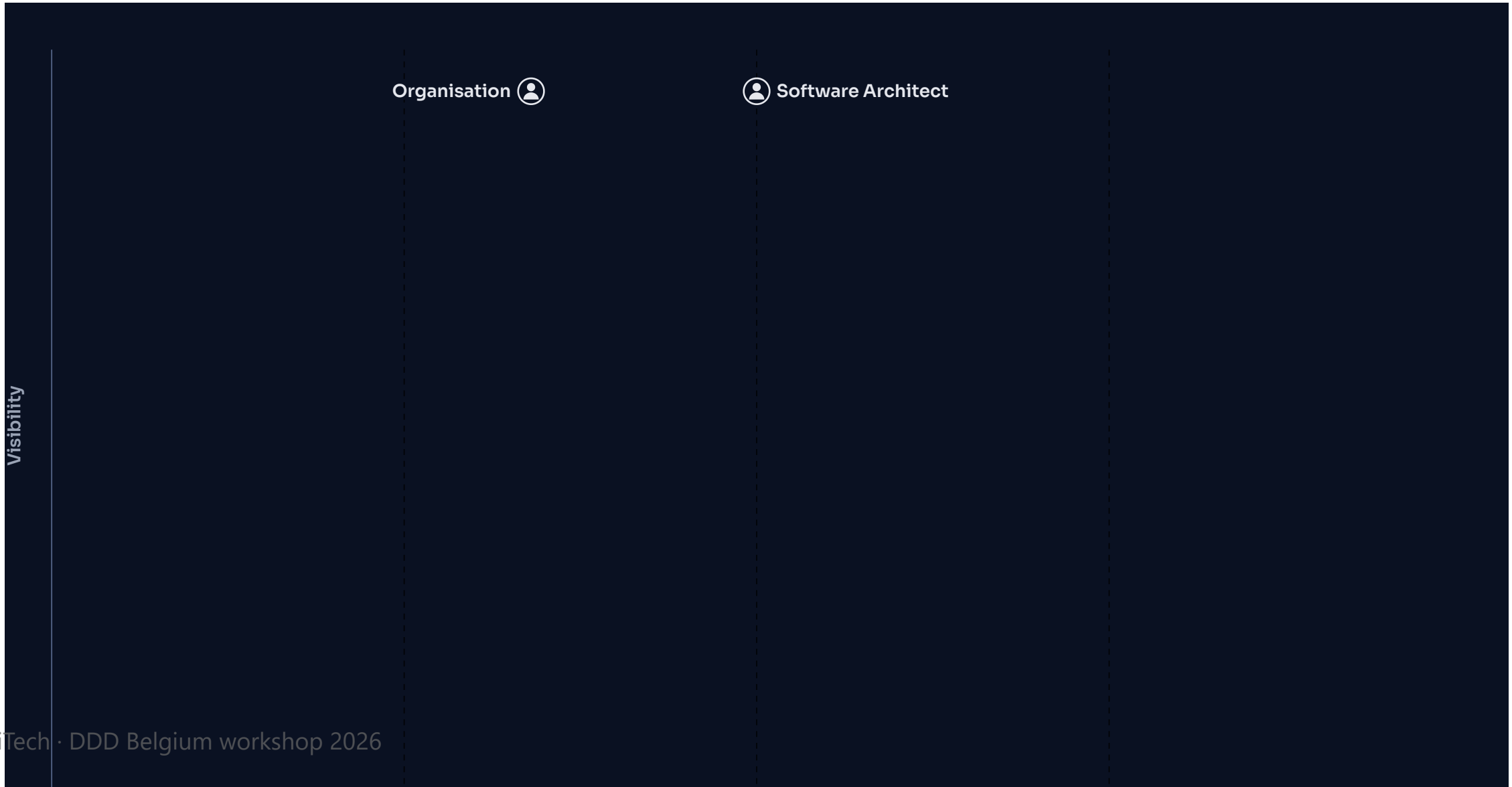
# Exercise 5 · one way to map it



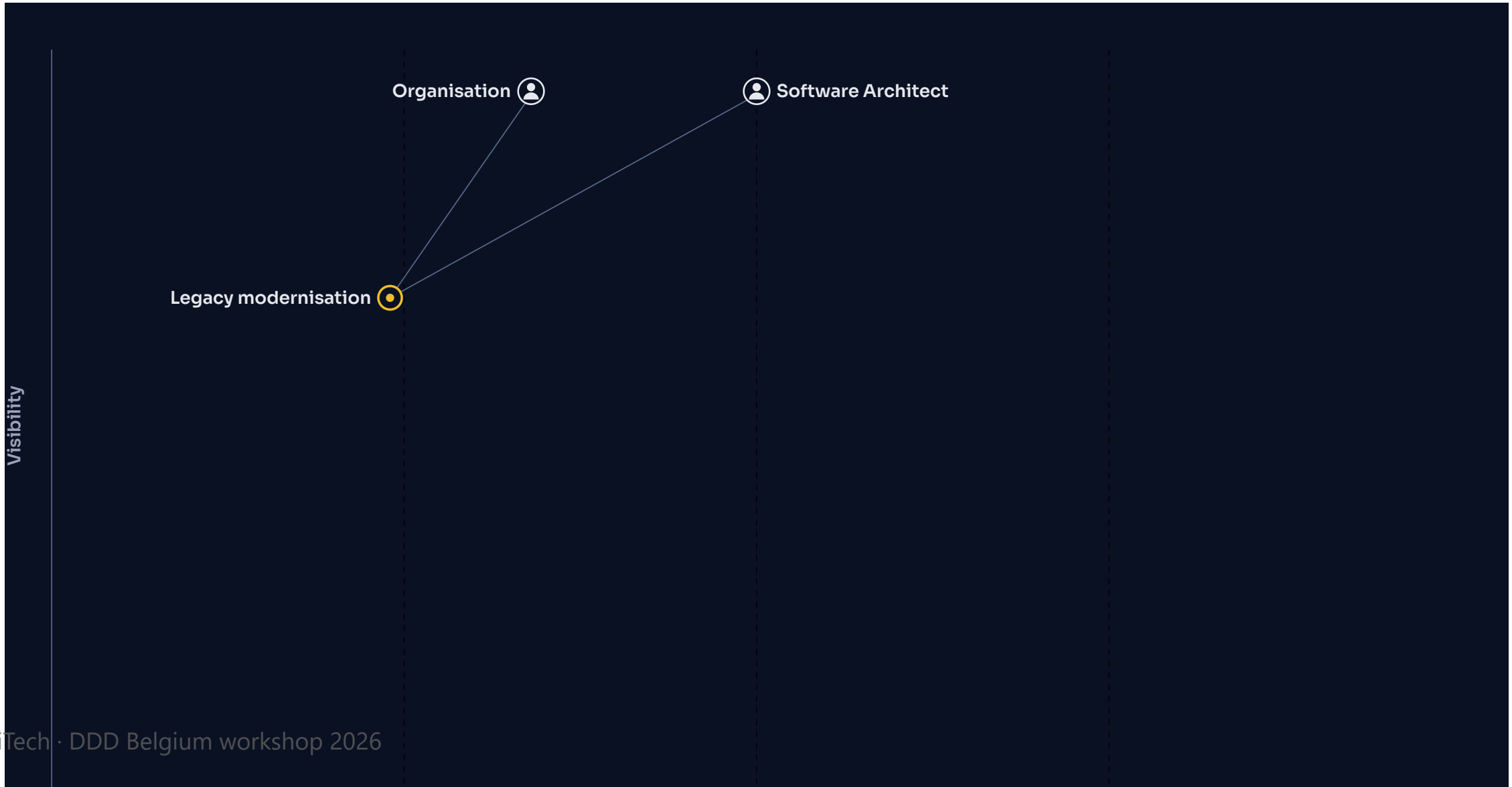
06

# A real map

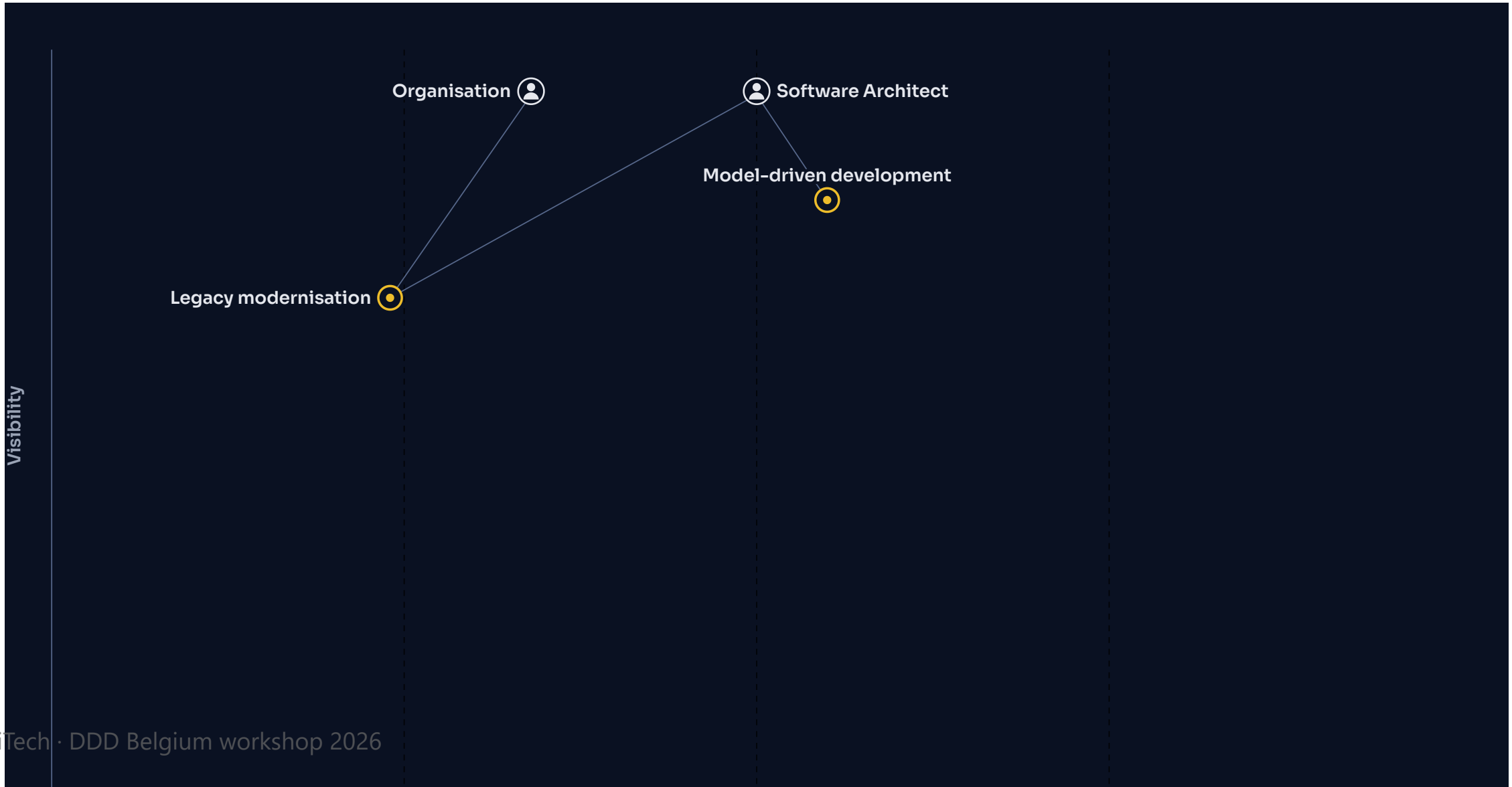
# EM tooling · 1 of 13 — the users



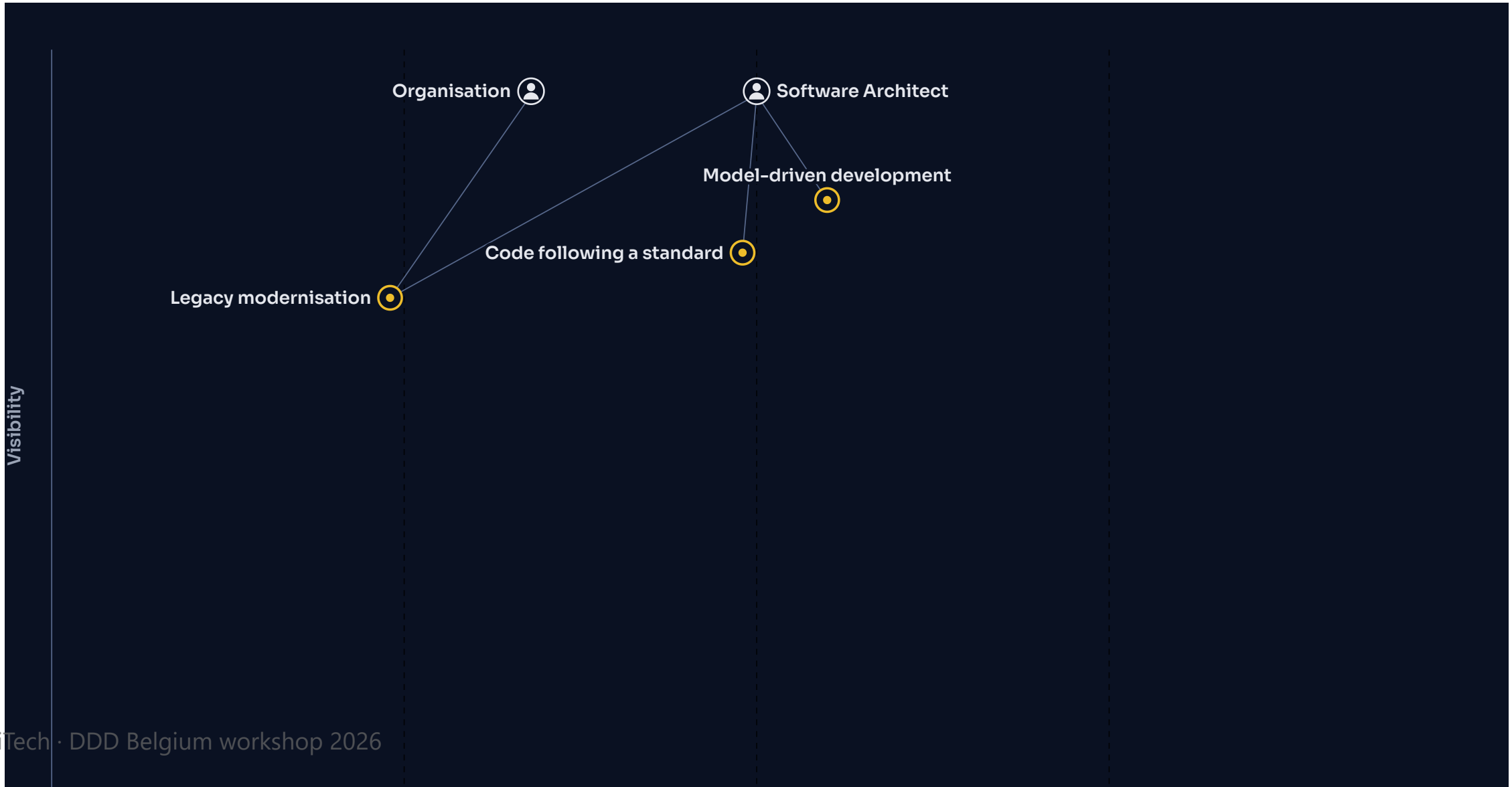
# EM tooling · 2 of 13 — the stakes



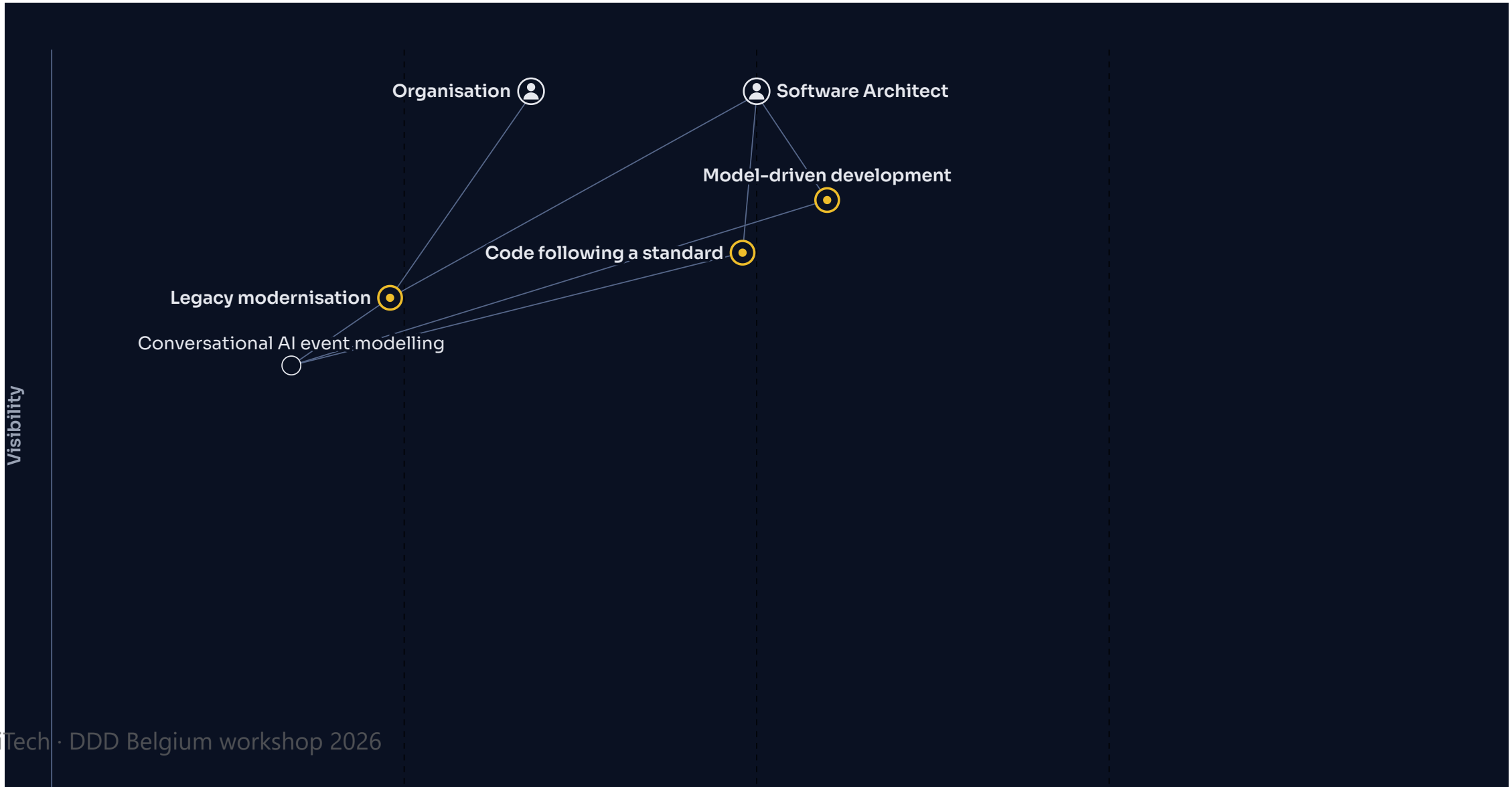
# EM tooling · 3 of 13 — the practice



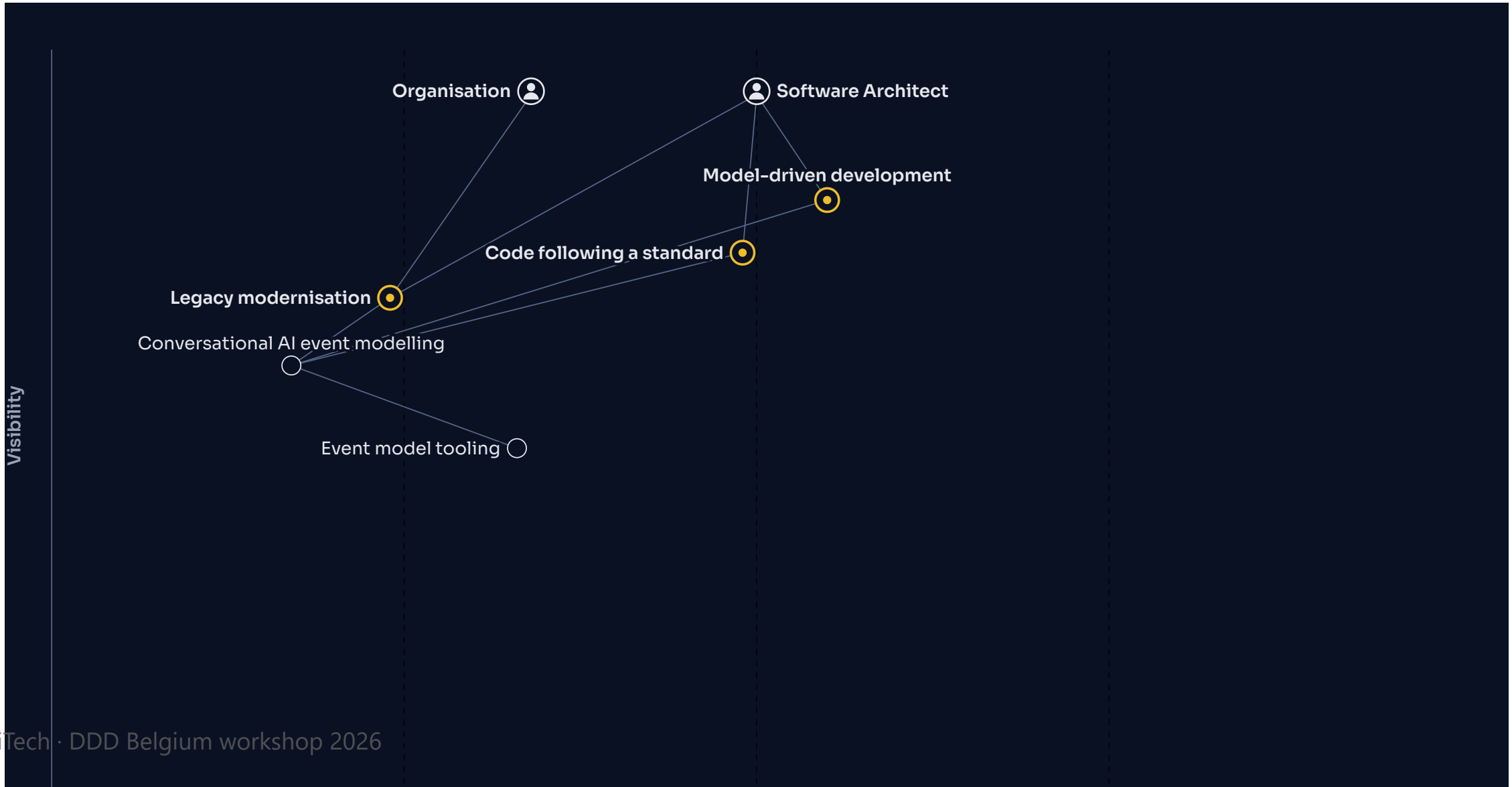
# EM tooling · 4 of 13 — the standard



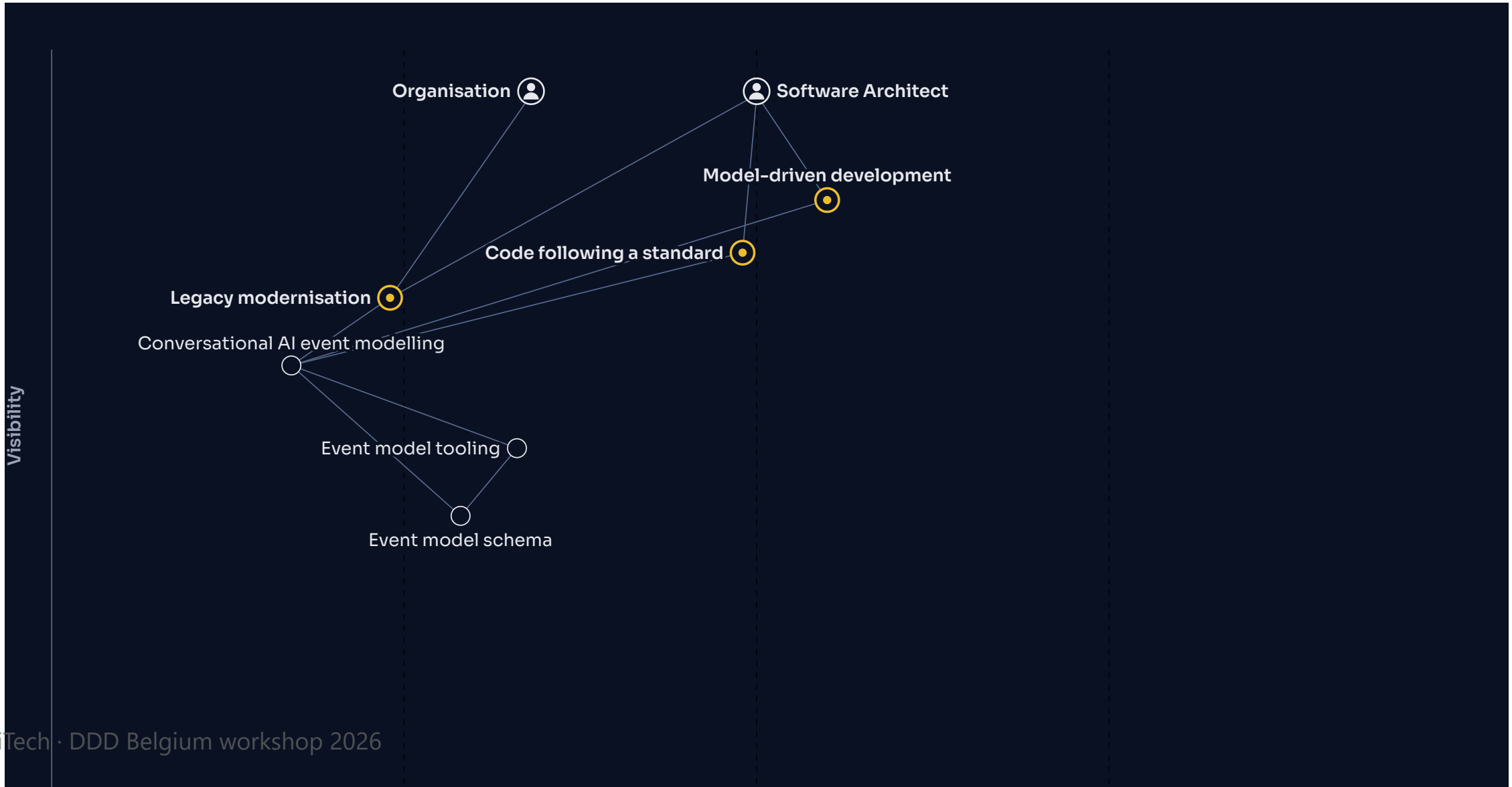
# EM tooling · 5 of 13 — the novelty



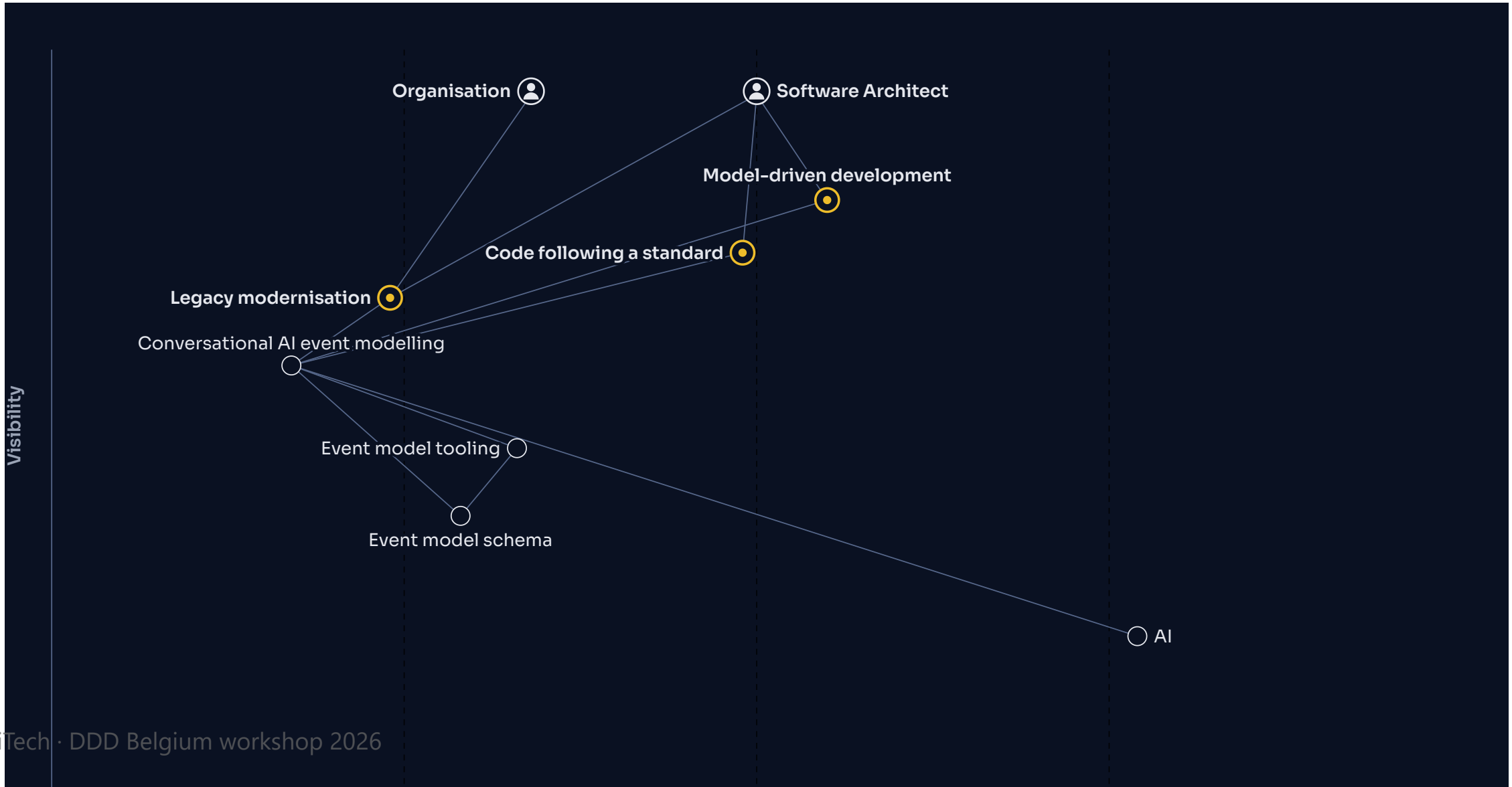
# EM tooling · 6 of 13 — the tooling



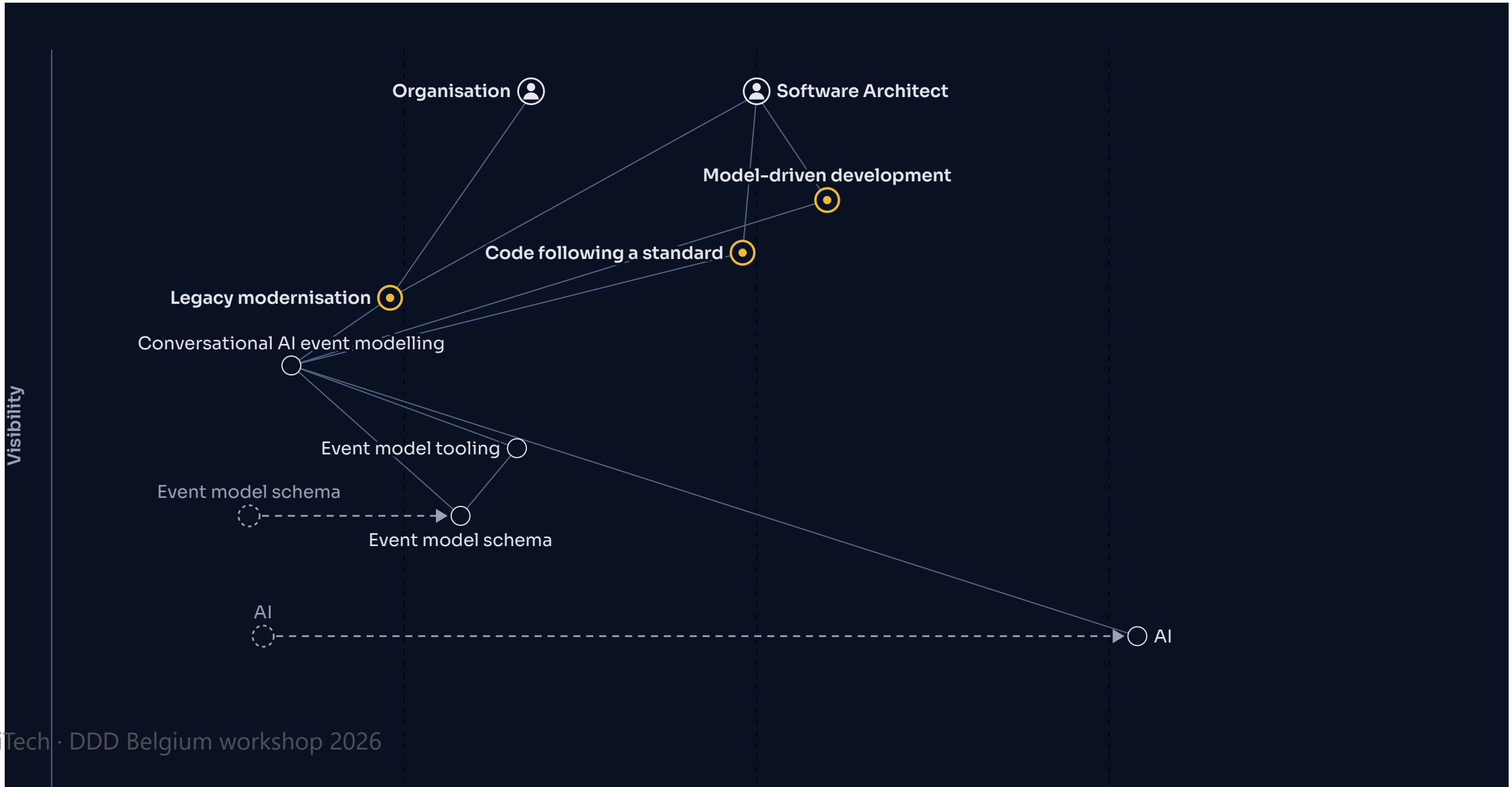
# EM tooling · 7 of 13 — the schema



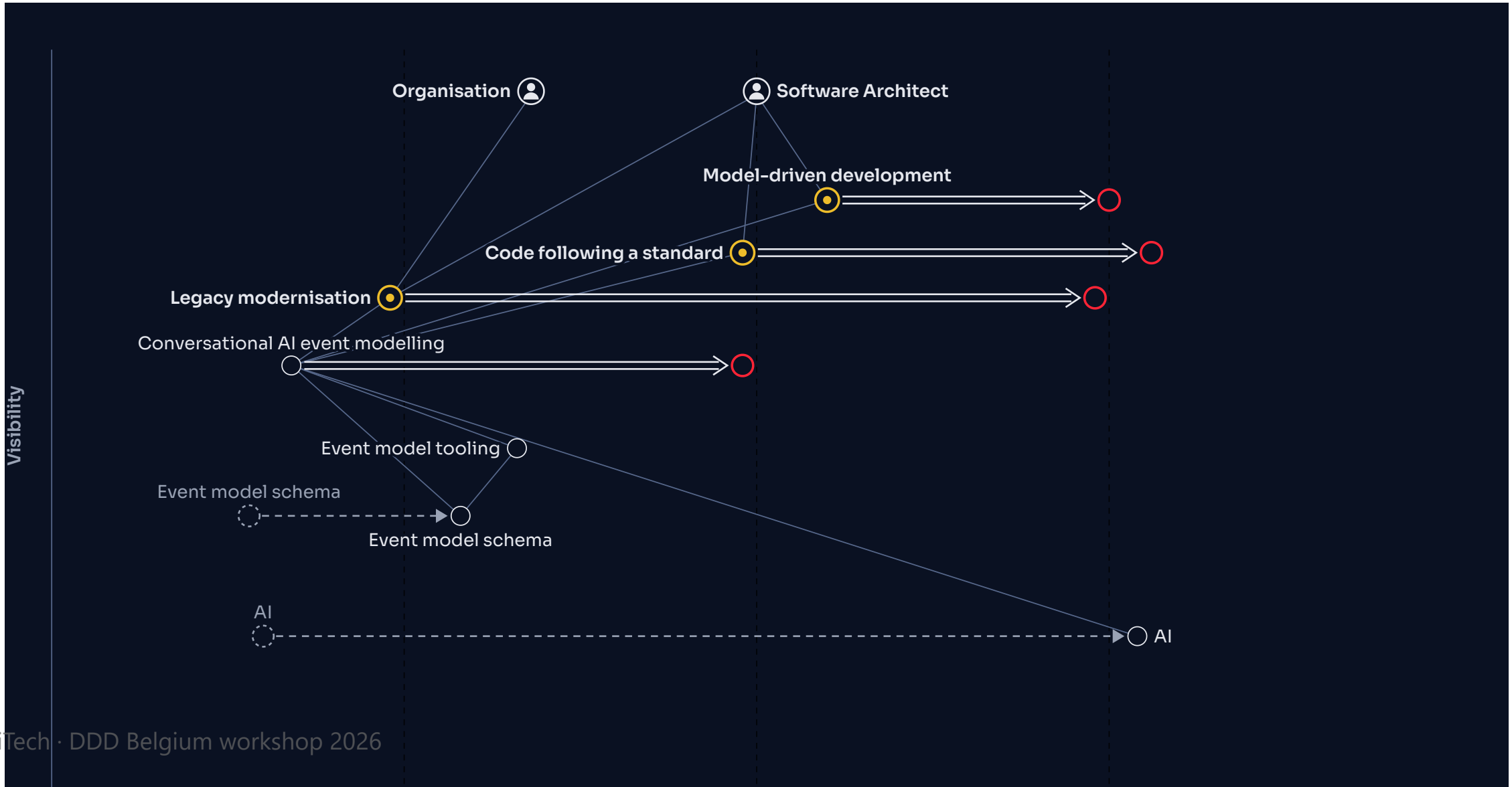
# EM tooling · 8 of 13 — the engine



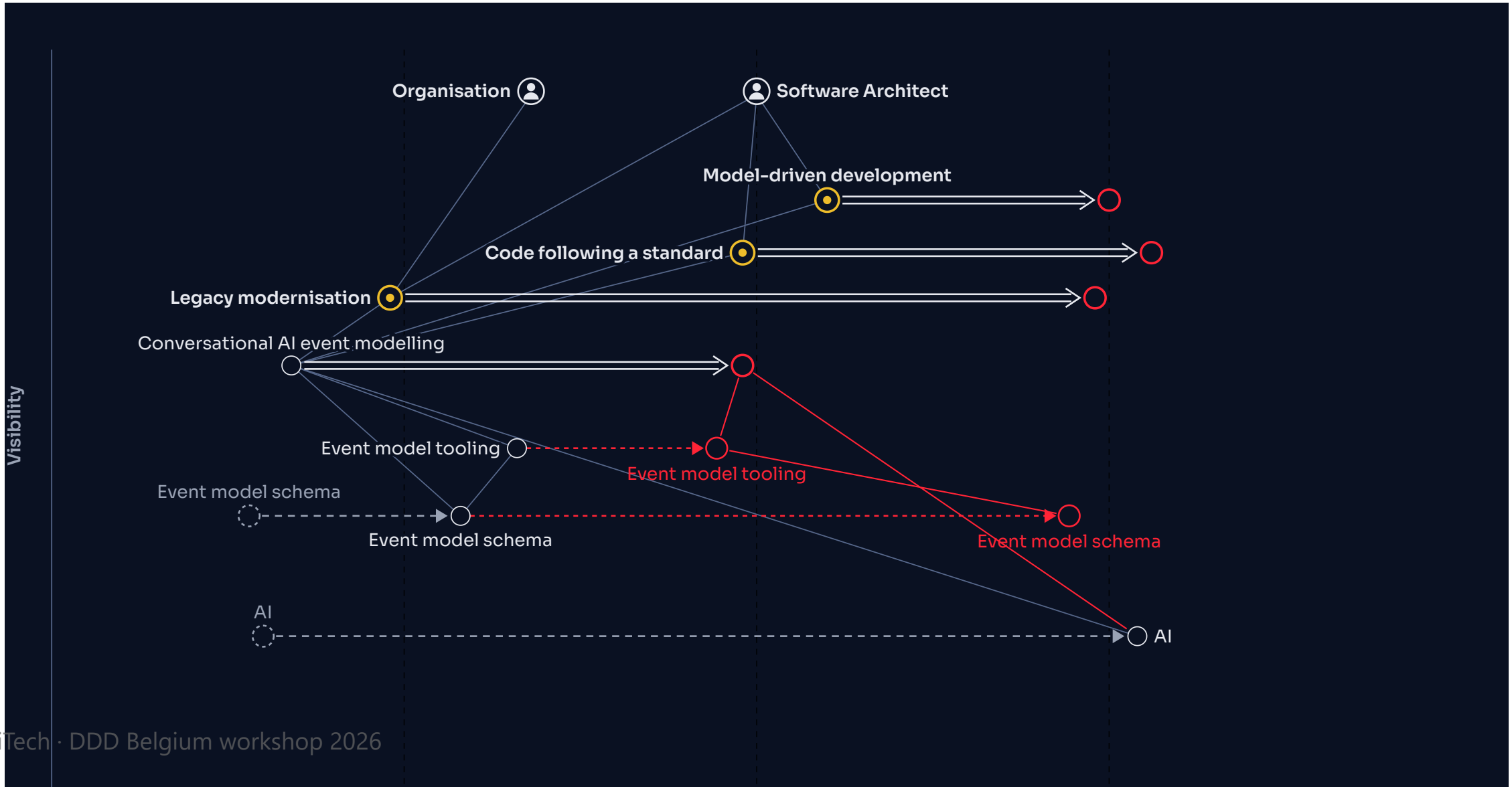
# EM tooling · 9 of 13 — the past



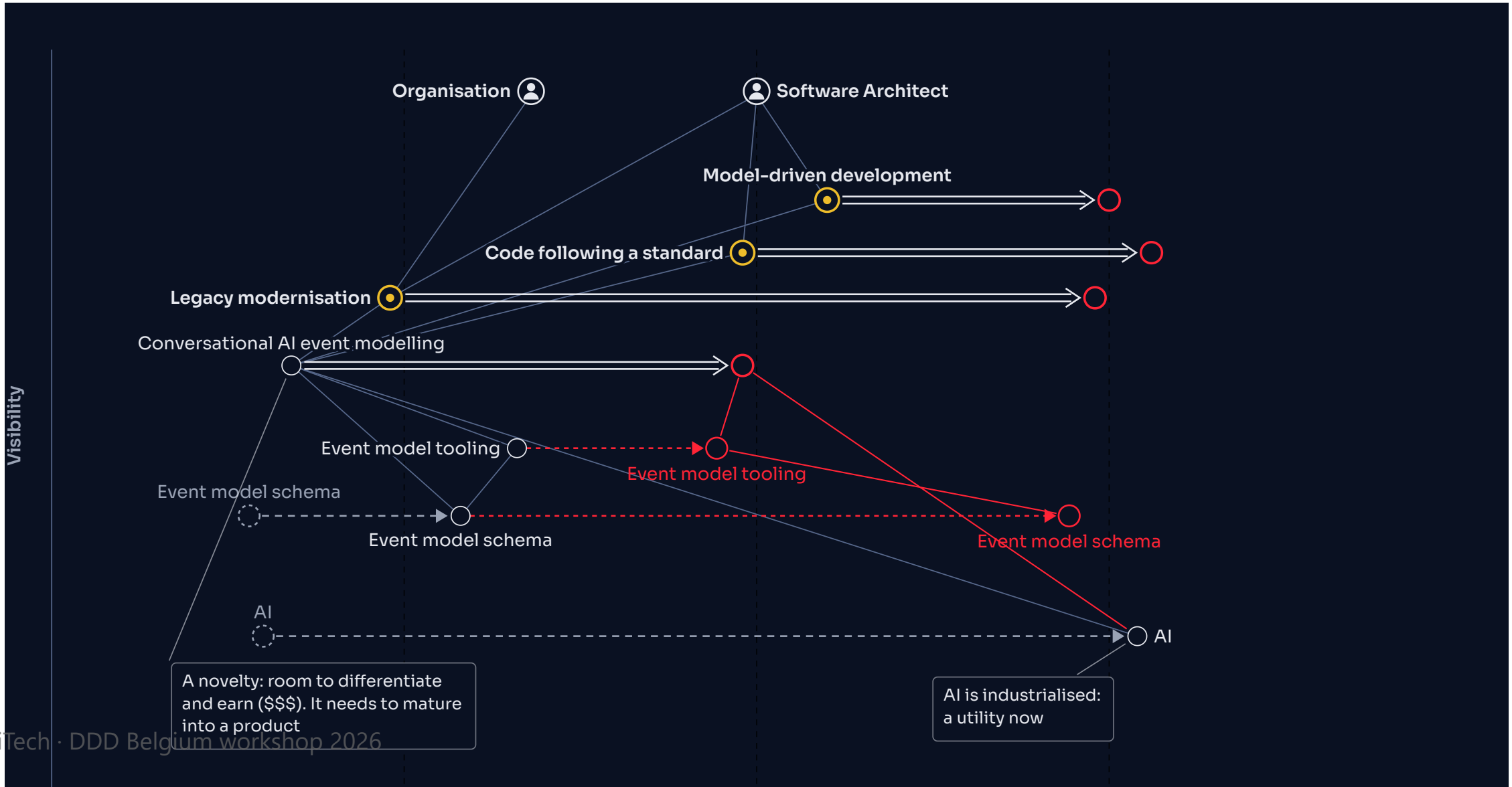
# EM tooling · 10 of 13 — our moves



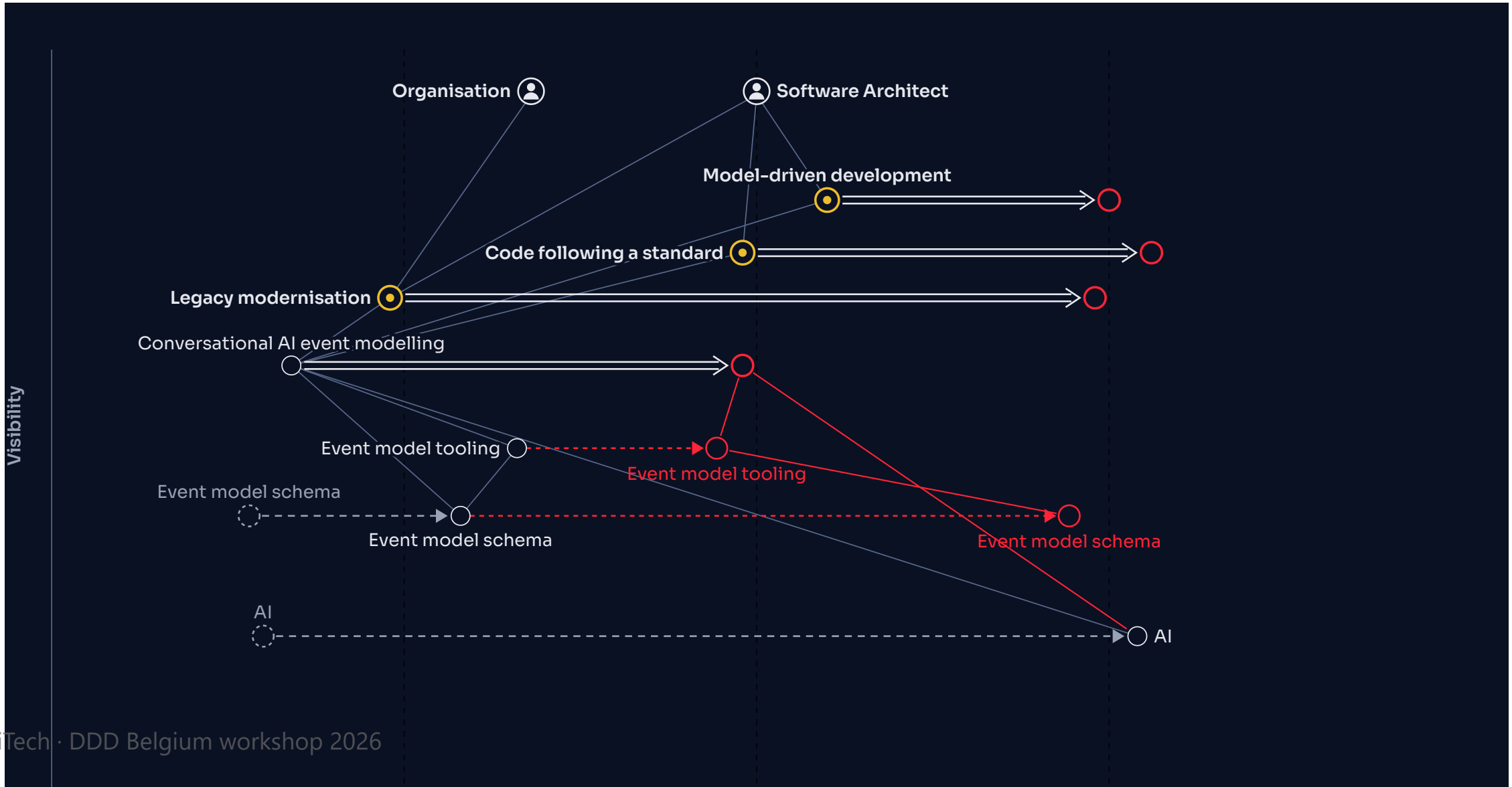
# EM tooling · 11 of 13 — the future



# EM tooling · 12 of 13 — comments on



# EM tooling · 13 of 13 — comments off

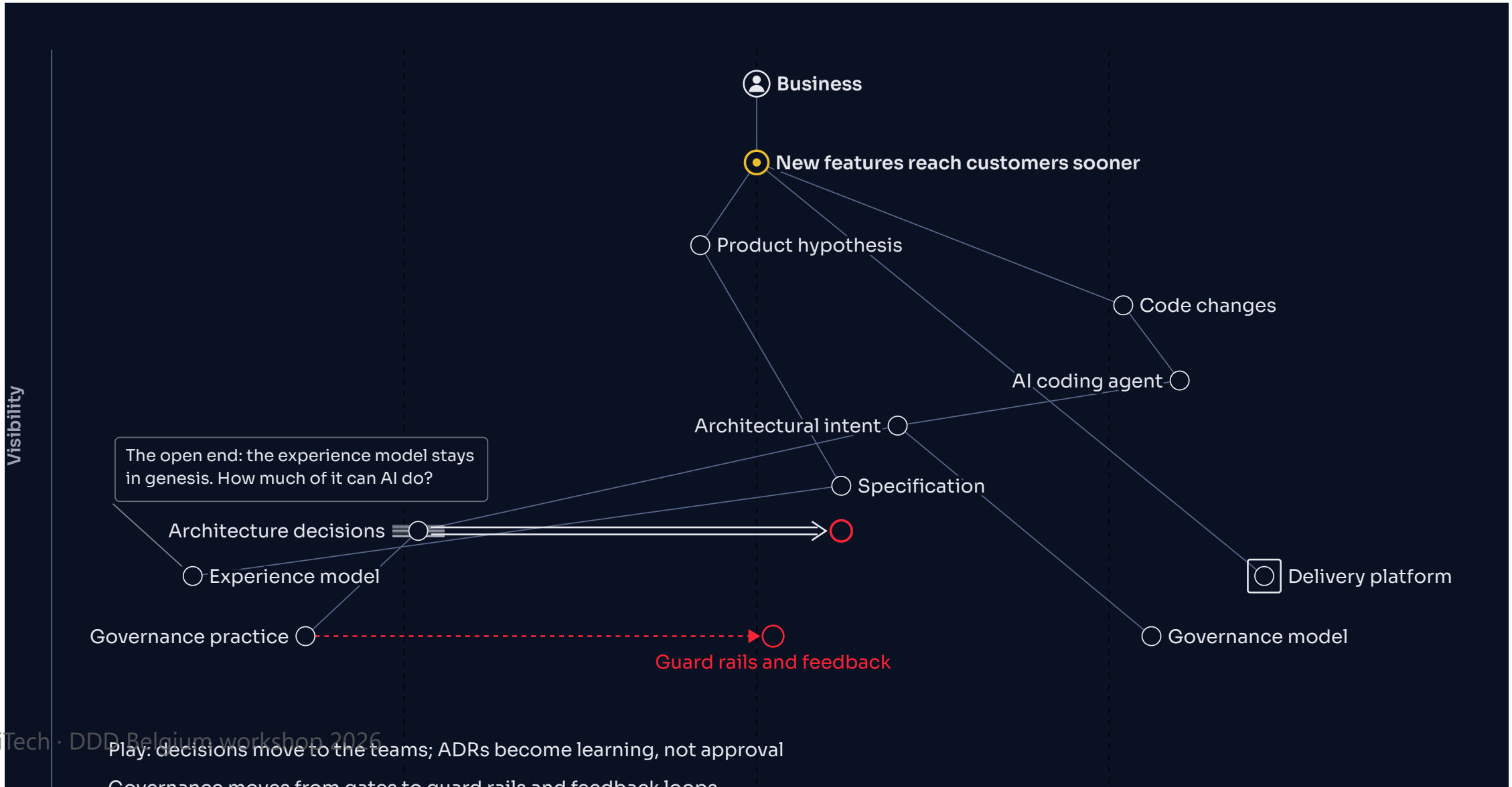


EM tooling

# Where this can go

- Evolve the schema — it becomes the standard
- Mature the tooling past custom built
- Treat AI as a utility — it already is
- The worth is in the conversation

# A real map · one way to map it




Play: decisions move to the teams; ADRs become learning, not approval

Governance moves from gates to guard rails and feedback loops

A real map · the same story in this week's research

# One day of research news, 17 October 2026

- Martin Rosén-Lidholm reads the agentic engineering papers every day and writes a digest
- About today's batch he wrote: "one of the strongest Software Civil Engineering signals we've seen. Multiple papers independently converge on the same thesis from different angles."
- The thesis: code generation is cheap, so the work moves to formal workflows, verified approvals and clear specifications
- That is the map you just drew. Next: his six pillars, then his five papers on one map

A real map · Martin Rosén-Lidholm's six pillars of an engineering discipline, 1 of 2

# Six pillars of Software Civil Engineering

| Pillar, from civil engineering | Software today | What Martin proposes |
|---|---|---|
| Formal specification and verification | user stories, ad hoc documents | the event model as the specification, checked by evaluations |
| Standardised material datasheets | no performance profiles for frameworks or infrastructure | provider profiles: known latency, throughput, limits |
| Building codes and norms | fragments: OWASP, well-architected frameworks | a policy model of rules that must always hold |

A real map · Martin Rosén-Lidholm's six pillars of an engineering discipline, 2 of 2

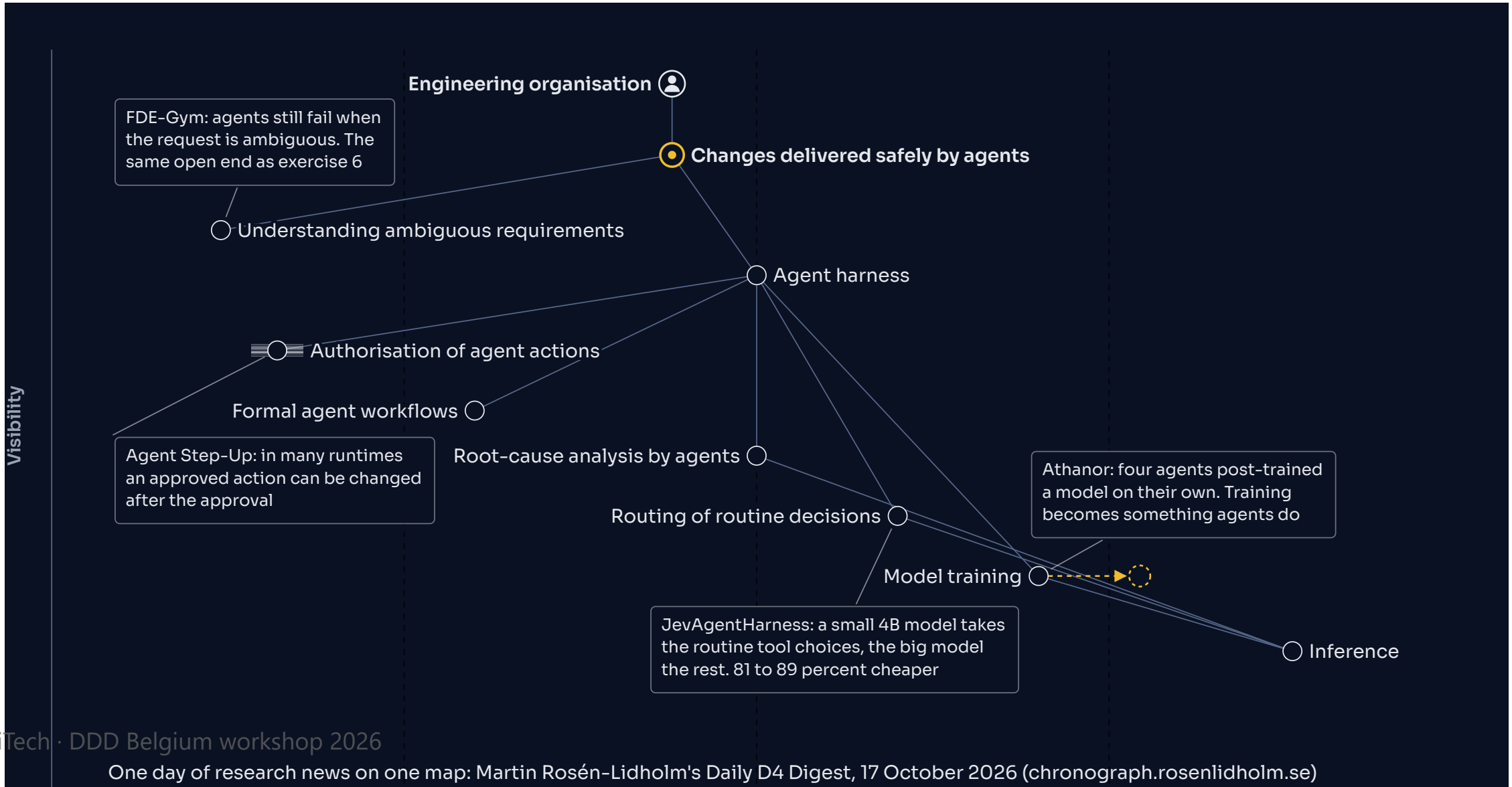
# Six pillars of Software Civil Engineering

| Pillar, from civil engineering | Software today | What Martin proposes |
|---|---|---|
| Simulation and modelling | almost none for application logic | run the behaviour as pure functions first |
| Professional licensure and liability | no personal accountability | an open gap, nothing proposed yet |
| Formal education and examination | varies, no proof of competence | specification as an examined skill |



Source: Martin Rosén-Lidholm, Software Civil Engineering: From Craft to Discipline, 22

# A real map · one day of research news on one map




One day of research news on one map: Martin Rosén-Lidholm's Daily D4 Digest, 17 October 2026 (chronograph.rosenlidholm.se)

07

# Wrap-up · the whole evening

Back together

# What did the maps say?

- One insight per table, one sentence
- Where did the groups disagree?
- What moves right first?

08

# Extra exercise, if there is time · your own domain

**Exercise · 10 min**

# Draw one chain

- Pick a real user and one real need
- Add three things the need depends on
- Stack them: most visible at the top
- Place each on the evolution axis

**Exercise · 10 min**

# Add the truth

- Mark what you would buy tomorrow
- Mark what hurts to change
- Mark one thing that should move right
- Find the constraint. Focus your map on it, and remove what is not strictly needed

**Exercise · 15 min**

# Swap and stress-test

- Swap maps with another pair
- Ask: what did we leave off the chain?
- Ask: where would we place this differently?
- One decision per map: buy, build or evolve

09

# Extra, if there is time · buy vs build

**Extra, if there is time · buy vs build**

# Kettles and whiteboards

- The kettle: a product. Buy it, never build
- The whiteboard: rent, don't make
- Build only where customers pay you to differ
- If you build a kettle, you are the map's problem

10

# Extra, if there is time · inertia

**Extra, if there is time · inertia**

# What resists moving right

- Good facilitation is a craft. It is hard to turn into a product
- When something won't move, ask who would lose if it did
- Turning a craft into a product can kill it, or let it scale
- Help it evolve, or work around it

11

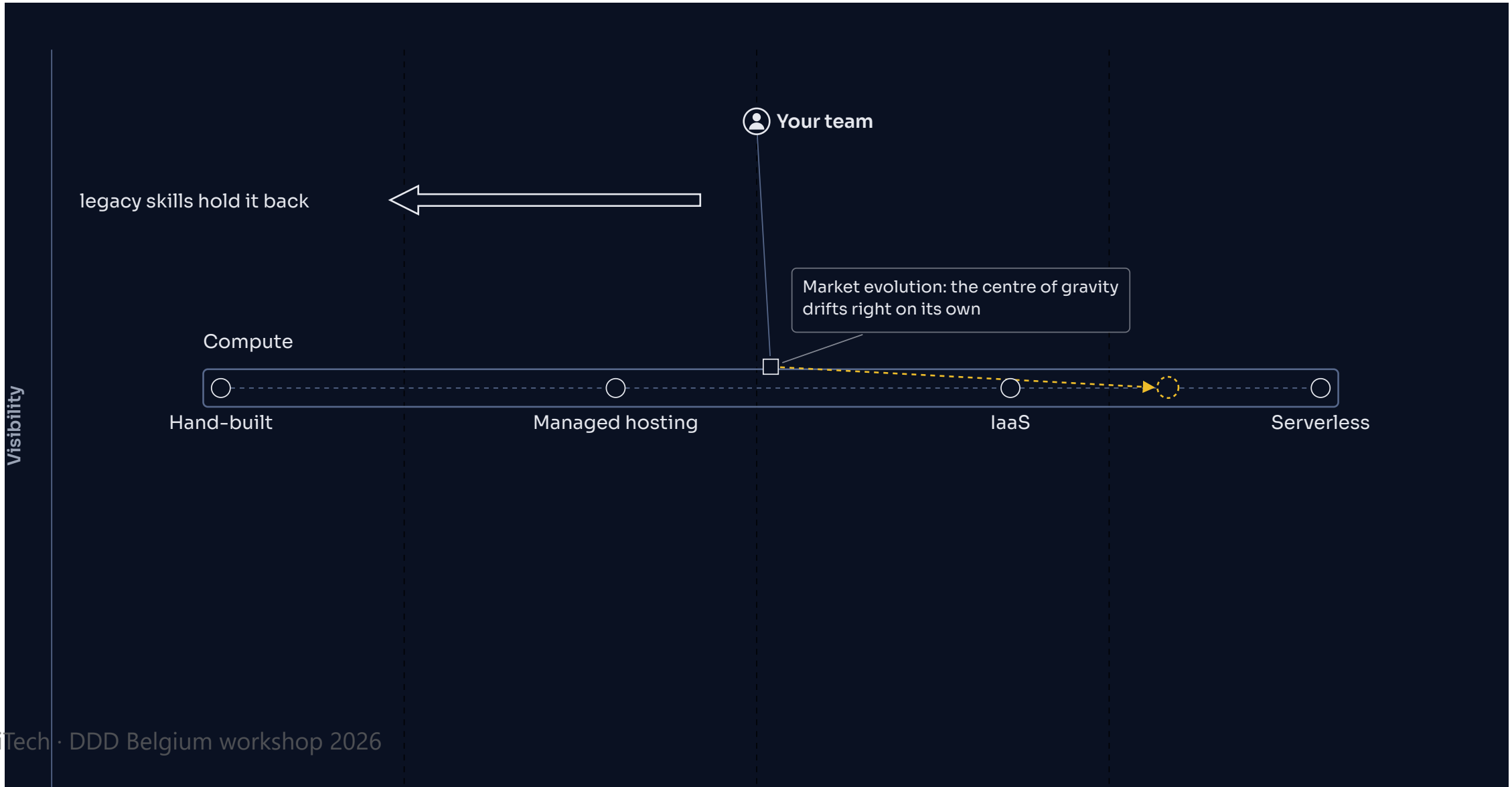
# Extra, if there is time · the pipeline

Extra, if there is time · the pipeline

# Several ways to do one thing

- One need, several ways to meet it
- Each way sits at its own stage of evolution
- Each way evolves at its own pace
- We can speed a move up: open source, standards
- Or slow it down: patents, lock-in

# The pipeline — compute



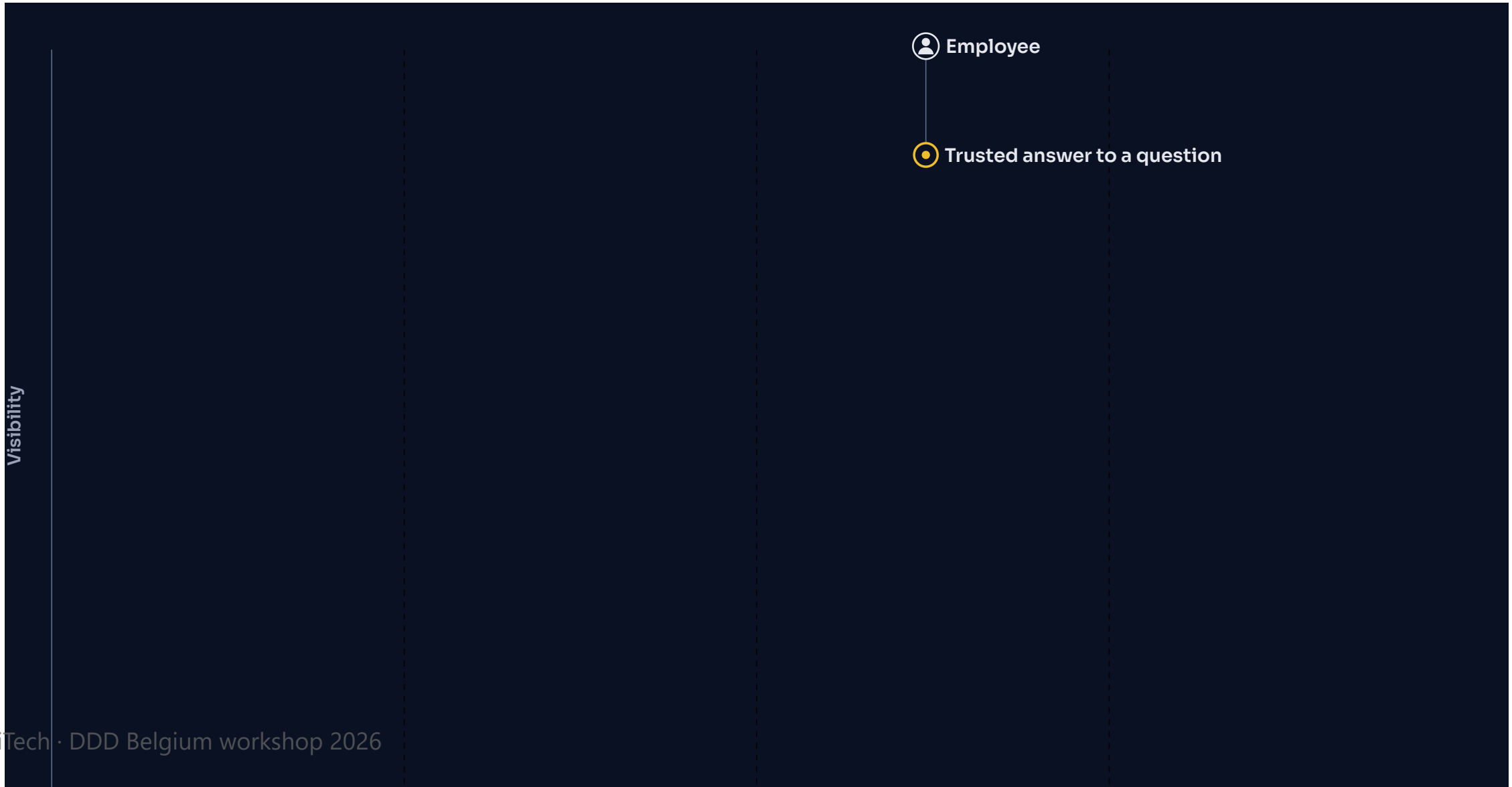
12

# Extra, if there is time · knowledge base

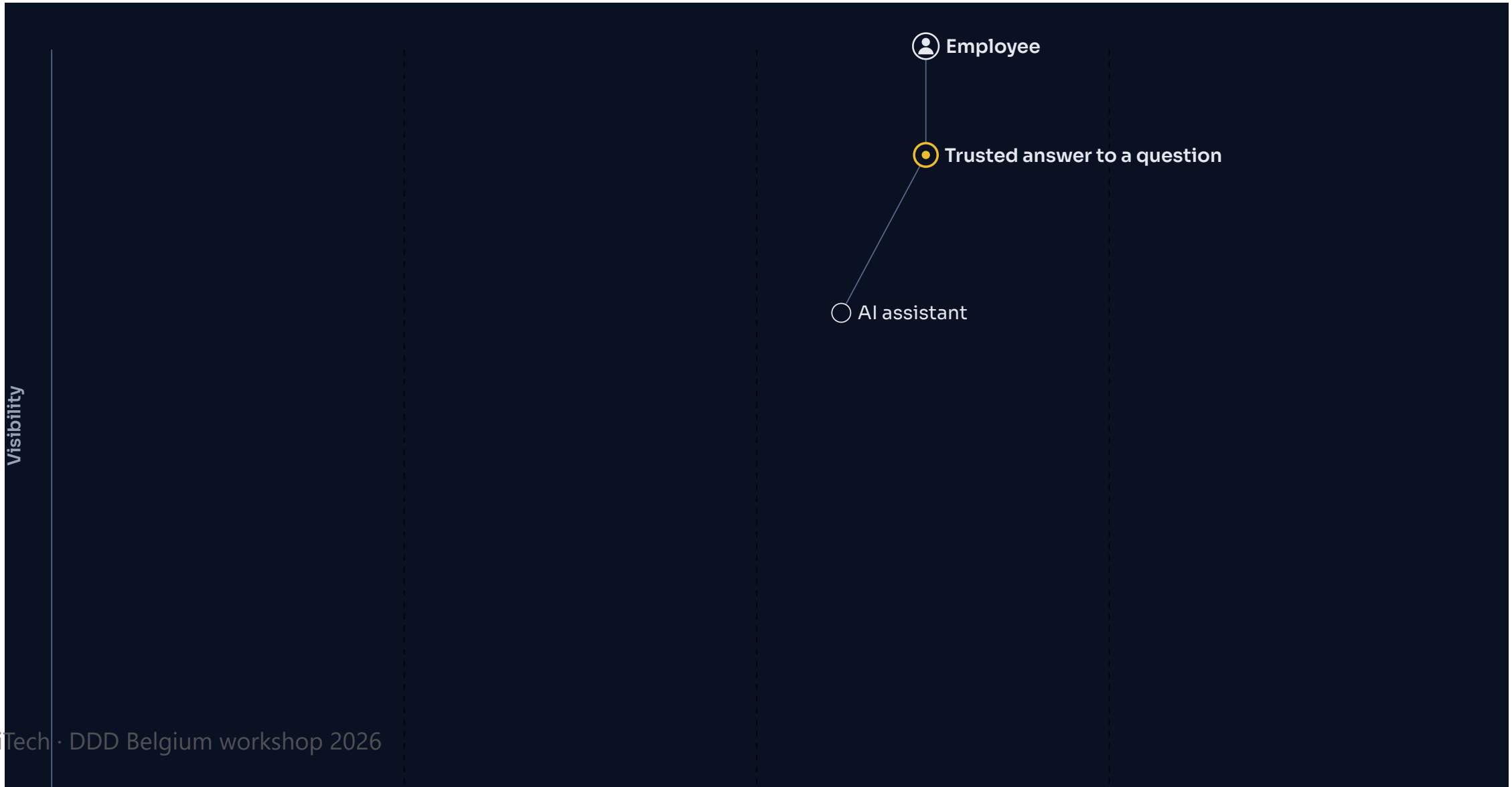
# Knowledge base · 1 of 9 — the user



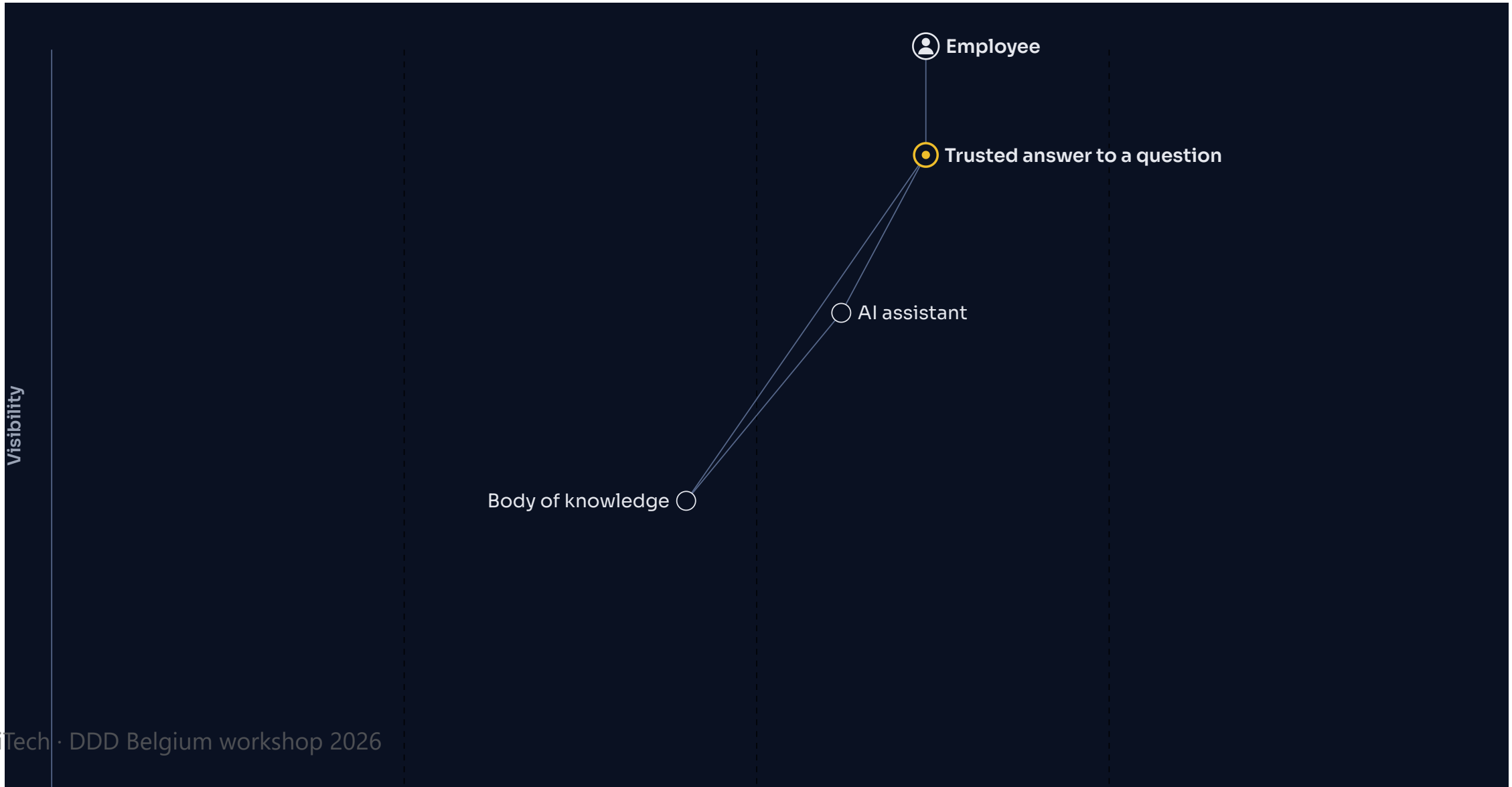
# Knowledge base · 2 of 9 — the need



# Knowledge base · 3 of 9 — the assistant



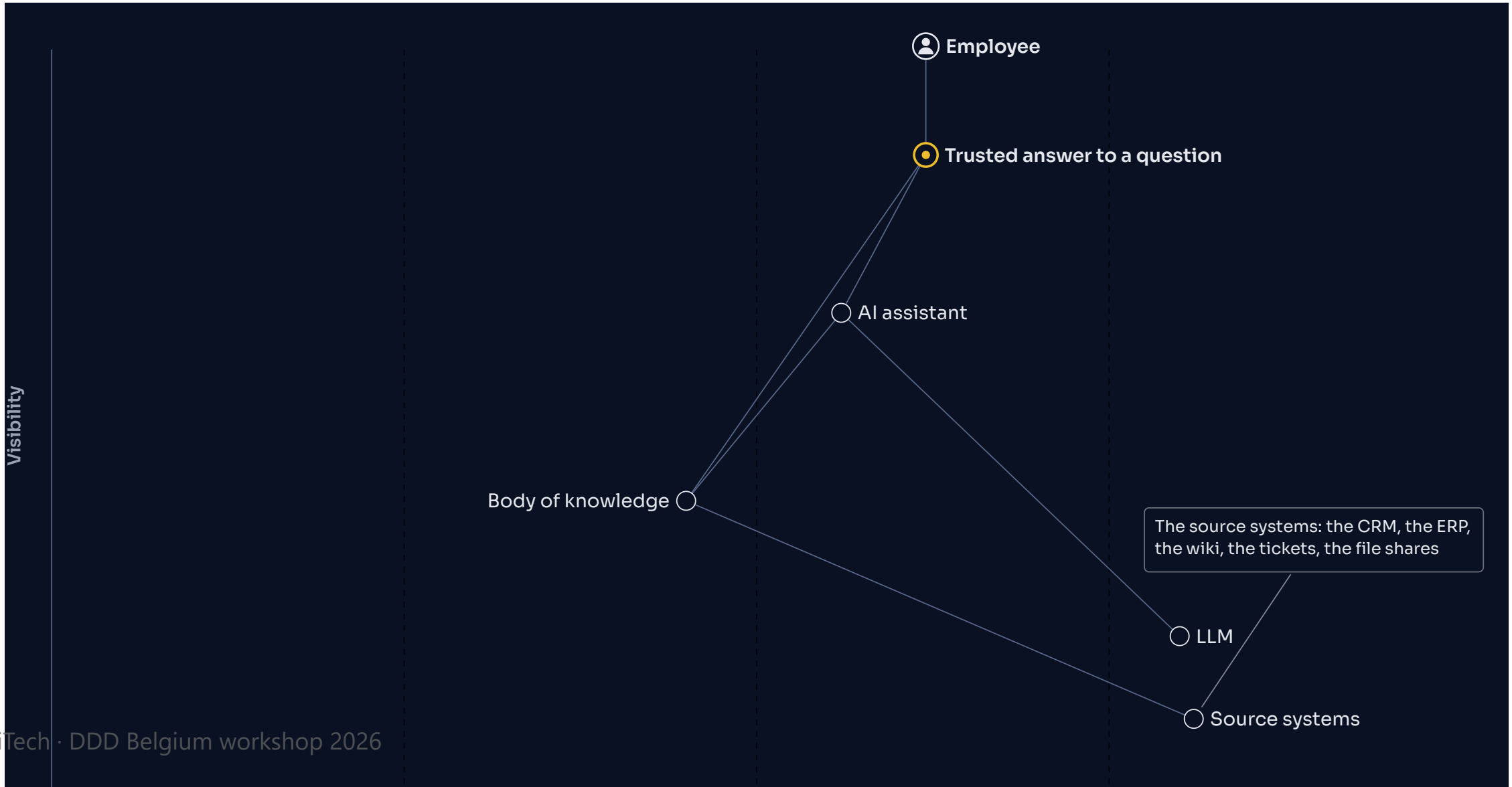
# Knowledge base · 4 of 9 — the knowledge



# Knowledge base · 5 of 9 — the sources



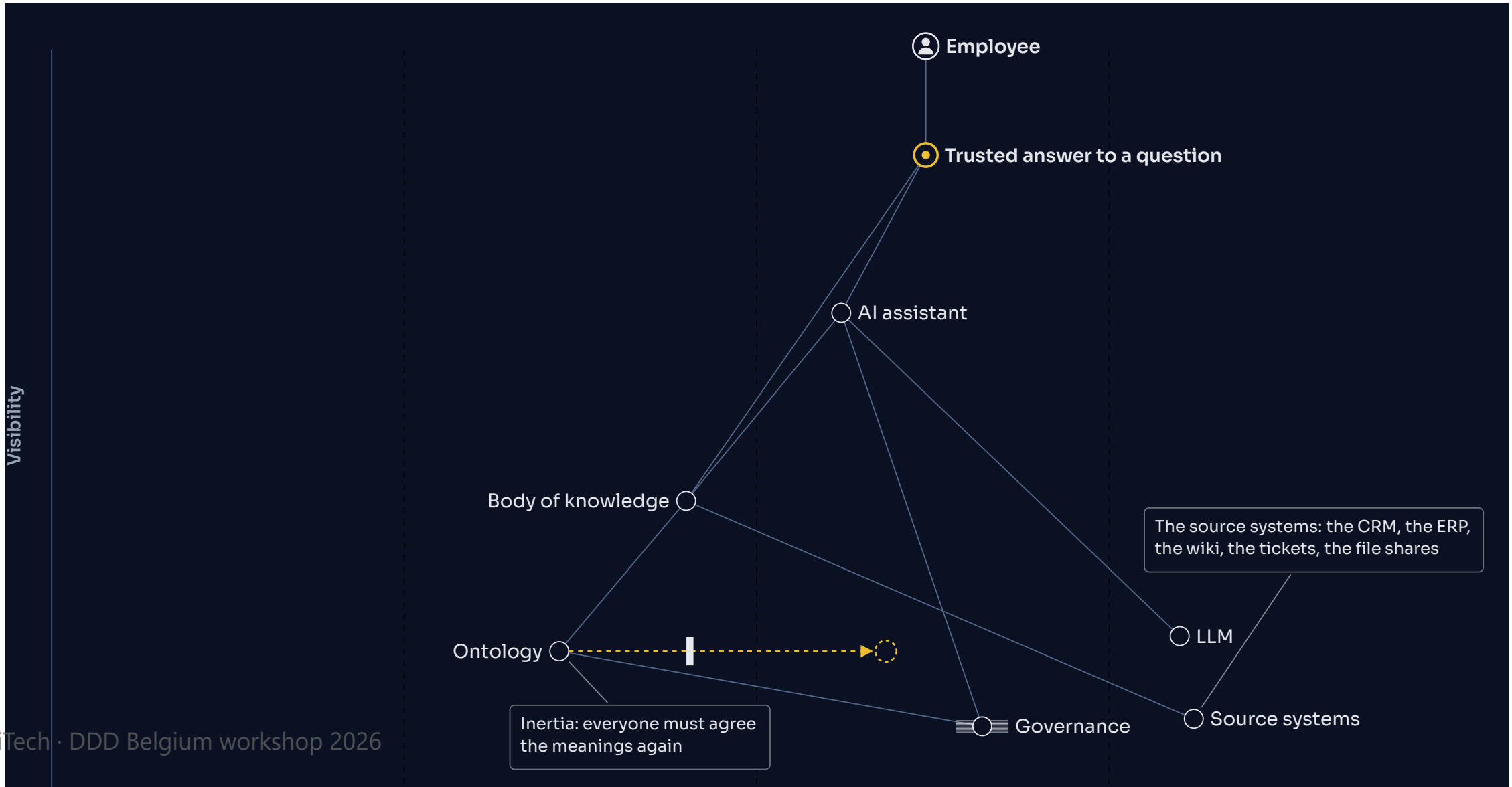
# Knowledge base · 6 of 9 — the model



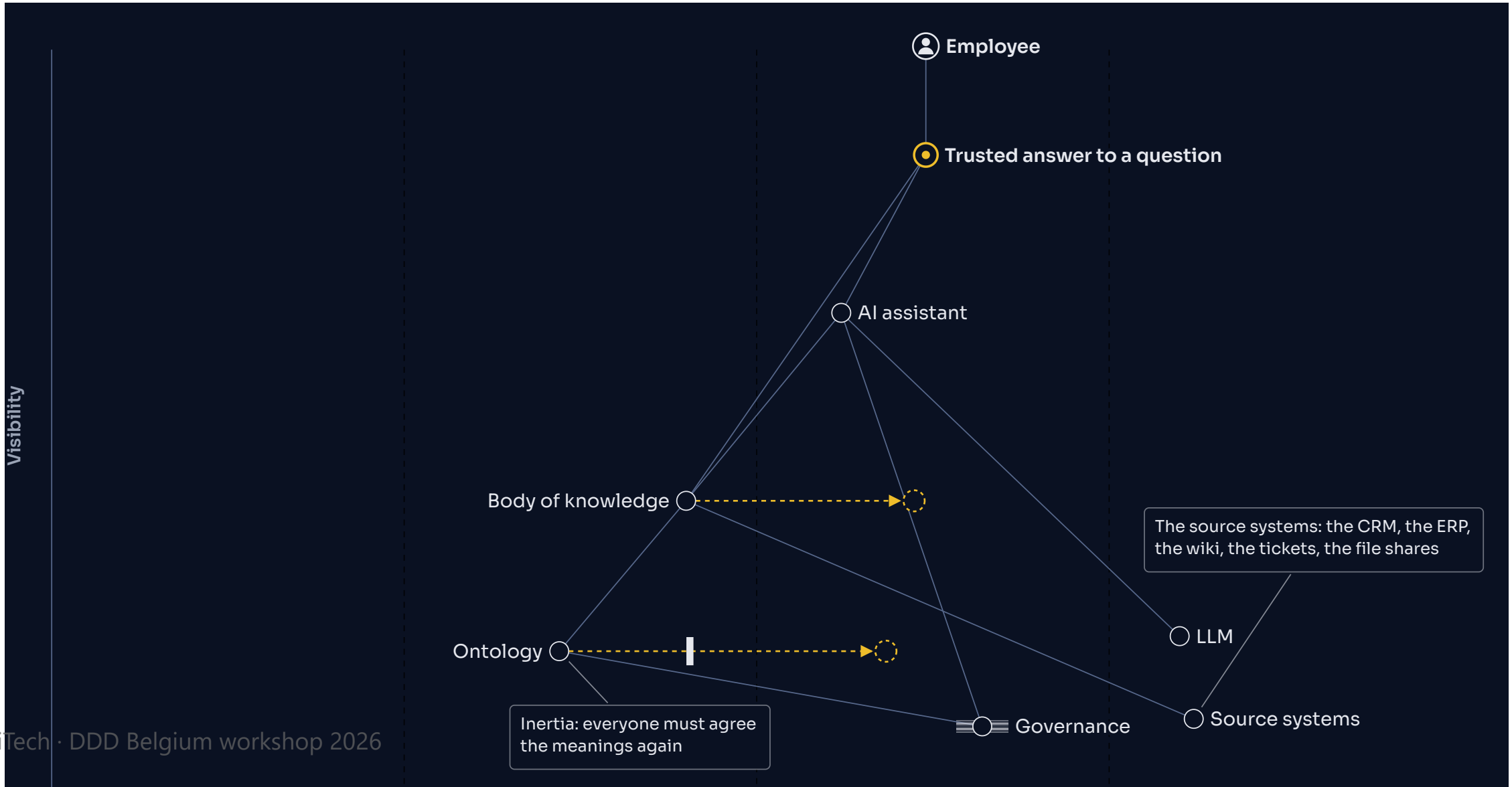
# Knowledge base · 7 of 9 — the trust



# Knowledge base · 8 of 9 — the snag



# Knowledge base · 9 of 9 — the move



Knowledge base

# Trust is the product

- Buy the assistant, the model, the plumbing
- Build the body of knowledge — where you differ
- The ontology resists: everyone must agree the meanings again
- The constraint sits at governance — map there

13

# Appendix · cheat sheet

Appendix · the evolution axis

# Evolution: four sets of stage labels

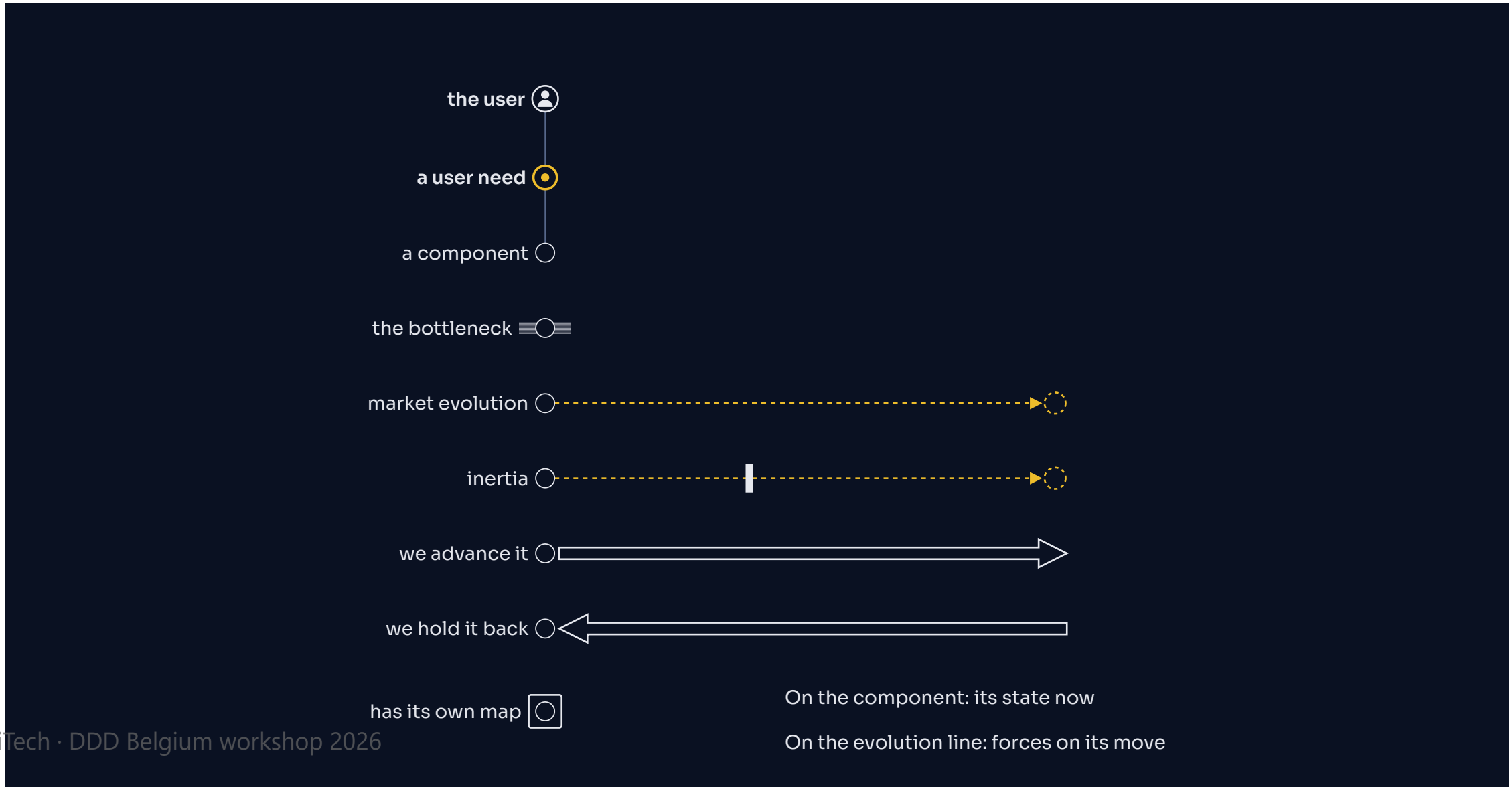
| Activities | Genesis | Custom built | Product (+ rental) | Commodity (+ utility) |
|---|---|---|---|---|
| Practices | Novel | Emerging | Good | Best |
| Data | Unmodelled | Divergent | Convergent | Modelled |
| Knowledge | Concept | Hypothesis | Theory | Accepted |

Evolution runs left to right, as on the map.

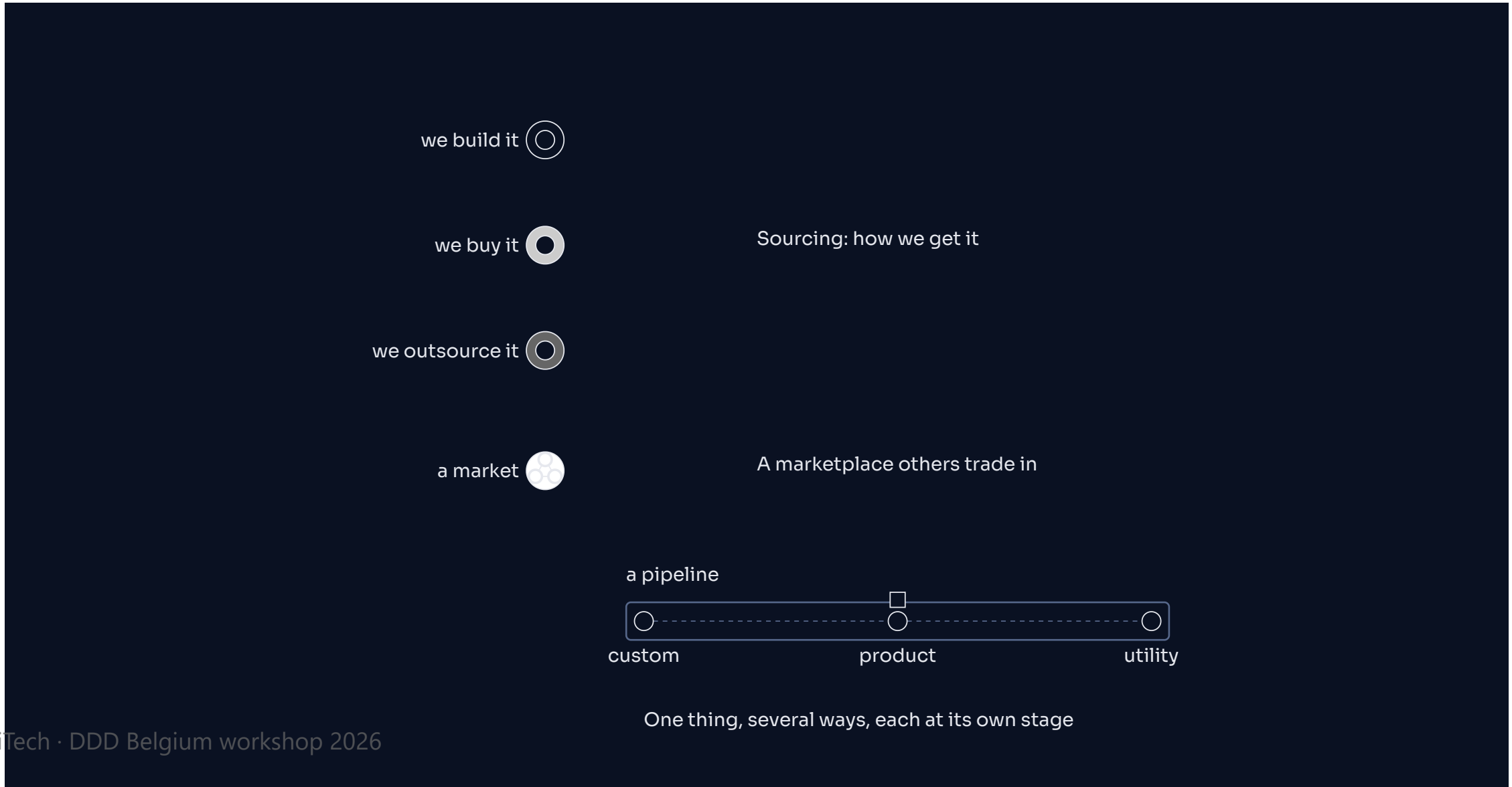
# Users and needs: start from the user

- The anchor is a user: a person, a team or an organisation
- Their needs sit just below, in their words
- Everything else is there to meet a need
- Ask "how?" to go down the chain, "why?" to go up
- Several users? Where their needs differ, look closer

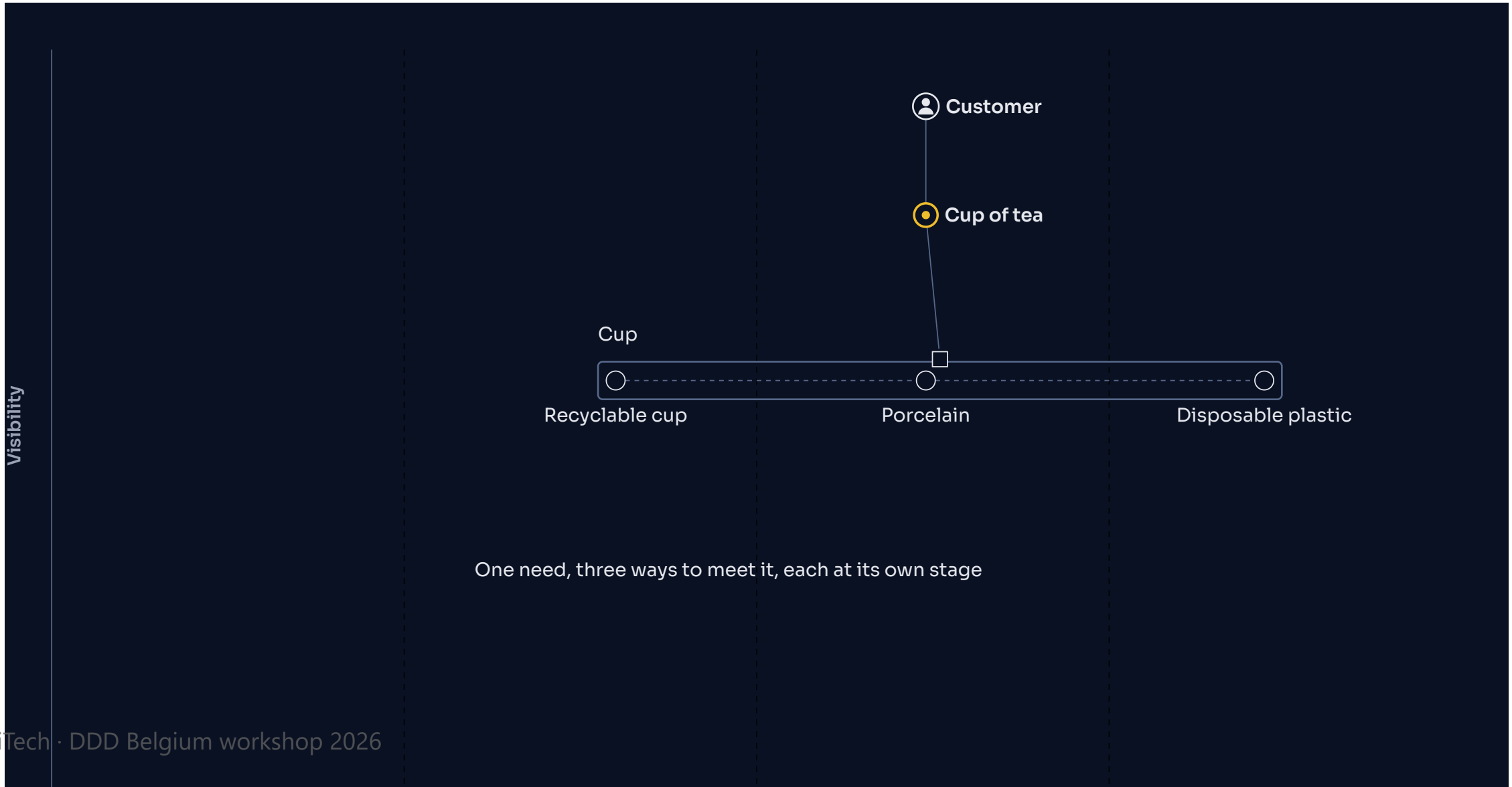
# Appendix · legend



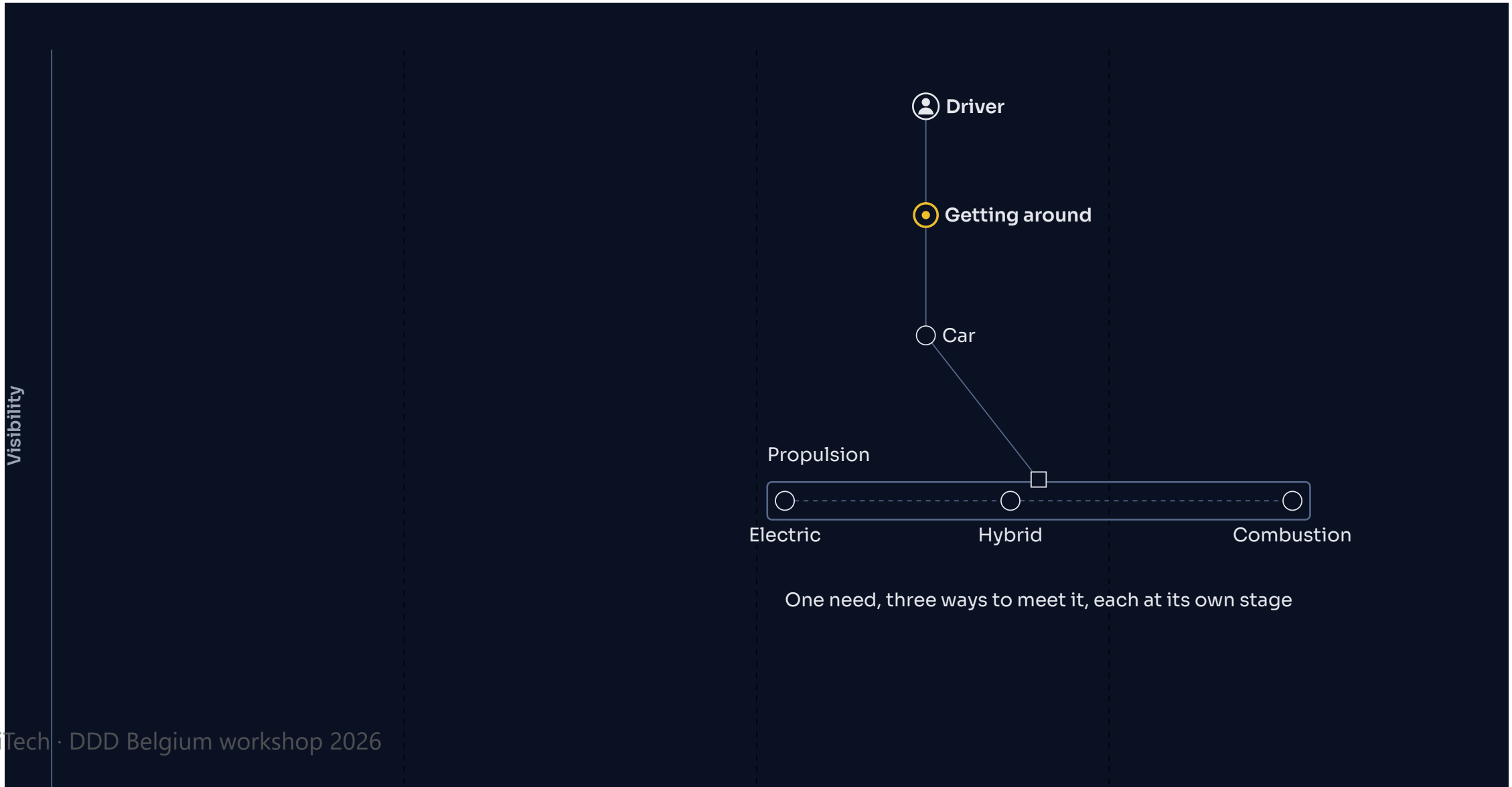
# Appendix · the rest of the legend



# Appendix · pipeline — the cup



# Appendix · pipeline — cars



Appendix · submaps

# Submaps: a map inside a component

- A square around a component: it has its own map
- Keep the top map to the decision at hand
- Open a submap only for the part your question is about
- Inside, the same rules: users, needs, evolution

# Constraint vs inertia

- Constraint: a capacity limit. Work piles up in front of it
- Inertia: the old way is kept although a better one exists
- Constraint sits on the component: the striped bar
- Inertia sits on its evolution line: the solid bar
- Ask who would lose if it moved: that is the inertia

# Goldratt’s Theory of Constraints: five steps

- Identify the constraint
- Exploit it: get the most out of what is there
- Subordinate everything else to it
- Elevate it: add capacity, only now
- Repeat: the constraint has moved

Appendix · climate · after Simon Wardley

# Wardley’s climate: patterns you can't stop

- Everything evolves through supply and demand competition
- Characteristics change as components evolve
- No choice over evolution
- Practices co-evolve with the activities they support
- Efficiency enables innovation
- Higher-order systems create new sources of worth
- Past success breeds inertia

Appendix · doctrine · after Simon Wardley

# Wardley’s doctrine: principles that work anywhere

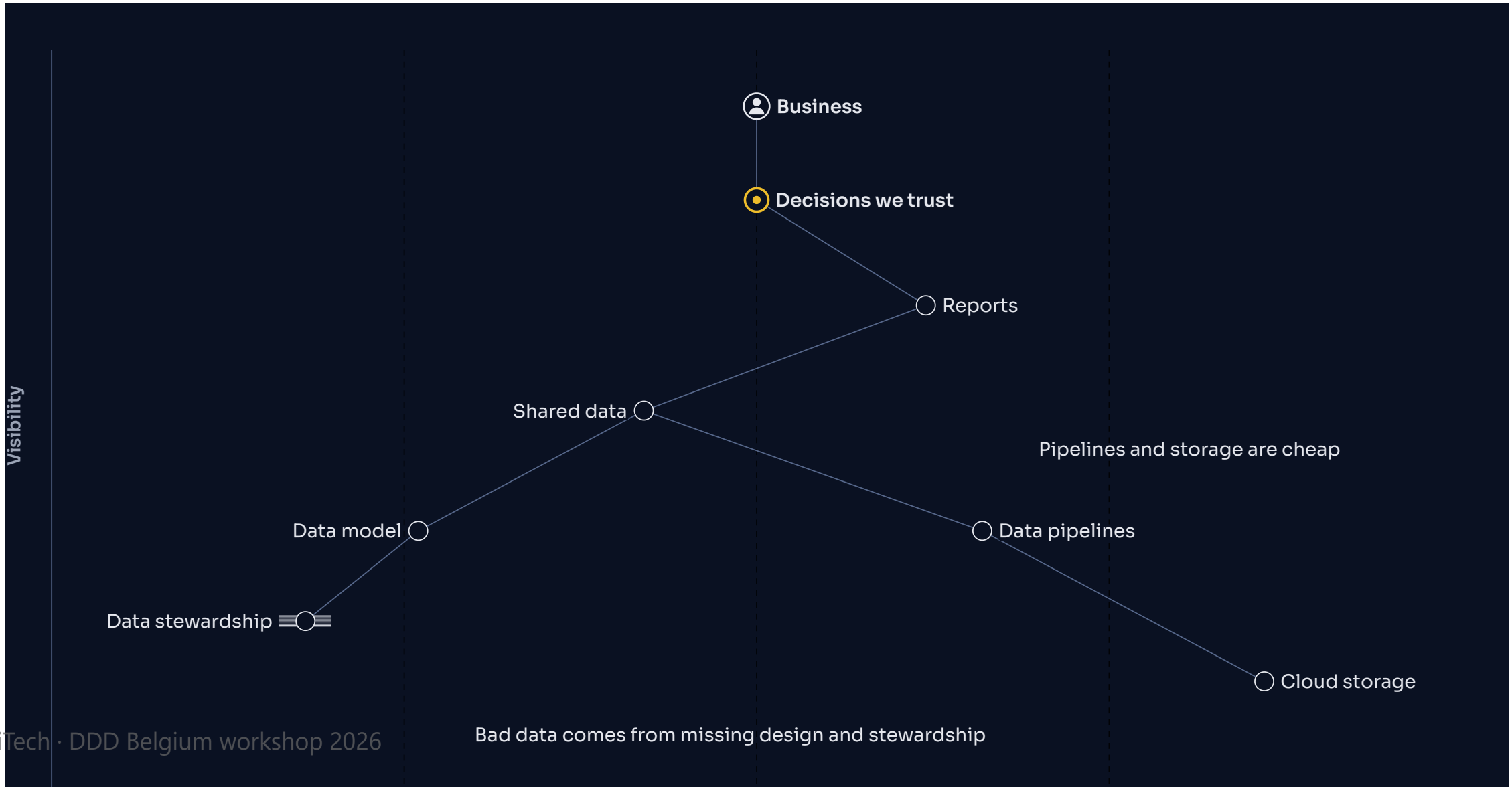
- Focus on user needs
- Know your users
- Use a common language
- Challenge assumptions
- Remove duplication and bias
- Use appropriate methods
- Think small: small teams, small parts

Appendix · gameplay · after Simon Wardley

# Wardley’s gameplay: moves you can choose

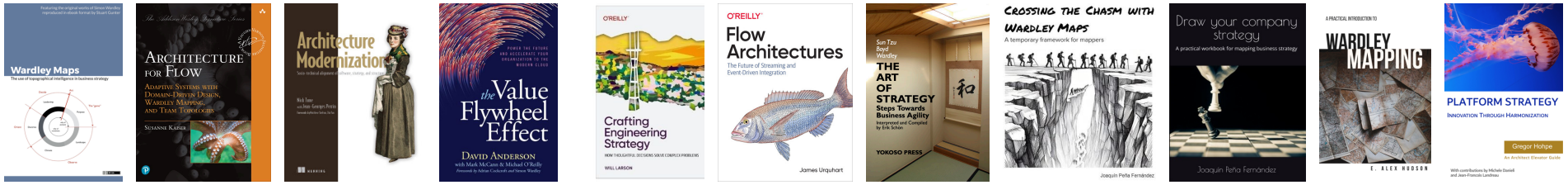
- Open approaches: commoditise a layer someone else owns
- ILC: innovate, leverage, commoditise
- Standards game: set the rules others build on
- Exploit inertia: move where a rival can't follow
- Co-creation: build it with users, not for them

# Appendix · a parallel case — data (draft)



## Reading · books that use Wardley maps

# If you want to read on




- First: Simon Wardley's own book, free to read at medium.com/wardleymaps and on Leanpub
- All the others map with Wardley maps. Kaiser and Tune write for a DDD audience
- Erik Schön's annex to The Art of Strategy collects 168 Wardley maps from 90 mappers, free to read. One of them is my logistics map

GALAXITECH

Thank you

# Questions

galaxi.tech · cj@galaxi.tech

linkedin.com/in/cjjohansen

`github.com/cjjohansen/DDD-Belgium-Wardley-Map-2026`

Book a workshop